

목차

CHAPTER 1. 리플렉션	1
1.1 스프링의 실행 원리	3
1.1.1 프레임워크	3
1.1.2 제어의 역전	4
1.1.3 리플렉션	5
1.2 리플렉션 구현	6
1.2.1 if-else 라우팅	6
1.2.2 어노테이션과 리플렉션	8
1.2.3 컴포넌트 스캔	10
1.3 객체 생성과 연결	14
1.3.1 빈과 컨테이너	14
1.3.2 의존성 주입	14
1.4 스프링의 요청 흐름	15
1.5 학습 지도	16
CHAPTER 2. 게시판 CRUD	18
2.1 REST API	21
2.1.1 REST API 탄생 배경	21
2.1.2 JSON	22
2.1.3 메서드와 주소	23
2.1.4 요청과 응답	24
2.2 프로젝트 생성과 구조	25
2.2.1 프로젝트 생성	25
2.3 하이버네이트	29
2.4 데이터베이스와 엔티티	30
2.4.1 데이터베이스 설정	30
2.4.2 엔티티	31
2.4.3 더미 데이터	32
2.5 3계층 아키텍처	34
2.6 공통 응답	36
2.7 리포지토리(Repository)	36
2.7.1 게시글 한 건 조회	37
2.7.2 게시글 전체 조회	37
2.7.3 게시글 저장	38
2.7.4 게시글 수정	38
2.7.5 게시글 삭제	39
2.8 JPQL	39
2.9 영속성 컨텍스트	40
2.9.1 캐싱	40
2.9.2 쓰기 지연	41
2.9.3 더티체킹	42
2.10 단위 테스트	43
2.10.1 given-when-then 패턴	44
2.10.2 리포지토리 테스트 작성	45
2.11 게시글 목록	50
2.11.1 서비스	50
2.11.2 컨트롤러	51
2.11.3 API 요청	51

2.12 게시물 상세	53
2.12.1 서비스	53
2.12.2 컨트롤러	53
2.13 게시물 추가	54
2.13.1 서비스	54
2.13.2 컨트롤러	55
2.14 게시물 수정	56
2.14.1 서비스	56
2.14.2 컨트롤러	57
2.15 게시물 삭제	58
2.15.1 서비스	58
2.15.2 컨트롤러	58
챕터 3. 예외 처리와 DTO	61
3.1 요청 DTO	63
3.1.1 DTO	63
3.1.2 엔티티 생성자	64
3.1.3 요청 DTO 만들기	65
3.2 응답 DTO	66
3.3 JpaRepository 적용	67
3.4 예외 처리 추가	68
3.4.1 예외 처리 흐름	68
3.4.2 커스텀 예외 만들기	69
3.4.3 전역 예외 처리	70
3.5 서비스와 컨트롤러에 적용	71
3.5.1 서비스	71
3.5.2 컨트롤러	72
챕터 4. 인증과 인가	75
4.1 인증과 인가	77
4.2 JWT	78
4.2.1 토큰 인증	78
4.2.2 상태 유지와 무상태	79
4.2.3 JWT의 구조	81
4.3 테이블 관계	82
4.3.1 1:N 관계	82
4.3.2 N:M 관계	83
4.4 테이블 만들기	85
4.4.1 회원 엔티티	85
4.4.2 게시물에 작성자 연결	86
4.4.3 더미 데이터	86
4.5 core 폴더 살펴보기	87
4.5.1 JwtUtil	87
4.5.2 JwtProvider	88
4.5.3 인증 필터	89
4.6 회원가입	90
4.6.1 리포지토리	90
4.6.2 요청 DTO	90
4.6.3 응답 DTO	91
4.6.4 서비스	91
4.6.5 컨트롤러	92
4.7 로그인	93

4.7.1 요청 DTO	94
4.7.2 서비스	94
4.7.3 컨트롤러	94
4.8 게시글 상세	95
4.8.1 JPQL 조인문	96
4.8.2 리포지토리	97
4.8.3 응답 DTO	97
4.8.4 서비스	98
4.8.5 컨트롤러	98
4.9 게시글 추가	100
4.9.1 요청 DTO	100
4.9.2 서비스	101
4.9.3 컨트롤러	101
4.10 게시글 수정	102
4.10.1 서비스	102
4.10.2 컨트롤러	103
4.11 게시글 삭제	105
4.11.1 서비스	105
4.11.2 컨트롤러	105
챕터 5. 댓글과 JPA 심화	107
5.1 양방향 매핑	109
5.1.1 댓글 엔티티	111
5.1.2 게시글에 댓글 목록 추가	112
5.1.3 더미 데이터	112
5.2 즉시 로딩과 지연 로딩	113
5.2.1 즉시 로딩	113
5.2.2 지연 로딩	114
5.3 댓글 목록	115
5.3.1 리포지토리	115
5.3.2 응답 DTO	115
5.3.3 서비스	116
5.4 댓글 추가	117
5.4.1 리포지토리	117
5.4.2 요청 DTO	118
5.4.3 응답 DTO	118
5.4.4 서비스	119
5.4.5 컨트롤러	119
5.5 댓글 삭제	121
5.5.1 서비스	121
5.5.2 컨트롤러	122
에필로그	124

CHAPTER

1

리플렉션

프레임워크가 내 코드를 실행하는 원리

이번 챕터가 끝나면

- **제어의 역전**으로 프레임워크가 개발자의 코드를 대신 실행하는 구조를 이해합니다.
- **리플렉션**으로 어노테이션이 붙은 메서드를 찾아 호출합니다.
- **컴포넌트 스캔**으로 클래스를 찾아 객체로 만드는 과정을 직접 구현합니다.
- **스프링 컨테이너**가 빈을 보관하고 필요한 곳에 주입하는 구조를 이해합니다.

챕터 1. 리플렉션

오픈이는 이제 막 자바(Java)를 배우기 시작한 학생입니다. 이번 학기 과제인 **게시판 웹 서버**를 만들기 위해 호기롭게 코드를 짜기 시작했지만, 몇 줄 쓰지도 못하고 벽에 부딪혔습니다.

문제는 게시판 기능이 아니라 그 이전 단계에 있었습니다. 사용자의 요청을 처리하려면, 어떤 주소로 들어왔는지 문자열을 일일이 잘라내어 파악하고 주소마다 실행할 코드를 짚지어주는 작업이 먼저 필요했습니다.

정작 게시글 화면은 하나도 만들지 못했는데, 요청을 분류하기 위한 분기문(if-else)만 에너지를 쏟고 있었습니다. 오픈이는 멍하니 모니터를 보며 한숨을 쉬었습니다.

과제는 게시판인데, 서버 뼈대만 만들다 학기가 끝나겠네.

다음 날, 오픈이는 같은 수업을 듣는 선배에게 사정을 털어놓았습니다. 선배는 오픈이가 지금까지 쓴 코드를 훑어보더니 말했습니다.

선배 : "이런 걸 처음부터 직접 만드는 사람은 없어요. 프레임워크를 사용하면 요청을 받고 주소를 분배하는 공통 기능은 이미 다 갖춰져 있거든요. 자바 웹이면 보통 스프링(Spring)을 쓰니까, 그 위에 만들고 싶은 게시판 기능만 구현해 봐요."



그림 1-1. 챕터 1의 실습 흐름

소스코드 준비

이번 챕터의 코드는 스프링 프로젝트가 아니라 순수 자바입니다. 빌드 도구 없이 IDE에서 각 `App.java`의 `main()` 메서드를 실행합니다.

앞의 소스코드 준비에서 클론한 예제 저장소로 이동합니다.

터미널 **챕터 1 폴더로 이동**

```
cd spring-start/ch01
```

ch01 폴더는 패키지 `com.reflection` 아래 세 단계로 나뉩니다.

ch01 파일 구조

```
spring-start/ch01/src/main/java/com/reflection/
├─ ex01/
│   ├── App.java          # [작성] if-else 라우팅
│   └─ BoardController.java # [작성] insert·delete·update 메서드
├─ ex02/
│   ├── App.java          # [작성] 리플렉션으로 호출
│   ├── BoardController.java # [작성] 표식 붙은 컨트롤러
│   └─ RequestMapping.java # [작성] 커스텀 어노테이션
└─ ex03/
    ├── App.java          # [작성] 클래스로더로 폴더 스캔
    ├── BoardController.java # [작성] @Controller 부착
    ├── Controller.java    # [작성] 타입 어노테이션
    └─ RequestMapping.java # [참고] ex02와 동일
```

핵심 로직 자리는 비어 있습니다. 챕터를 따라 코드를 채우고, 막히면 `spring-end`의 완성 코드를 참고하세요.

1.1 스프링의 실행 원리

1.1.1 프레임워크

어떤 웹 서버든 사용자의 요청을 받고, 주소를 확인해 알맞은 코드로 넘기는 과정이 반드시 필요합니다. 순수 자바만으로 개발할 때는 이 공통 작업을 개발자가 일일이 직접 구현해야 했습니다.

하지만 스프링 같은 **프레임워크(Framework)**는 이런 뼈대를 이미 갖춘 상태로 제공합니다. 건축에 비유하자면 기둥과 배관 공사가 모두 끝난 건물과 같습니다. 개발자는 바닥부터 건물을 새로 올릴 필요 없이, 완성된 뼈대 안에서 원하는 기능만 채워 넣으면 됩니다.

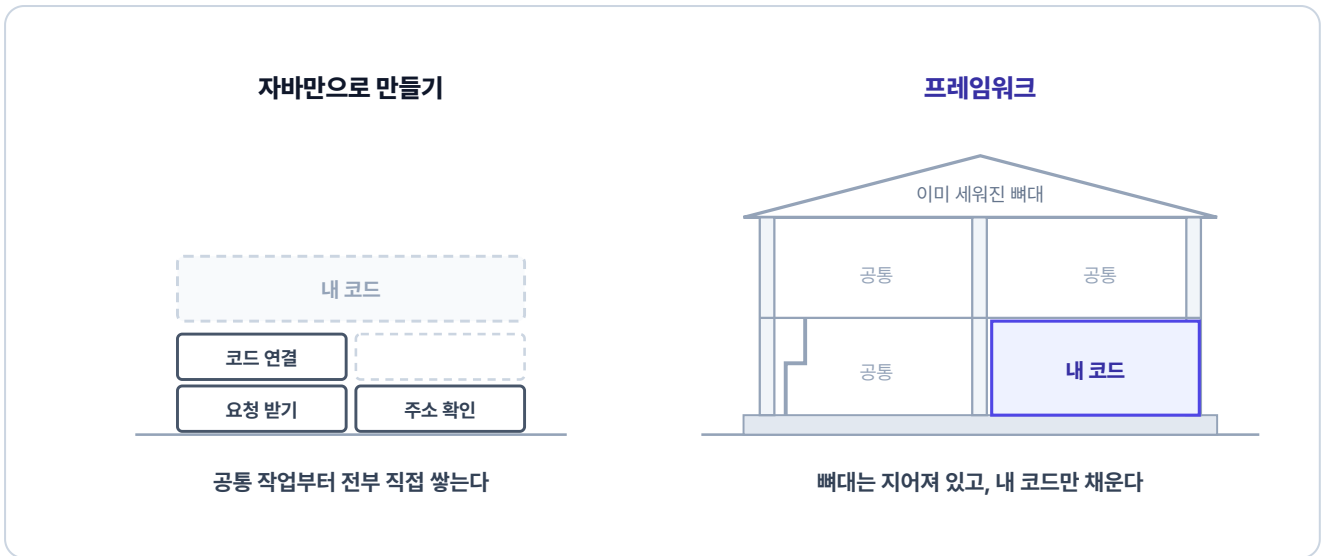


그림 1-2. 자바와 프레임워크의 차이

프레임워크에서는 개발의 출발선이 달라집니다. 번거로운 초기 과정을 건너뛰고 곧바로 핵심 기능을 구현할 수 있습니다. 웹 개발에 필수적인 로그인이나 데이터베이스 연동 역시 이미 검증된 형태로 제공되어 개발 시간을 크게 단축해 줍니다.

또한, 프레임워크는 코드를 배치하는 규칙을 정해둡니다. 일관된 규칙이 적용된 덕분에 다른 사람이 작업한 프로젝트를 열어봐도 전체적인 구조와 역할을 쉽게 파악할 수 있습니다.

1.1.2 제어의 역전

프레임워크의 가장 큰 특징은 코드의 호출 방향이 뒤바뀐다는 점입니다. 순수 자바만으로 개발할 때는 개발자가 전체 흐름을 통제하며 필요한 기능을 직접 호출합니다. 하지만 프레임워크 환경에서는 그 반대입니다. 개발자가 정해진 규칙에 맞춰 코드를 준비해 두면, 프레임워크가 필요한 시점에 개발자의 코드를 호출합니다.



그림 1-3. 호출 방향의 역전

이처럼 프로그램의 실행 주도권이 개발자에서 프레임워크로 뒤바뀌는 현상을 **제어의 역전(IoC, Inversion of Control)** 이라고 합니다.

스프링 역시 이 원칙을 따릅니다. 요청을 받아 담당 코드로 넘기는 일부터 필요한 객체를 만들어 연결하는 일까지, 애플리케이션이 돌아가는 데 필요한 공통 작업을 스프링이 전담합니다. 개발자는 각 요청에 어떤 결과를 응답할지 핵심 기능에만 집중할 수 있습니다.

1.1.3 리플렉션

주도권이 프레임워크로 넘어갔다는 것은, 개발자가 작성한 코드를 스프링이 대신 실행해 준다는 뜻입니다. 하지만 여기에는 한 가지 큰 제약이 있습니다.

보통의 자바 프로그램에서는 개발자가 호출할 대상을 코드에 직접 명시합니다. 자신이 만든 클래스이므로 이름을 정확히 알고 있고, `new` 키워드로 객체를 생성해 필요한 메서드를 직접 적어두면 됩니다.

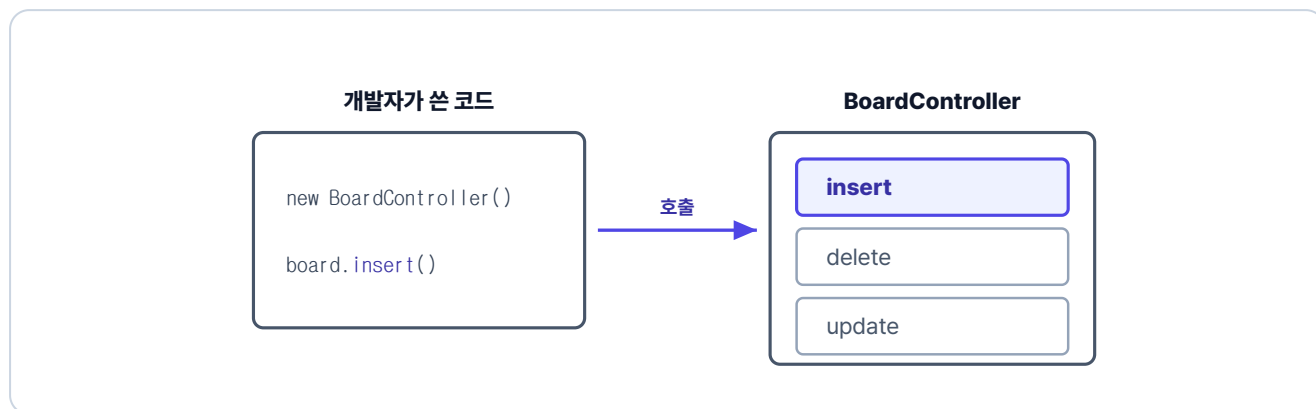


그림 1-4. 자바의 직접 호출

반면 스프링은 개발자가 미래에 어떤 클래스와 메서드를 만들지 미리 알 수 없습니다. 그런데도 이름조차 모르는 코드를 실행 중에 찾아내 호출해야 합니다.

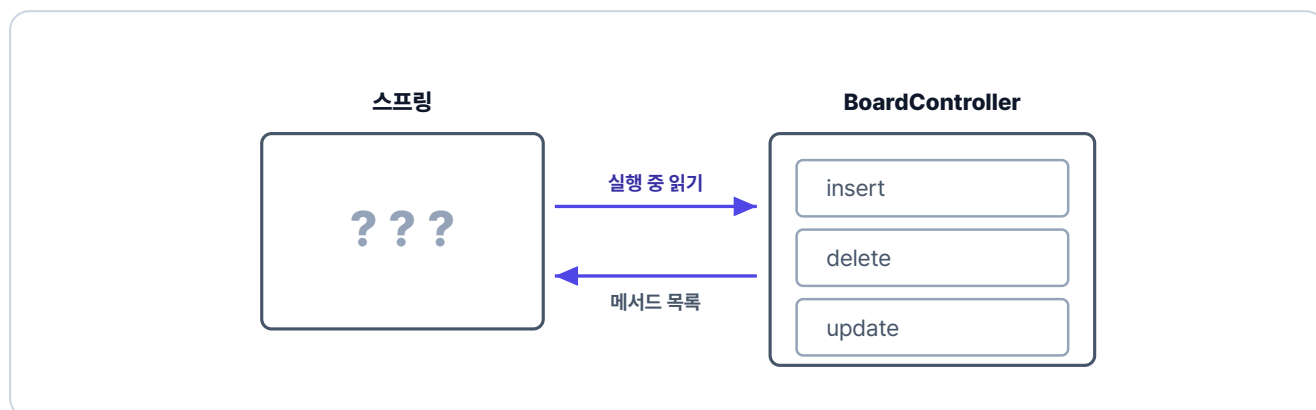


그림 1-5. 스프링의 메서드 탐색

이러한 모순을 해결해 주는 기술이 바로 **리플렉션(Reflection)** 입니다. 리플렉션은 프로그램이 실행되는 도중(런타임)에 클래스와 메서드 정보를 읽어내고, 객체 생성과 호출까지 할 수 있는 자바의 핵심 기능입니다.

1.2 리플렉션 구현

리플렉션이 실제로 어떻게 동작하는지 자바 코드로 직접 구현해 보겠습니다. 가장 단순한 형태의 코드부터 시작해 한 단계씩 기능을 확장해 나갑니다.

1.2.1 if-else 라우팅

가장 기본적인 요청 처리 방식은 들어온 요청 주소를 if 문으로 비교하여 알맞은 메서드를 직접 호출하는 것입니다. 이를 확인하기 위해 먼저 게시글을 다루는 **BoardController**에 등록(insert), 수정(update), 삭제(delete) 메서드를 작성하겠습니다. 각 메서드는 호출될 때 콘솔에 자신의 이름을 출력합니다.

`ex01/BoardController.java` 를 열어 아래와 같이 작성합니다.

실습 1 ex01/BoardController.java. 요청을 처리할 컨트롤러

```
public class BoardController {

    public void insert() {
        System.out.println("insert 호출됨");
    }

    public void delete() {
        System.out.println("delete 호출됨");
    }

    public void update() {
        System.out.println("update 호출됨");
    }
}
```

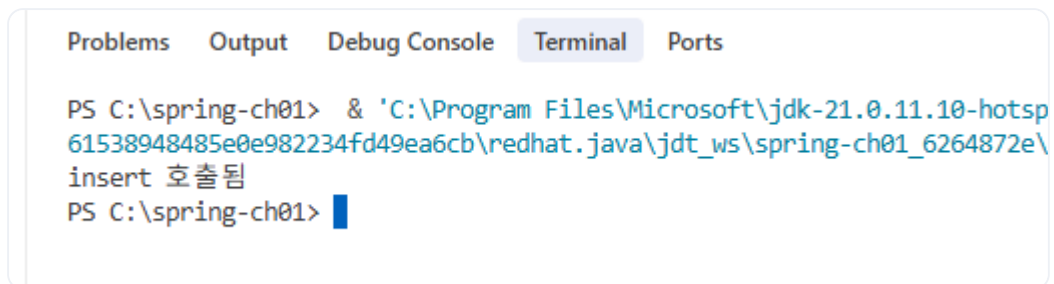
이제 요청 주소를 알맞은 담당 메서드로 연결해 주는 **라우팅(Routing)** 처리를 위해 `main()` 메서드를 작성해 보겠습니다.

`ex01/App.java` 를 열어 아래와 같이 작성합니다.

실습 2 ex01/App.java. 주소에 따라 메서드 호출

```
public class App {  
    public static void main(String[] args) {  
  
        String uri = "/insert";  
        BoardController boardController = new BoardController();  
  
        // TODO: uri에 맞는 메서드를 직접 호출한다  
        // 1. 주소가 /insert면 insert() 호출  
        if (uri.equals("/insert")) {  
            boardController.insert();  
        }  
        // 2. /update면 update() 호출  
        } else if (uri.equals("/update")) {  
            boardController.update();  
        }  
        // 3. /delete면 delete() 호출  
        } else if (uri.equals("/delete")) {  
            boardController.delete();  
        }  
    }  
}
```

uri가 /insert이므로 실행하면 insert()가 호출됩니다.



```
Problems  Output  Debug Console  Terminal  Ports  
  
PS C:\spring-ch01> & 'C:\Program Files\Microsoft\jdk-21.0.11-hotsp  
61538948485e0e982234fd49ea6cb\redhat.java\jdt_ws\spring-ch01_6264872e\  
insert 호출됨  
PS C:\spring-ch01> █
```

그림 1-6. if 문 라우팅 실행 결과

여기서 만약 게시글을 조회하는 select() 메서드를 하나 더 추가한다면 어떨까요?

참고 ex01/BoardController.java. 메서드 추가

```
public void select() {  
    System.out.println("select 호출됨");  
}
```

컨트롤러에 `select()` 메서드를 추가했지만, 정작 `main()` 메서드에는 `/select` 주소를 처리할 분기문이 없습니다. 이 방식은 새로운 메서드를 추가할 때마다 매번 `else if` 문을 추가해야 합니다. 이처럼 컨트롤러뿐만 아니라 호출부의 코드까지 함께 수정해야 한다는 점이 이 구조의 가장 큰 한계입니다.

1.2.2 어노테이션과 리플렉션

앞서 확인한 한계를 극복하려면, 호출부 코드를 매번 수정하지 않고도 프로그램이 실행 중에 알맞은 메서드를 찾아 실행하는 구조가 필요합니다. 이때 사용하는 것이 어노테이션과 리플렉션입니다.

방식은 이렇습니다. 주소를 `if` 문에 일일이 나열하는 대신, 각 메서드 위에 담당할 주소를 표식으로 붙여 둡니다. 클래스나 메서드에 붙이는 이 표식을 **어노테이션(Annotation)** 이라고 합니다.

그리고 프로그램이 실행될 때 컨트롤러의 메서드를 모두 확인하여 이 표식을 읽고, 요청 주소와 일치하는 메서드를 찾아 호출합니다. 실행 중에 이렇게 코드를 분석하고 알맞은 메서드를 호출하는 일을 리플렉션이 담당합니다.

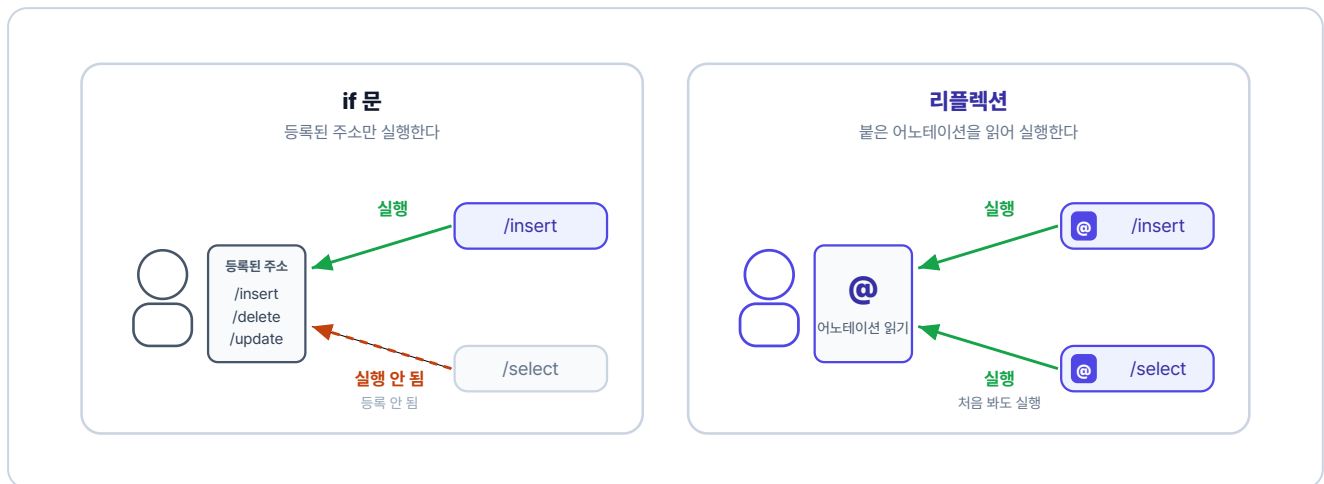


그림 1-7. if 문과 리플렉션의 차이

이제 각 메서드가 담당할 요청 주소를 지정하는 **@RequestMapping** 어노테이션을 직접 만들어 보겠습니다. 어노테이션은 클래스나 인터페이스와 마찬가지로 `@interface` 키워드를 사용하여 정의합니다.

`ex02/RequestMapping.java` 를 열어 아래와 같이 작성합니다.

실습 3 ex02/RequestMapping.java. 주소를 담는 커스텀 어노테이션

```
// 1. 컴파일한 뒤에도 정보를 남겨, 실행 중에 리플렉션이 이 어노테이션을 읽을 수 있게 한다
@Retention(RetentionPolicy.RUNTIME)

// 2. 붙일 수 있는 대상을 메서드로 제한한다
```

```
@Target(ElementType.METHOD)
// 3. 스프링이 제공하는 @RequestMapping과 이름만 같은 어노테이션을 직접 정의한다
public @interface RequestMapping {
    // 4. 메서드가 담당할 요청 주소를 이 속성에 적어 둔다
    String uri();
}
```

그리고 **BoardController**의 각 메서드에 어노테이션을 추가합니다.

`ex02/BoardController.java`를 열어 아래와 같이 변경합니다.

실습 4 ex02/BoardController.java. 메서드마다 담당 주소를 어노테이션으로

```
public class BoardController {

    @RequestMapping(uri = "/insert")
    public void insert() {
        System.out.println("insert 호출됨");
    }

    @RequestMapping(uri = "/delete")
    public void delete() {
        System.out.println("delete 호출됨");
    }

    @RequestMapping(uri = "/update")
    public void update() {
        System.out.println("update 호출됨");
    }

    @RequestMapping(uri = "/select")
    public void select() {
        System.out.println("select 호출됨");
    }
}
```

이어서 `main()` 메서드에서 어노테이션을 읽을 수 있도록 수정합니다.

`ex02/App.java`를 열어 아래와 같이 작성합니다.

실습 5 ex02/App.java. 어노테이션을 읽어 맞는 메서드를 호출

```
public class App {
    public static void main(String[] args) {
```

```

String uri = "/update";
BoardController boardController = new BoardController();

// TODO: 어노테이션을 읽어 uri가 일치하는 메서드를 실행한다
// 1. 컨트롤러가 가진 모든 메서드를 꺼낸다
Method[] methods = boardController.getClass().getDeclaredMethods();
for (Method method : methods) {
    // 2. 그 메서드에 붙은 @RequestMapping 어노테이션을 읽는다
    RequestMapping rm = method.getDeclaredAnnotation(RequestMapping.class);
    // 3. 어노테이션의 uri가 요청 주소와 같으면 그 메서드를 실행한다
    if (rm.uri().equals(uri)) {
        try {
            method.invoke(boardController);
            break;
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
}
}
}

```

이제 `main()` 메서드에서는 컨트롤러에 어떤 메서드가 있는지 알지 못해도, 리플렉션이 실행 중에 어노테이션을 읽어 요청 주소와 맞는 메서드를 호출합니다.

```

PS C:\spring-ch01> c:; cd 'c:\spring-ch01'; & 'C:\Program F
User\workspaceStorage\30d61538948485e0e982234fd49ea6cb\redha
update 호출됨
PS C:\spring-ch01> 

```

그림 1-8. 리플렉션 실행 결과

이러한 구조 덕분에 새 메서드에 어노테이션만 추가해 두면, 호출부 수정 없이도 리플렉션이 이를 읽고 담당 메서드를 찾아 호출할 수 있습니다.

1.2.3 컴포넌트 스캔

현재 구조는 `main()` 메서드에서 개발자가 직접 컨트롤러 객체를 생성하고 있습니다. 컨트롤러가 하나일 때는 괜찮지만, 게시판이나 회원처럼 기능이 늘어나면 매번 객체 생성 코드를 추가해야 합니다. 객체 생성을 개발자가 직접 관리하면 프로그램이 어떤 컨트롤러를 가지고 있는지 스스로 파악할 수 없어, 결국 코드를 계속 수정해야 하는 한계에 부딪힙니다.

이를 극복하려면 개발자가 직접 객체를 만드는 대신, 프로그램 실행 중에 어노테이션이 붙은 클래스를 찾아 자동으로 객체를 생성하는 구조가 필요합니다. 이러한 기능을 **컴포넌트 스캔(Component Scan)** 이라고 부릅니다.

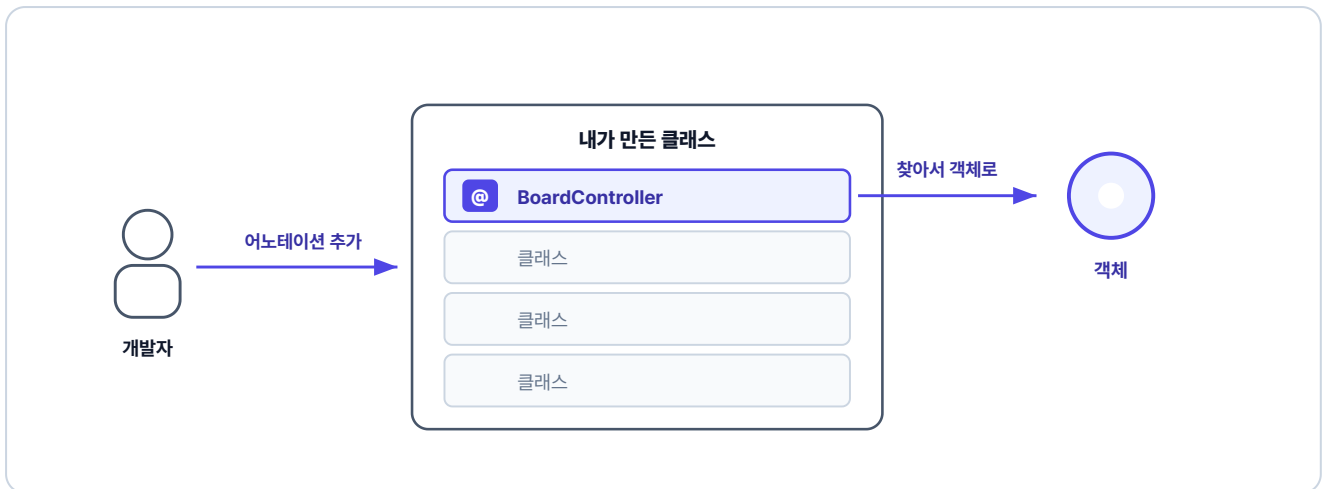


그림 1-9. 컴포넌트 스캔의 객체 생성

이번에는 메서드가 아니라 클래스에 붙일 어노테이션이 필요합니다. 컨트롤러 역할을 하는 클래스임을 나타내는 **@Controller**를 정의합니다.

`ex03/Controller.java`를 열어 아래와 같이 작성합니다.

실습 6 ex03/Controller.java. 컨트롤러임을 나타내는 어노테이션

```
// 1. 컴파일한 뒤에도 정보를 남겨, 실행 중에 리플렉션이 이 어노테이션을 읽을 수 있게 한다
@Retention(RetentionPolicy.RUNTIME)
// 2. 붙일 수 있는 대상을 클래스와 인터페이스로 제한한다
@Target(ElementType.TYPE)
public @interface Controller {
}
```

이어서 `ex03/BoardController.java`를 열어 클래스 선언 위에 **@Controller**를 추가합니다.

실습 7 ex03/BoardController.java. 클래스 위에 @Controller 어노테이션

```
@Controller
public class BoardController {

    @RequestMapping(uri = "/insert")
    public void insert() { System.out.println("insert 호출됨"); }

    @RequestMapping(uri = "/delete")
```

```

public void delete() { System.out.println("delete 호출됨"); }

@RequestMapping(uri = "/update")
public void update() { System.out.println("update 호출됨"); }

@RequestMapping(uri = "/select")
public void select() { System.out.println("select 호출됨"); }

@RequestMapping(uri = "/create")
public void create() { System.out.println("create 호출됨"); }
}

```

이제 핵심이 되는 `main()` 메서드를 살펴보겠습니다. 코드를 보면 직접 객체를 생성하는 구문은 어디에도 없습니다. 대신 `ex03` 폴더의 실제 경로를 읽고 그 안의 클래스 파일을 하나씩 확인하여, **@Controller** 어노테이션이 붙은 클래스만 찾아 객체로 생성합니다.

`ex03/App.java` 를 열어 아래와 같이 작성합니다.

실습 8 ex03/App.java. 폴더를 뒤져 @Controller 클래스를 등록하고 주소로 호출

```

public class App {
    public static void main(String[] args)
        throws URISyntaxException, ReflectiveOperationException {
        // TODO: ex03 폴더를 뒤져 @Controller 클래스만 객체로 만든다
        // 1. ex03 폴더의 실제 경로를 찾는다
        ClassLoader classLoader = Thread.currentThread().getContextClassLoader();
        URL packageUrl = classLoader.getResource("com/reflection/ex03");
        File ex03 = new File(packageUrl.toURI());

        List<Object> instances = new ArrayList<>();

        // 2. 폴더 안의 .class 파일을 하나씩 살핀다
        for (File file : ex03.listFiles()) {
            System.out.println("파일명: " + file.getName());
            if (file.getName().endsWith(".class")) {
                String className = "com.reflection.ex03."
                    + file.getName().replace(".class", "");
                Class<?> cls = Class.forName(className);
                // 3. @Controller 어노테이션이 붙은 클래스만 객체로 만들어 보관한다
                if (cls.isAnnotationPresent(Controller.class)) {
                    System.out.println("어노테이션이 있는 클래스 : " + file.getName());
                    Object instance = cls.getDeclaredConstructor().newInstance();
                    instances.add(instance);
                }
            }
        }
    }
}

```

```

        System.out.println("-----");

        // 4. 보관한 객체에서 주소에 맞는 메서드를 찾아 실행한다
        findUri(instances, "/delete");
    }

    // 5. 보관한 객체를 하나씩 확인해 주소가 일치하는 메서드를 실행한다
    public static void findUri(List<Object> instances, String uri) {
        for (Object instance : instances) {
            Method[] methods = instance.getClass().getDeclaredMethods();
            for (Method method : methods) {
                RequestMapping rm = method.getDeclaredAnnotation(RequestMapping.class);

                if (rm.uri().equals(uri)) {
                    try {
                        method.invoke(instance);
                        break;
                    } catch (Exception e) {
                        e.printStackTrace();
                    }
                }
            }
        }
    }
}

```

`getDeclaredConstructor().newInstance()` 는 클래스의 기본 생성자를 찾아 객체를 만듭니다. 리플렉션으로 클래스와 생성자를 다루는 동안 생기는 예외는 **ReflectiveOperationException** 하나로 묶어 선언했습니다.

실행하면 콘솔에 `ex03` 폴더 안의 파일 이름이 하나씩 출력되고, 그중 **@Controller**가 붙은 클래스가 객체로 등록됩니다. 이어서 `findUri()` 가 등록된 객체에서 `/delete`와 주소가 일치하는 `delete()` 를 찾아 호출합니다.

```

파일명: App.class
파일명: BoardController.class
어노테이션이 있는 클래스 : BoardController.class
파일명: Controller.class
파일명: RequestMapping.class
-----
delete 호출됨

```

그림 1-10. 컴포넌트 스캔 실행 결과

이제 개발자는 클래스에는 **@Controller**를, 메서드에는 **@RequestMapping**을 추가하면 됩니다. 객체를 만드는 일도, 메서드를 찾아 호출하는 일도 프로그램이 대신 처리합니다.

1.3 객체 생성과 연결

1.3.1 빈과 컨테이너

스프링 프레임워크의 가장 핵심적인 역할은 개발자 대신 객체를 생성하고 관리하는 것입니다. 애플리케이션이 실행되면 스프링은 컴포넌트 스캔을 통해 어노테이션이 붙은 클래스를 찾고, 리플렉션을 활용해 자동으로 객체를 생성합니다.

이때 만들어진 객체들을 모아두고 관리하는 전용 공간을 **스프링 컨테이너(Spring Container)**, 컨테이너 안에 보관된 객체를 **빈(Bean)** 이라고 부릅니다.

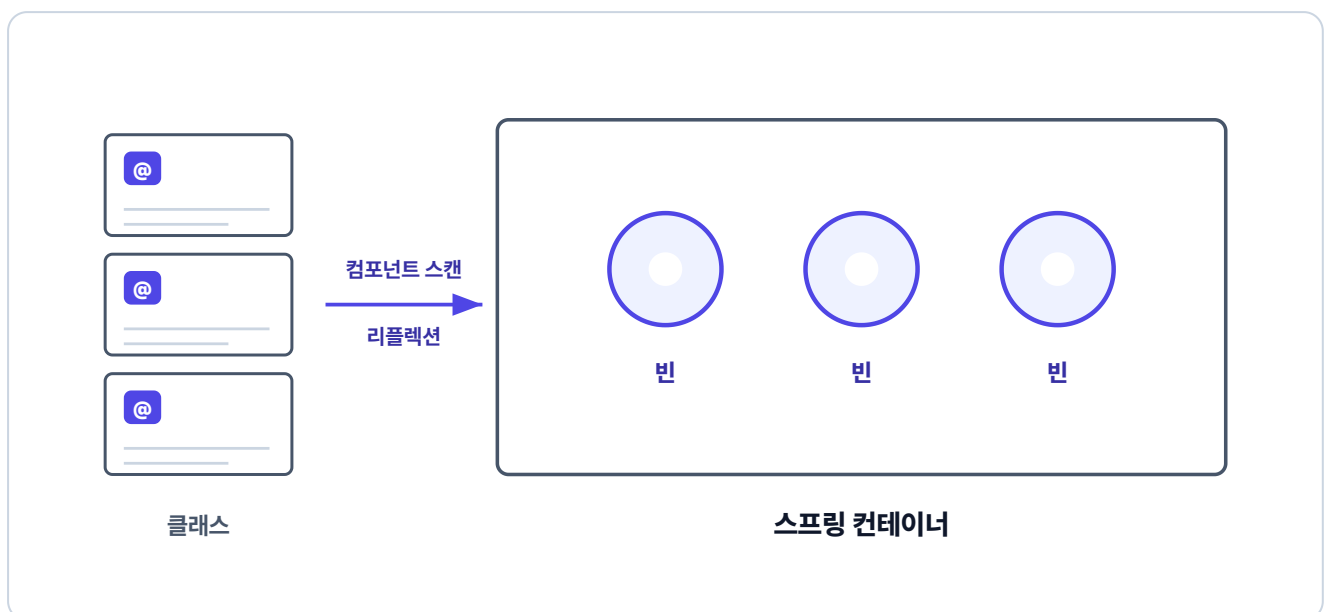


그림 1-11. 스프링 컨테이너와 빈

1.3.2 의존성 주입

그렇다면 이렇게 만들어진 빈들은 어떻게 서로 기능을 주고받을까요? 한 객체가 다른 객체의 기능을 필요로 하는 상태를 **의존성을 가진다**고 합니다. 이때 필요한 객체를 직접 생성해서 쓰면 객체 간의 결합도가 높아져, 나중에 코드를 고치거나 테스트하기가 매우 까다로워집니다.

스프링은 이 문제를 **의존성 주입(Dependency Injection)** 으로 해결합니다. 개발자가 필요한 객체를 직접 만들지 않고 코드에 선언만 해두면, 컨테이너가 실행될 때 알맞은 빈을 찾아 넣어 주는 방식입니다.

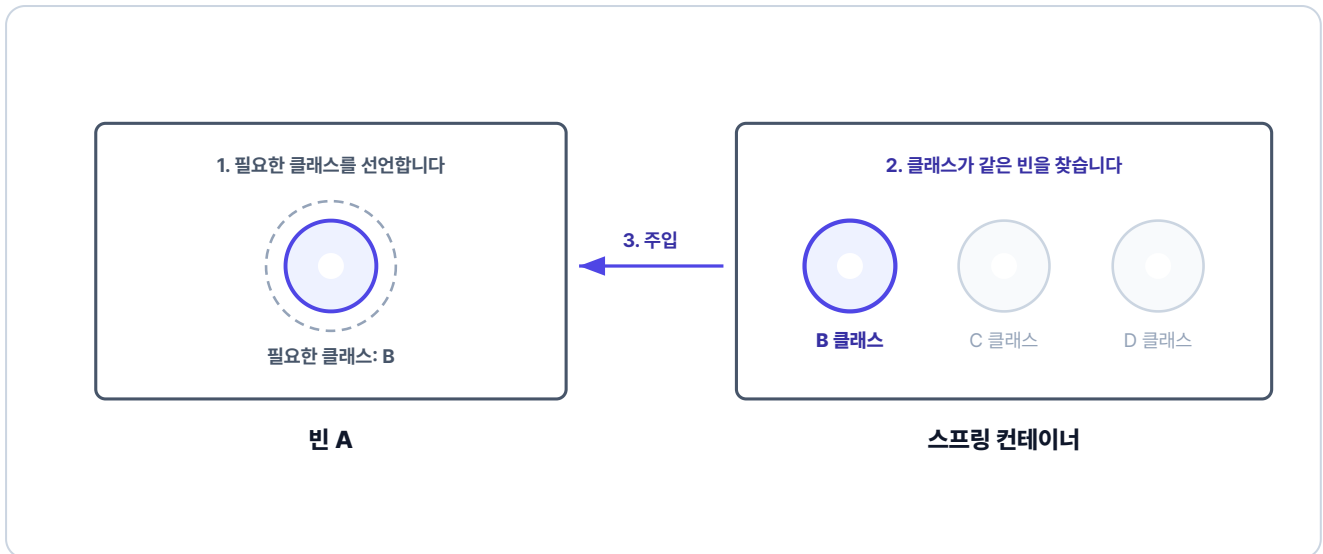


그림 1-12. 컨테이너의 빈 주입

스프링 컨테이너가 객체를 만들고 주입하는 과정을 대신해 줍니다. 덕분에 개발자는 필요한 빈을 선언만 하면 되고, 코드의 결합도 역시 낮아집니다.

1.4 스프링의 요청 흐름

하나의 요청이 서버에 도착해서 컨트롤러까지 전달되기까지 어떤 단계를 거치는지 따라가 보겠습니다.

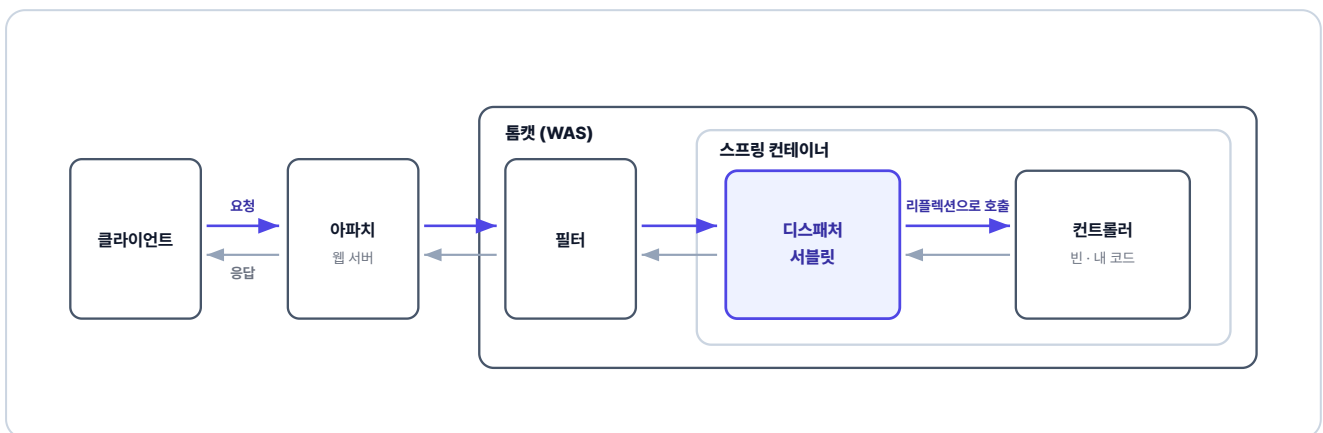


그림 1-13. 요청 처리 흐름

외부 요청이 컨트롤러로 전달되는 흐름에서 각 단계가 맡는 역할은 다음과 같습니다.

1. **웹 서버(아파치).** 클라이언트의 요청을 가장 먼저 받습니다. 이미지나 단순 HTML처럼 미리 만들어져 있는 정적인 데이터는 여기서 바로 처리하고, 자바 코드를 실행해야 하는 요청은 뒤에 있는 WAS로 넘깁니다.

2. **통캣(WAS)과 필터**. 통캣은 자바 코드를 실행해 동적인 결과를 만들어 내는 **웹 애플리케이션 서버(WAS)** 입니다. 통캣은 본격적인 처리에 앞서 요청을 필터에 통과시키고, 필터에서는 로그인 확인처럼 모든 요청에 공통으로 필요한 작업을 미리 처리합니다.
3. **디스패처 서블릿**. 필터를 거친 요청은 스프링 컨테이너 안의 **디스패처 서블릿(DispatcherServlet)** 으로 전달됩니다. 디스패처 서블릿은 요청 주소를 분석해 이 작업을 담당할 컨트롤러를 찾아냅니다.
4. **컨트롤러 호출**. 어떤 주소를 어떤 컨트롤러에 연결할지 실행 중에 찾아내는 일은 스프링이 리플렉션으로 대신 처리합니다. 따라서 개발자는 컨트롤러 코드만 작성해 두면 됩니다.

1.5 학습 지도

이 책의 학습 여정은 다섯 챕터로 이어집니다.

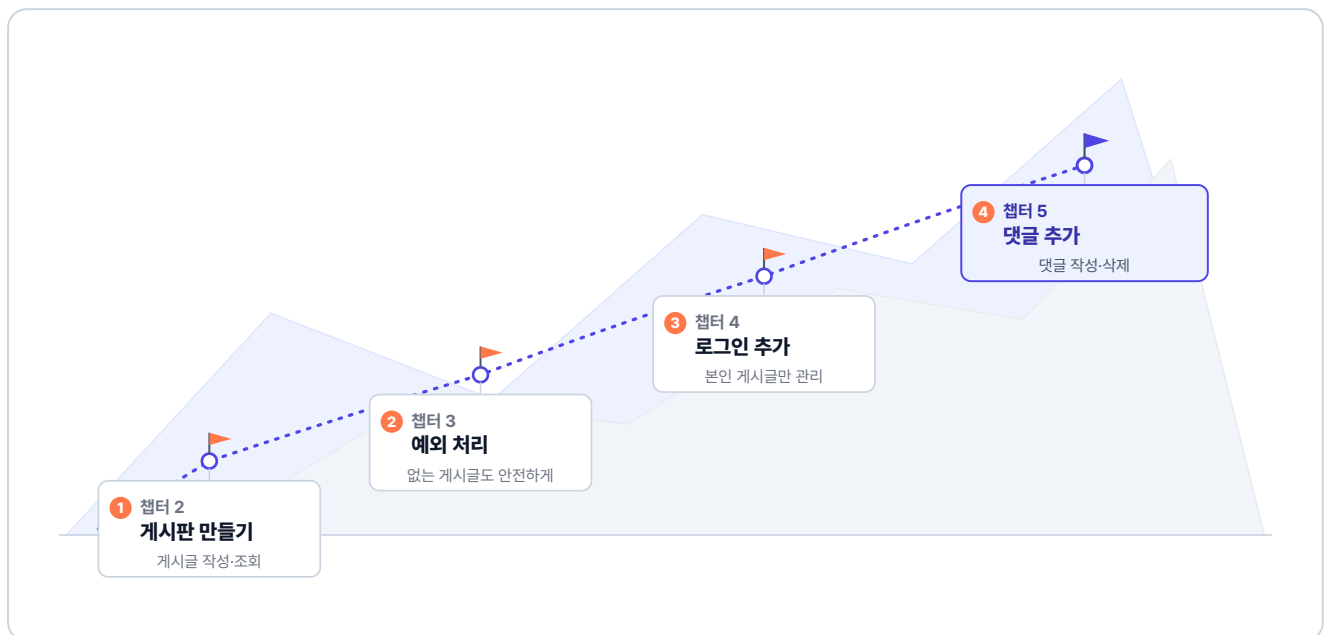


그림 1-14. 앞으로의 학습 여정

각 챕터에서 다루는 내용은 다음과 같습니다.

- 챕터 2에서는 JPA와 3계층 구조로 게시판을 만들어 게시글 등록·조회·수정·삭제 기능을 완성합니다.
- 챕터 3에서는 DTO와 전역 예외 처리를 적용해 응답 형식을 정리합니다.
- 챕터 4에서는 JWT를 활용해 로그인 기능을 추가하고, 본인만 자기 게시글을 관리하게 합니다.
- 챕터 5에서는 양방향 매핑으로 게시글과 댓글을 연결해 댓글 기능을 추가합니다.

다음 챕터에서는 컨트롤러와 서비스, 리포지토리를 직접 만들어 게시글을 저장하고 조회하는 REST API를 완성합니다.

이것만은 기억하자

- 프레임워크는 공통으로 필요한 뼈대를 미리 갖춘 틀입니다.
- 스프링은 컴포넌트 스캔으로 클래스를 찾아 객체를 등록하고, 어노테이션을 읽어 실행할 대상을 결정합니다. 이처럼 개발자가 아닌 프레임워크가 리플렉션을 통해 알맞은 메서드를 직접 찾아 실행하는 구조를 **제어의 역전**이라고 합니다.
- 스프링 컨테이너는 이렇게 생성한 객체(빈)를 보관하다가 필요한 곳에 주입해 줍니다. 개발자는 필요한 빈을 선언하기만 하면 되므로, 코드 간의 결합도가 한결 낮아집니다.
- 클라이언트의 요청은 아파치와 톰캣을 거쳐 디스패처 서블릿으로 전달됩니다. 디스패처 서블릿은 요청 주소를 분석해 담당 컨트롤러를 호출하며, 실행 중에 알맞은 대상을 찾아 연결하는 작업 역시 리플렉션이 처리합니다.

CHAPTER

2

게시판 CRUD

글을 쓰고 읽고 고치고 지우다

이번 챕터가 끝나면

- **REST API**로 게시글을 주소로 가리키고 HTTP 메서드로 구분해 요청과 응답을 주고받습니다.
- **JPA**로 자바 객체와 데이터베이스 테이블을 연결하고, **JPQL**로 원하는 조회를 작성합니다.
- **3계층 아키텍처**로 컨트롤러, 서비스, 리포지토리의 역할을 나눕니다.
- **영속성 컨텍스트**의 캐싱, 쓰기 지연, 더티체킹이 동작하는 과정을 이해합니다.
- **단위 테스트**로 리포지토리를 검증하고 게시판 CRUD를 완성합니다.

챕터 2. 게시판 CRUD

스프링이 내 코드를 어떻게 찾아 실행하는지 원리를 깨친 오픈이는 이제 본격적으로 스프링을 활용해 보기로 합니다. 첫 번째 목표는 웹 서비스의 가장 기본이 되는 **게시판** 만들기입니다. 새 글을 작성하고, 전체 목록을 조회하며, 작성한 글을 수정하거나 삭제하는 필수 기능들을 직접 구현해 볼 계획입니다.

게시판을 만드는 작업 중 중요한 것은 데이터베이스(DB)와의 통신입니다. 사용자가 작성한 글을 데이터베이스에 안전하게 저장해 두고, 누군가 목록이나 특정 글을 요청하면 보관해 둔 데이터를 꺼내 약속된 형식으로 돌려주는 것이 핵심입니다. 이제 오픈이와 함께 이 일련의 흐름을 코드로 완성해 보겠습니다.

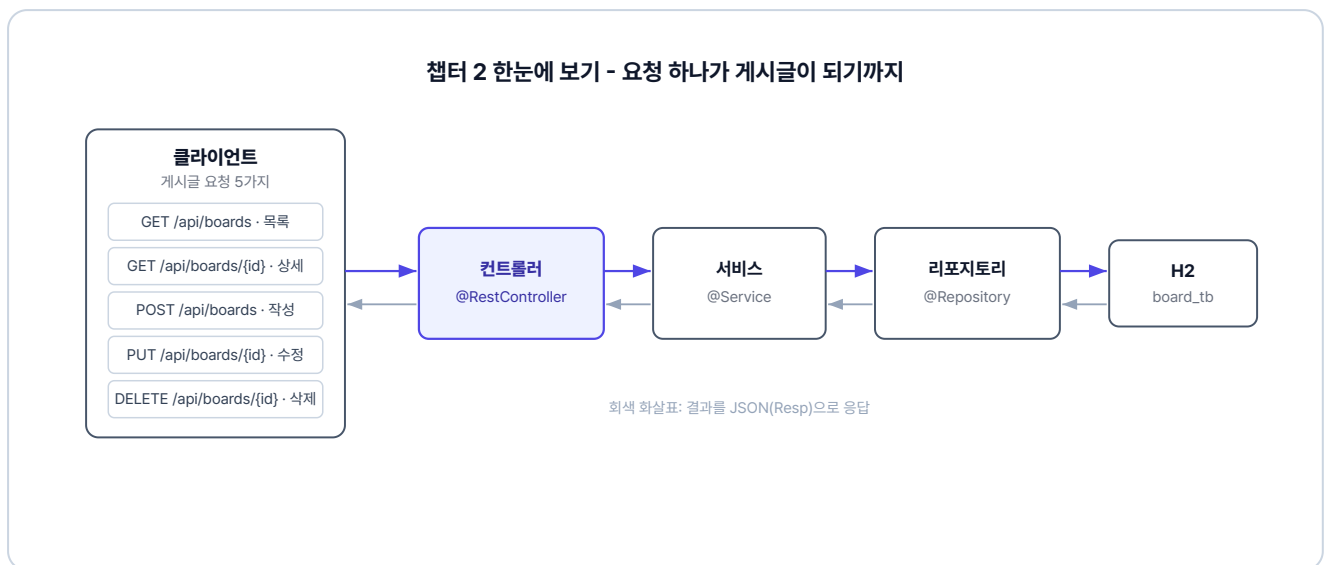


그림 2-1. 챕터 2의 실습 흐름

소스코드 준비

소스코드 준비에서 클론한 예제 저장소에서 이번 챕터 폴더로 이동합니다.

터미널 챕터 2 폴더로 이동

```
cd spring-start/ch02
```

ch02 폴더는 다음과 같이 구성되어 있습니다. 패키지 루트는 `com.metacoding.spring` 입니다.

ch02 파일 구조

```
spring-start/ch02/src/main/java/com/metacoding/spring/
├─ board/
│   ├── Board.java           # [작성] 게시물 클래스
│   ├── BoardController.java # [작성] REST 엔드포인트 5개
│   ├── BoardRepository.java # [작성] DB 저장·조회·삭제
│   └── BoardService.java     # [작성] 게시물 처리 흐름
└─ core/util/Resp.java       # [참고] 공통 응답 형식

spring-start/ch02/src/main/resources/
├─ application.properties    # [참고] H2·JPA 설정
└─ db/data.sql               # [참고] 더미 데이터

spring-start/ch02/src/test/java/com/metacoding/spring/board/
└─ BoardRepositoryTest.java   # [작성] 단위 테스트
```

핵심 로직 자리는 비어 있습니다. 챕터를 따라 코드를 채우고, 막히면 `spring-end` 의 완성 코드를 참고하세요.

이 파일들이 앞의 그림 2-1에서 본 세 층을 이룹니다. 각 클래스가 담당하는 역할은 다음과 같습니다.

클래스	역할
Board	게시글 한 건을 담는 클래스입니다. 데이터베이스의 게시물 표 한 줄에 대응합니다.
BoardRepository	게시글을 데이터베이스에 저장하고, 데이터베이스에서 조회하고 삭제합니다.
BoardService	목록, 상세, 작성, 수정, 삭제의 처리 흐름을 담당합니다.
BoardController	REST 요청을 받아 서비스로 넘기는 입구입니다.
Resp	모든 응답을 한 가지 형식으로 통일합니다.

2.1 REST API

REST API(Representational State Transfer API) 는 서버와 클라이언트가 데이터를 주고받는 방식입니다. 클라이언트는 주소(URI)로 무엇을 원하는지 가리키고, HTTP 메서드로 그것에 무엇을 할지 나타냅니다.

이때 주소가 가리키는 것을 **자원(Resource)** 이라고 합니다. 예를 들어 게시판 서버라면 게시글 목록과 1번 게시글이 각각 자원이고, `/boards` 와 `/boards/1` 이 그 주소가 됩니다.

2.1.1 REST API 탄생 배경

REST API가 왜 지금의 모습이 되었는지, 웹의 발전 과정을 세 단계로 거슬러 올라가 보겠습니다.

초창기의 웹 서버는 미리 만들어 둔 문서나 이미지 같은 정적 자원을 그대로 돌려주는 역할만 했습니다. 브라우저가 특정 주소를 요청하면, 서버는 그 주소에 있는 파일을 찾아 있는 그대로 보내주었습니다.

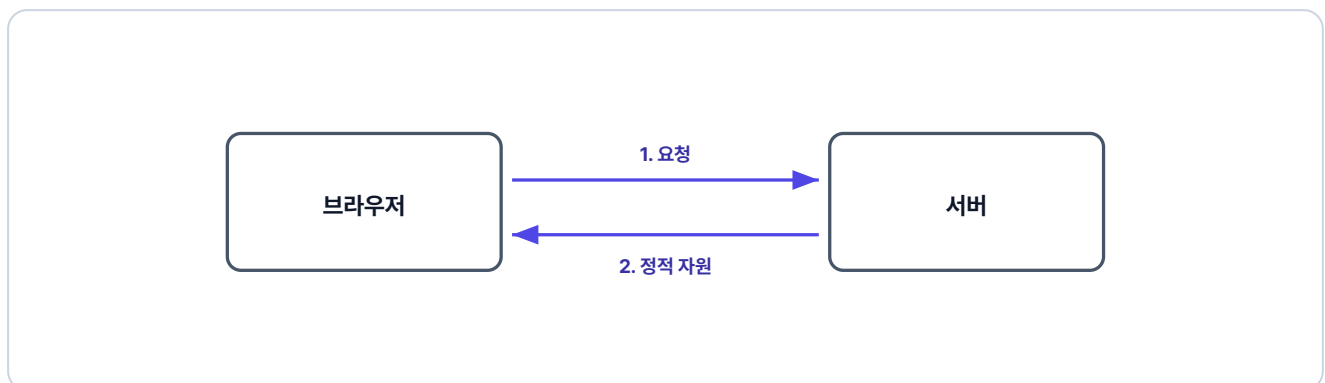


그림 2-2. 초창기 서버의 정적 응답

이후 인터넷이 발전하면서 서버는 단순한 파일 전달을 넘어 WAS(웹 애플리케이션 서버) 형태로 진화했습니다. 요청이 들어올 때마다 서버가 상황에 맞는 HTML 화면을 새로 만들어 응답하기 시작한 것입니다. 하지만 이때까지만 해도 서버가 만든 HTML 화면을 받아보는 대상은 오직 **웹 브라우저**뿐이었습니다.

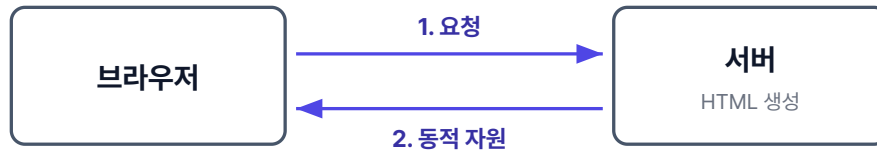


그림 2-3. WAS의 동적 화면 생성

문제는 스마트폰 앱, 스마트 TV, 심지어 다른 서버들까지 등장하면서 시작됐습니다. 다양한 기기들이 서버에 동일한 데이터를 요청하기 시작했는데, 브라우저 전용인 **HTML 화면**을 통째로 넘겨주는 것은 이들에게 전혀 맞지 않았습니다.

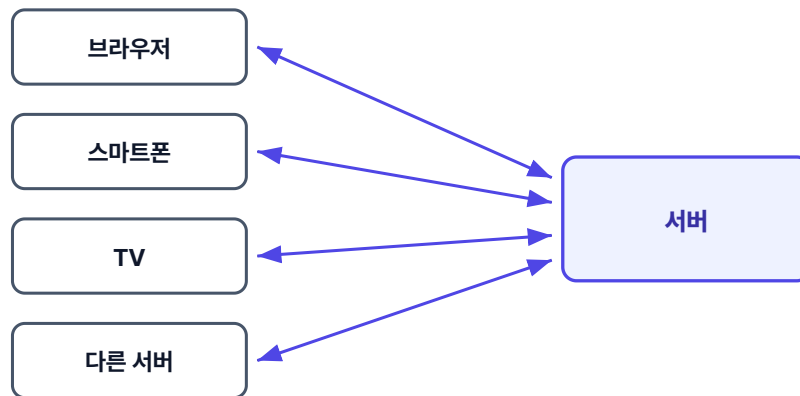


그림 2-4. REST API로 자원에 접근하는 여러 기기

결국 서버는 무거운 화면(HTML) 대신 **데이터** 그 자체만 전달하는 방향으로 변화했습니다. 서버가 HTML이 아닌 JSON이나 XML 같은 데이터를 반환하고, 클라이언트가 이를 받아 화면 구성이나 로직을 처리하는 방식이 REST API입니다.

REST API 방식을 활용하면 브라우저뿐 아니라 스마트폰, TV, IoT 기기, 다른 서버에서도 자원에 접근할 수 있어, 다양한 플랫폼에서 활용 가능한 서버로 확장할 수 있습니다.

2.1.2 JSON

그렇다면 서버는 어떤 형식으로 데이터를 넘겨줄까요? 이때 가장 널리 쓰이는 표준 형식이 바로 JSON입니다.

JSON(JavaScript Object Notation) 은 데이터를 **키(Key)** 와 **값(Value)** 의 쌍으로 묶어 표현하는 텍스트 형식입니다. 구조가 단순해서 기계뿐만 아니라 사람도 직관적으로 읽고 이해하기 쉽다는 장점이 있습니다. 게시글 한 건을 JSON으로 표현하면 이런 모습입니다.

```
{
  "id": 1,
  "title": "첫 번째 게시글",
  "content": "안녕하세요"
}
```

2.1.3 메서드와 주소

REST API의 핵심은 **무엇을(주소)**, **어떻게(메서드)** 할 것인가를 명확하게 나누는 데 있습니다. 주소(URI)가 조작할 대상(자원)을 가리킨다면, 그 대상에 어떤 작업을 할지는 HTTP 메서드가 결정합니다.

게시판을 예로 들면, 게시글이라는 자원을 다룰 때 주로 다음 네 가지 메서드를 사용합니다.

메서드	하는 일	예
GET	조회한다	게시글 목록을 가져온다
POST	새로 만든다	게시글을 작성한다
PUT	수정한다	게시글 내용을 고친다
DELETE	삭제한다	게시글을 지운다

이처럼 HTTP 메서드를 통해 **행위(어떻게)** 를 명확히 분리하고 나면, 주소에는 온전히 **자원(무엇을)** 만 남게 됩니다.

REST API에서는 다른 개발자가 주소만 보고도 어떤 자원을 다루는지 직관적으로 예측할 수 있도록, 자원의 이름을 지을 때 지켜야 할 몇 가지 권장 규칙을 두고 있습니다.

규칙	권장	피할 것
주소의 영어는 소문자로 쓴다	/boards	/Boards
주소에 행위를 넣지 않고 메서드로 표현한다	PUT /boards/1	/boards/1/put

규칙	권장	피할 것
자원은 단수형이 아닌 복수형으로 쓴다	<code>/boards/1</code>	<code>/board/1</code>
긴 단어는 하이픈으로 구분한다	<code>/check-username</code>	<code>/check_username</code>
확장자를 붙이지 않고 헤더에 타입을 담는다	<code>/users + 헤더 application/json</code>	<code>/users.json</code>

2.1.4 요청과 응답

클라이언트와 서버가 통신할 때는 미리 정해진 규격에 맞춰 메시지를 주고받습니다. 이 메시지는 클라이언트가 보내는 **요청(Request)** 과 서버가 돌려주는 **응답(Response)** 으로 나뉘며, 둘 다 세 부분으로 이루어집니다.

먼저 HTTP 요청의 세 부분은 **요청 라인(Request Line)**, **헤더(Header)**, **바디(Body)** 입니다.

- **요청 라인:** HTTP 메서드와 주소가 들어갑니다.
- **헤더:** 바디에 담긴 데이터의 형식(`Content-Type`) 등 서버가 참고할 부가 정보가 담깁니다.
- **바디:** 서버로 보낼 실제 데이터가 담깁니다.

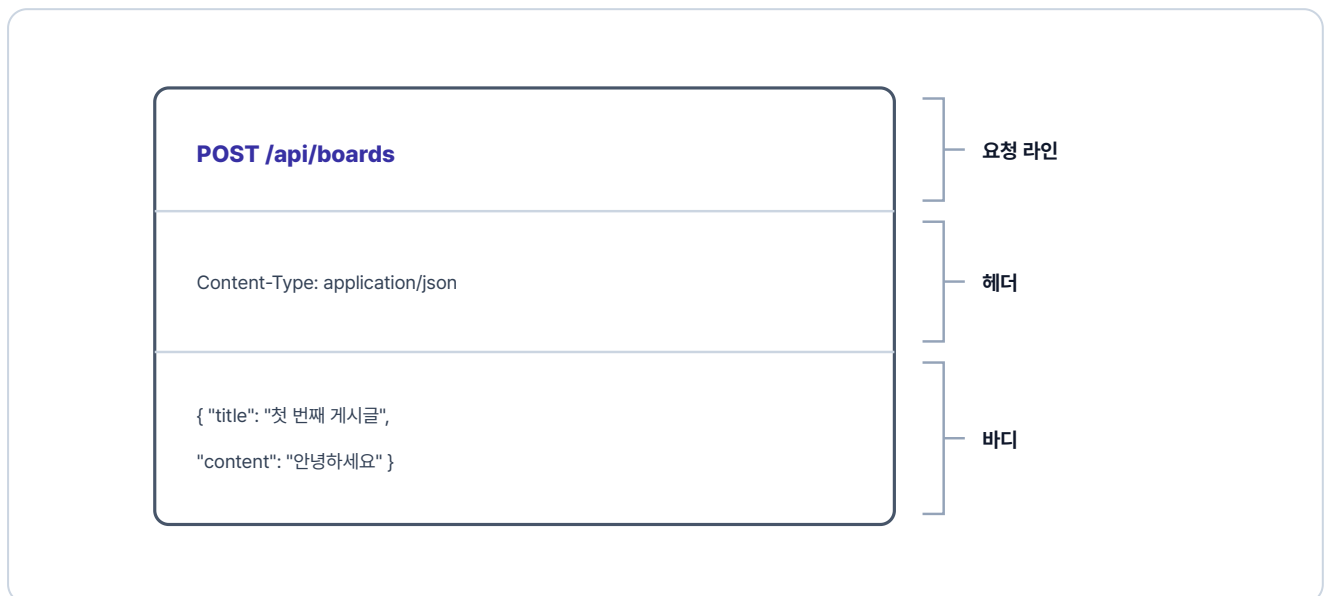


그림 2-5. HTTP 요청의 구조

요청을 받아 처리한 서버는 마찬가지로 세 부분으로 나뉜 HTTP 응답을 돌려줍니다. 다만 첫 줄은 요청 라인이 아니라 **응답 라인(Status Line)** 입니다.

- **응답 라인:** 요청이 성공했는지(200 OK), 아니면 문제가 생겼는지를 나타내는 **상태 코드(Status Code)** 가 들어갑니다.
- **헤더:** 클라이언트에게 돌려주는 데이터의 형식 등 부가 정보가 담깁니다.
- **바디:** 클라이언트가 요청한 실제 데이터가 담깁니다. 게시글 하나를 요청했다면 앞에서 본 JSON이 이 바디에 담겨 있습니다.

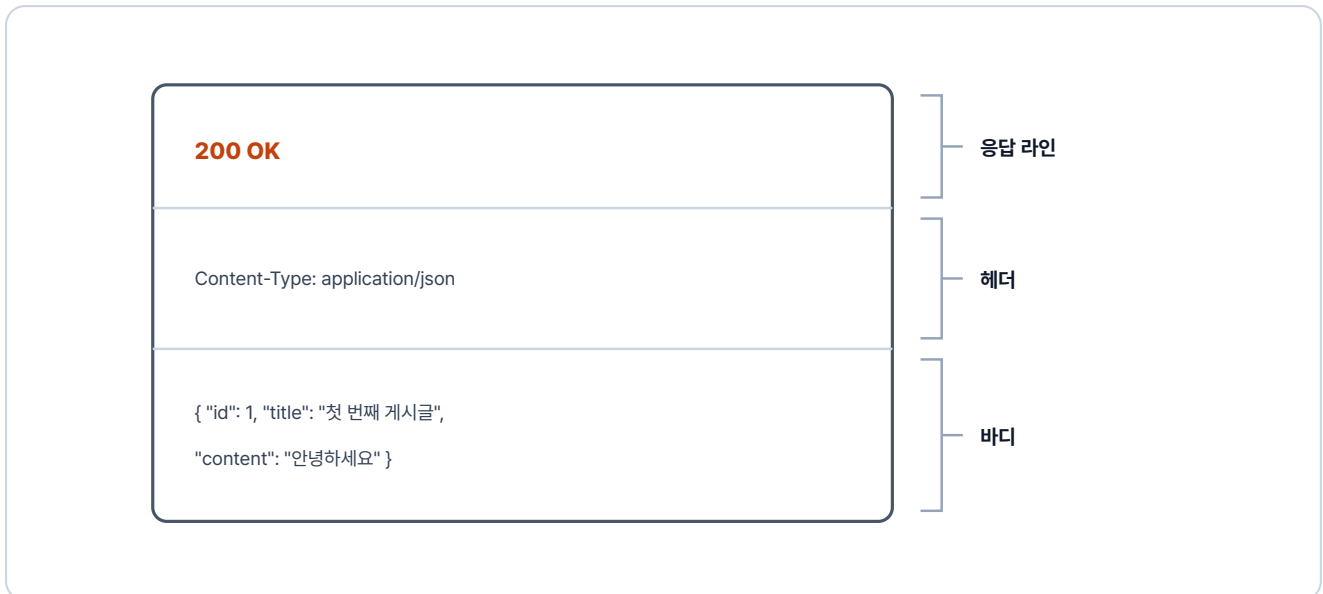


그림 2-6. HTTP 응답의 구조

2.2 프로젝트 생성과 구조

스프링 부트는 스프링을 사용하는 데 필요한 것들을 미리 묶어 둔 도구로, 내장 서버를 갖추고 설정을 맞추는 일을 대신 해 줍니다. 개발자는 명령 한 줄로 서버를 실행하고 기능부터 만들면 됩니다.

2.2.1 프로젝트 생성

이 책의 실습 방식

본격적인 실습은 미리 준비해 둔 **spring-start** 저장소에서 진행합니다. 다만 첫 프로젝트 생성 과정은 직접 눈으로 확인해 보는 것이 좋으므로 여기서는 프로젝트를 만드는 절차만 먼저 살펴보겠습니다.

먼저 자바와 스프링을 다룰 확장 프로그램을 설치합니다. 상단 탭에서 View > Extensions를 선택합니다.

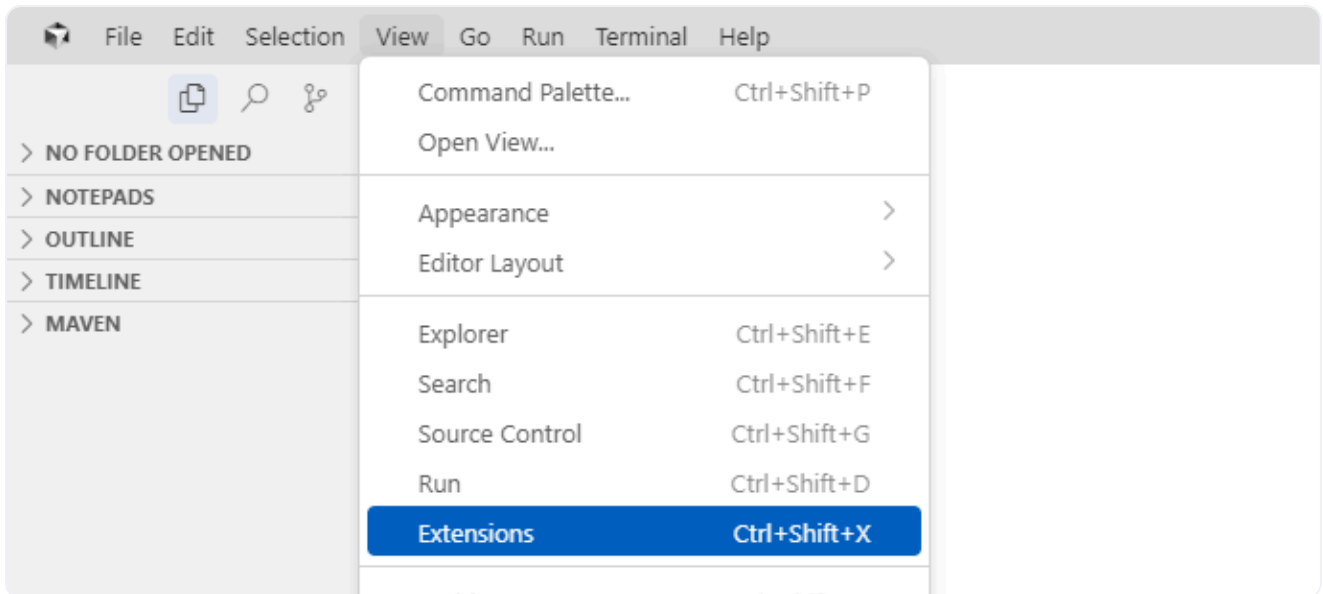


그림 2-7. Extensions 메뉴 선택

검색창에 java를 넣어 Extension Pack for Java를 설치합니다.

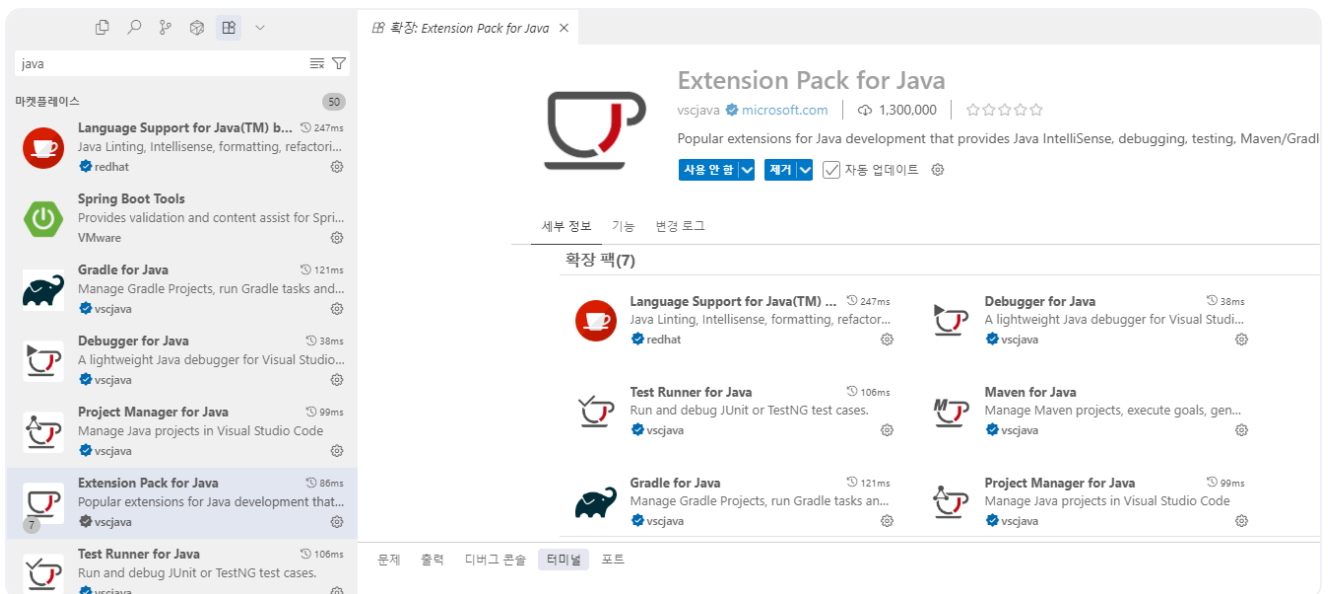


그림 2-8. Extension Pack for Java

이어서 spring으로 검색해 Spring Boot Extension Pack을 설치합니다. 이 확장에 스프링 프로젝트를 만들어 주는 Spring Initializr가 들어 있습니다.

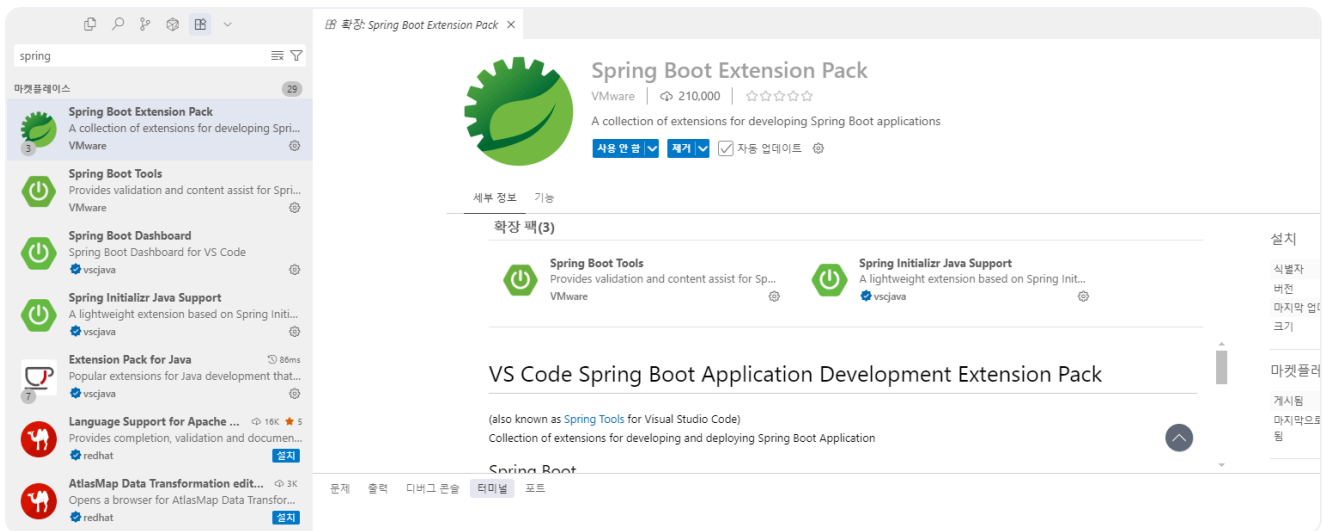


그림 2-9. Spring Boot Extension Pack

설치가 끝나면 View > Command Palette를 엽니다.

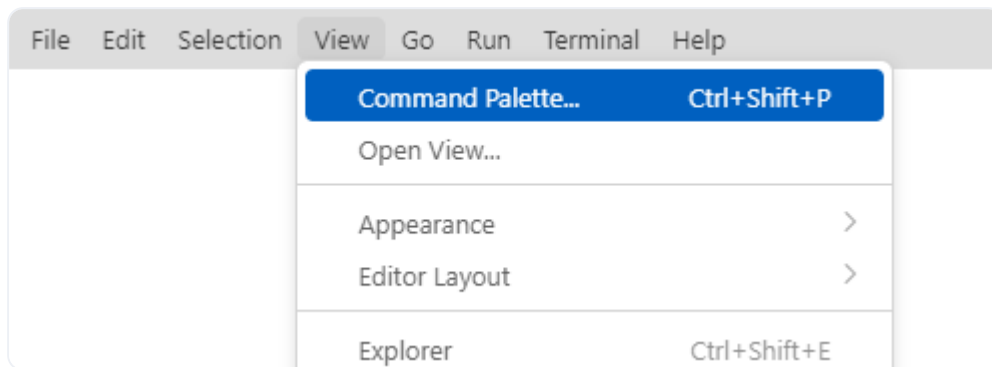


그림 2-10. Command Palette 선택

spring을 입력해 Spring Initializr: Create a Gradle Project를 실행합니다.

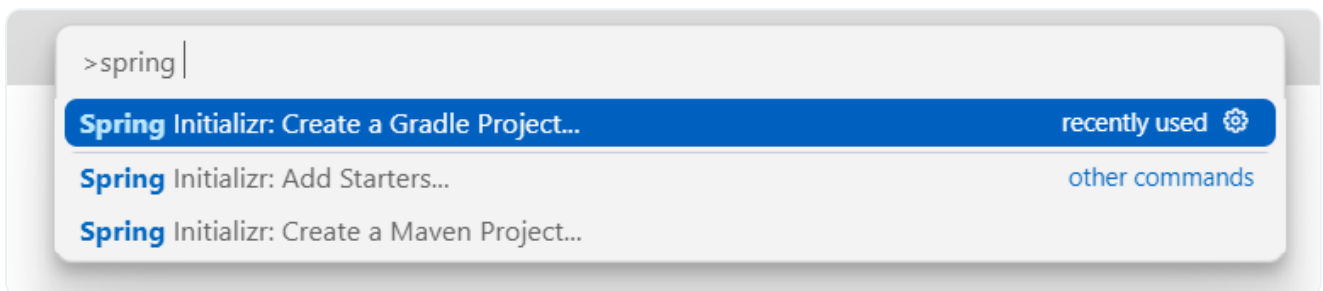


그림 2-11. Gradle 프로젝트 생성 명령

이후 묻는 항목에 이 값들을 넣습니다.

항목	값
스프링 부트 버전	4.0.7
언어	Java
Group Id	com.metacoding
Artifact Id	spring-ch02
Packaging	JAR
자바 버전	21

마지막으로 프로젝트에 넣을 의존성을 고릅니다.

의존성	역할
Spring Web	REST 요청을 받는 웹 계층과 내장 웹 서버가 들어 있습니다
Spring Data JPA	객체와 테이블을 잇는 JPA와 하이버네이트가 들어 있습니다
H2 Database	메모리에서만 동작하는 실습용 데이터베이스입니다
Lombok	게터·Setter 같은 반복 코드를 대신 만들어 줍니다

생성이 끝나면 이런 구조가 만들어집니다.

`build.gradle`에 방금 고른 의존성이 적혀 있고, 코드는 `src` 아래에 들어갑니다.

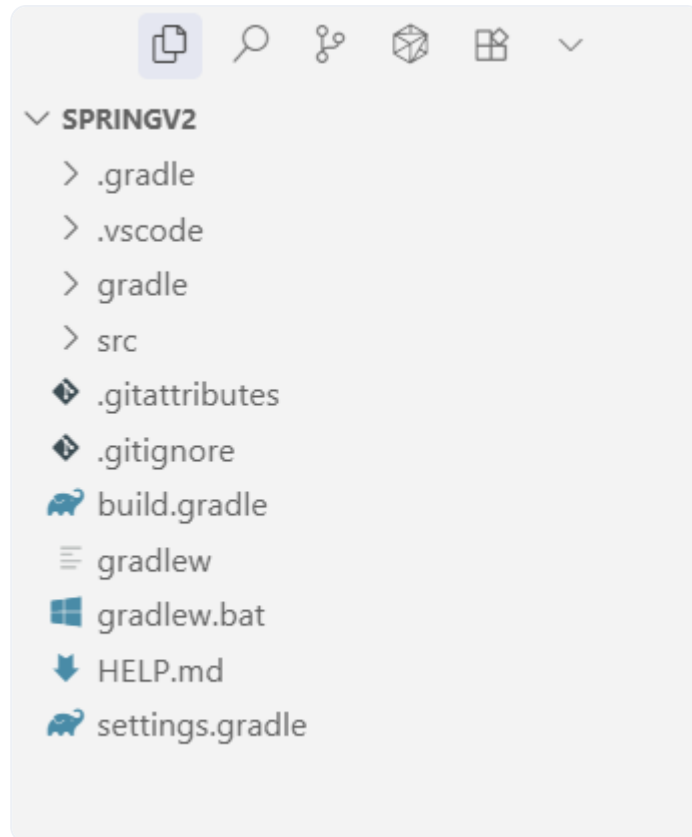


그림 2-12. 스프링 부트 프로젝트 구조

2.3 하이버네이트

220V 플러그는 110V 콘센트에 그대로 꽂히지 않습니다. 규격이 다르기 때문에 사이에 어댑터가 필요합니다. 자바와 데이터베이스도 데이터를 담는 구조가 서로 달라서, 객체를 표에 그대로 넣을 수 없습니다.

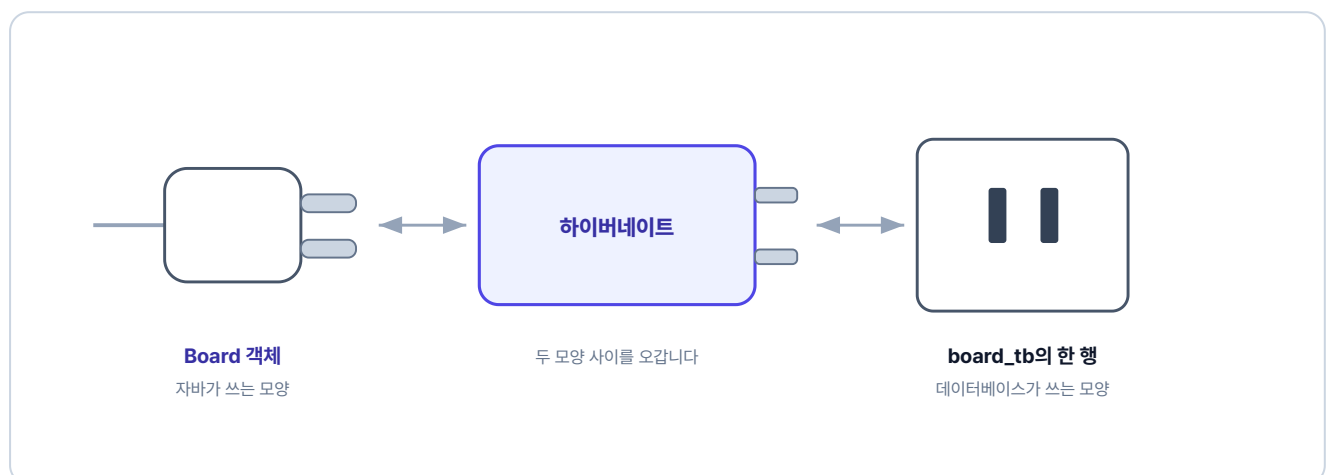


그림 2-13. 어댑터를 끼운 플러그와 콘센트

앞으로 만들 게시글 하나는 자바에서 **Board** 객체이고, 데이터베이스에서는 **board_tb** 테이블의 한 행입니다. 두 구조의 차이는 이렇습니다.

구분	데이터베이스 (RDB)	자바 (Java)
구조	행과 열로 이루어진 2차원의 평면적인 표입니다. 각 칸에는 숫자나 문자 같은 단순한 값만 들어갑니다.	상태(필드)와 행위(메서드)를 함께 가지는 입체적인 구조입니다.
연관관계	데이터 간의 관계를 외래 키 (Foreign Key) 를 통해 맺습니다. 한 칸에 표나 배열을 통째로 넣을 수 없습니다.	객체 안에 다른 객체를 품거나, List나 Map 같은 컬렉션을 담아 훨씬 복잡한 관계를 표현할 수 있습니다.
상속	부모와 자식이라는 상속의 개념 자체가 없습니다.	부모 클래스의 특징을 물려받는 상속 과 다형성 이 핵심입니다.

이러한 구조적 차이 때문에, 객체를 저장하거나 꺼낼 때는 매번 값을 일일이 옮겨 담는 변환 작업이 필요합니다. 마치 어댑터처럼 이 간극을 메워주는 기술을 **ORM(Object-Relational Mapping)** 이라고 합니다. 자바에서는 이 ORM이 갖춰야 할 기능을 **JPA(Java Persistence API)** 라는 규칙으로 정해두었고, 이 규칙대로 만든 대표적인 엔진이 바로 **하이버네이트(Hibernate)** 입니다. 우리가 앞서 추가한 Spring Data JPA 의존성에는 이 하이버네이트가 기본적으로 포함되어 있습니다.

2.4 데이터베이스와 엔티티

2.4.1 데이터베이스 설정

프로젝트의 환경 설정은 `resources/application.properties` 파일에서 관리합니다. 서버 포트나 데이터베이스 연결 정보, JPA 동작 방식 등을 이곳에 지정합니다.

참고 `resources/application.properties`. 프로젝트 환경 설정

```
# 8080 포트로 실행하고, 요청과 응답의 문자를 UTF-8로 처리해 한글이 깨지지 않게 한다
server.port=8080
spring.servlet.encoding.charset=UTF-8
spring.servlet.encoding.enabled=true
spring.servlet.encoding.force=true
```

```
# 메모리에서 동작하는 H2에 sa 계정으로 접속한다
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.url=jdbc:h2:mem:test
spring.datasource.username=sa
spring.datasource.password=

# 브라우저에서 H2 콘솔로 테이블을 확인할 수 있게 한다
spring.h2.console.enabled=true

# 시작 시점에 실행할 SQL 파일의 위치
spring.sql.init.data-locations=classpath:db/data.sql

# 테이블이 만들어진 뒤에 위 파일을 실행한다
spring.jpa.defer-datasource-initialization=true

# 스프링 실행 시 @Entity가 붙은 클래스를 테이블로 자동 생성한다
spring.jpa.hibernate.ddl-auto=create

# JPA가 만든 SQL을 콘솔에 보기 좋게 출력한다
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

실습에 사용할 데이터베이스는 **H2**입니다. 자바로 만들어진 가벼운 관계형 데이터베이스로, 별도의 설치 과정이 필요 없습니다. 다만 메모리 기반으로 동작하기 때문에 서버를 종료하면 저장된 데이터는 모두 사라집니다.

ddl-auto=create는 개발 단계에서만

ddl-auto=create 는 스프링을 실행할 때마다 테이블을 새로 생성하는 설정입니다. 운영 중인 데이터베이스에 두면 재시작 한 번으로 데이터가 사라지므로, 개발 단계에서만 사용합니다.

2.4.2 엔티티

이어서 게시글을 표현하는 **엔티티(Entity)** 를 만들어 보겠습니다. 엔티티란 데이터베이스의 테이블과 매핑되는 자바 클래스를 말합니다. 즉, 엔티티 클래스로 생성한 객체 하나가 데이터베이스 테이블의 한 행 (Row)이 됩니다.

`board/Board.java` 를 열어 아래와 같이 작성합니다.

실습 1 board/Board.java. 게시글 엔티티

```
@Data // 롬복(Lombok). 게터·세터·toString을 컴파일 시점에 대신 만든다
@Entity
```

```
@Table(name = "board_tb") // 이 클래스를 board_tb 테이블에 매핑한다
public class Board {
    @Id // 기본 키. DB가 자동으로 1씩 증가시켜 채운다
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String title;
    private String content;

    @CreationTimestamp // 저장 시점의 현재 시간을 자동으로 기록한다
    private LocalDateTime createdAt;
}
```

애플리케이션을 실행하면 하이버네이트가 이 클래스를 읽어 **board_tb** 테이블을 생성합니다. 앞으로 게시글을 저장하고 조회할 때 필요한 SQL도 하이버네이트가 생성합니다.

2.4.3 더미 데이터

실습을 위해 미리 넣어 두는 가상의 데이터를 **더미 데이터(Dummy Data)** 라고 합니다.

참고 [resources/db/data.sql](#). 게시글 더미 데이터

```
insert into board_tb (title, content, created_at) values ('title1', 'content1', now());
insert into board_tb (title, content, created_at) values ('title2', 'content2', now());
```

터미널에 `./gradlew bootRun` 을 입력해 스프링을 실행합니다.

[illegible]

그림 2-14. 스프링 실행 결과

브라우저에서 `http://localhost:8080/h2-console` 을 입력하면 H2 콘솔에 접속할 수 있습니다.
접속할 User Name과 Password는 `application.properties` 에서 설정한 값입니다.

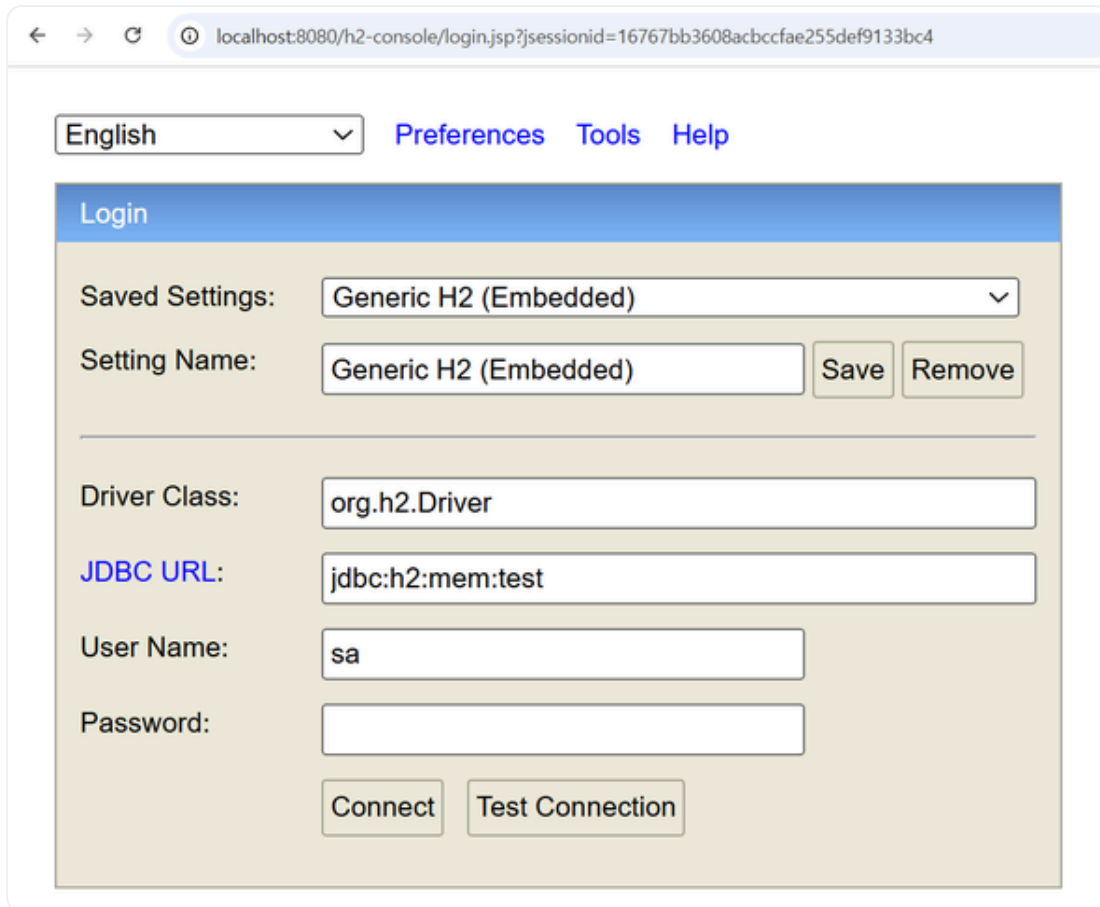


그림 2-15. H2 콘솔 로그인 화면

H2 콘솔에서 **board_tb** 테이블을 확인할 수 있습니다.

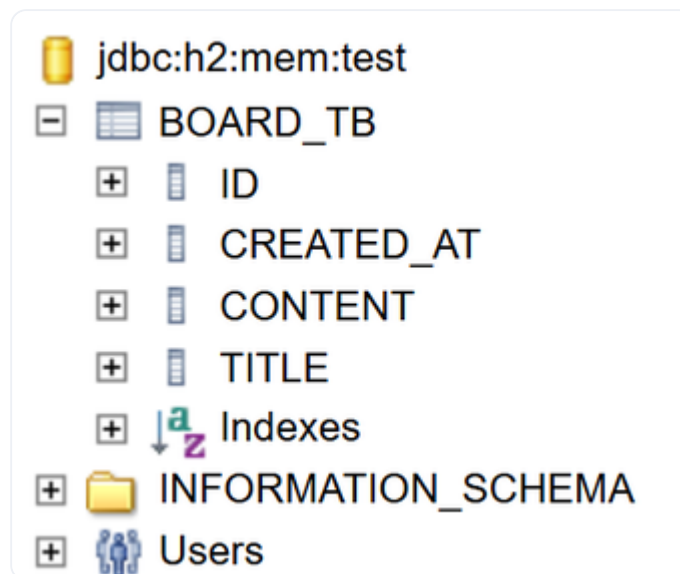


그림 2-16. H2 콘솔의 board_tb 테이블

H2 콘솔에서는 SQL문을 실행할 수 있습니다. SELECT 쿼리로 테이블을 조회하면 `data.sql`에 넣어둔 데이터가 들어와 있는 것을 확인할 수 있습니다.

RunRun SelectedAuto completeClearSQL statement:

SELECT * FROM BOARD_TB

SELECT * FROM BOARD_TB;

ID	CREATED_AT	CONTENT	TITLE
1	2025-09-11 21:51:56.910579	content1	title1
2	2025-09-11 21:51:56.91258	content2	title2

(2 rows, 3 ms)

그림 2-17. board_tb 조회 결과

자바의 카멜 케이스와 DB의 스네이크 케이스

자바는 단어의 첫 글자를 대문자로 적는 **카멜 표기법(createdAt)** 을 주로 사용하고, 데이터베이스는 단어 사이를 밑줄로 잇는 **스네이크 표기법(created_at)** 을 관례로 사용합니다. 하이버네이트는 엔티티 필드의 카멜 표기를 DB의 스네이크 표기 컬럼명으로 자동 변환해 주므로, 개발자는 자바의 네이밍 규칙만 신경 쓰면 됩니다.

2.5 3계층 아키텍처

게시판에 필요한 모든 코드를 하나의 클래스에 작성해도 프로그램은 정상적으로 돌아갑니다. 하지만 요청과 응답을 다루는 코드, 비즈니스 로직을 처리하는 코드, 데이터베이스에 접근하는 코드가 한 곳에 있으면 어디서 에러가 났는지 찾기도 힘든 복잡한 코드가 됩니다.



그림 2-18. 한 클래스에 섞인 코드

이런 문제를 막기 위해, 성격이 같은 코드끼리 층을 나누어 역할을 분리합니다. 이렇게 하면 각 클래스가 맡은 책임이 하나로 뚜렷해져 시스템에 변화가 생겼을 때 다른 코드에 미치는 영향을 최소화할 수 있습니다.



그림 2-19. 계층으로 나눈 코드

스프링 웹 개발에서는 이렇게 역할에 따라 코드를 3개의 층으로 나누는 **3계층 아키텍처(3-Layer Architecture)** 를 주로 사용합니다.

층	하는 일
컨트롤러(Controller)	HTTP 요청 주소를 받아 서비스를 호출하고, 돌려받은 결과를 응답합니다
서비스(Service)	리포지토리를 호출해 요청을 완성하는 비즈니스 로직 을 수행하고, 그 작업을 하나의 트랜잭션으로 처리합니다
리포지토리(Repository)	엔티티를 데이터베이스에서 조회하고 저장·수정·삭제합니다

Spring MVC와 JSON 응답

스프링에서 웹 요청을 다루는 기술을 **Spring MVC**라고 부릅니다. 본래는 화면(View), 요청 처리(Controller), 데이터(Model)를 나누는 전통적인 웹 개발 방식에서 유래한 이름입니다. 하지만 이 책에서는 별도의 화면을 만들지 않고 순수하게 데이터만 제공하는 API 서버를 만듭니다. 따라서 View 계층 없이, 컨트롤러가 반환한 자바 객체가 JSON 문자로 변환되어 응답 본문(Body)에 실립니다.

2.6 공통 응답

core 폴더는 프로젝트에 필요한 설정 파일, 도구 파일 등을 모아 놓은 폴더입니다.

Resp 클래스는 API 응답 구조를 정하는 역할을 합니다. 요청이 성공하면 **status** 를 200으로 두고 응답 데이터를 **body** 에 담아 응답하며, 실패하면 에러 상태 코드와 메시지를 응답합니다.

참고 core/util/Resp.java. 공통 응답 형식

```
public record Resp<T>(Integer status, String msg, T body) {

    public static <B> ResponseEntity<Resp<B>> ok(B body) {
        return new ResponseEntity<>(new Resp<>(200, "성공", body), HttpStatus.OK);
    }

    public static ResponseEntity<?> fail(HttpStatus status, String msg) {
        return new ResponseEntity<>(new Resp<>(status.value(), msg, null), status);
    }
}
```

2.7 리포지토리(Repository)

먼저 데이터베이스에 접근하는 리포지토리를 만들어 보겠습니다. 리포지토리가 데이터베이스에 접근할 때 사용하는 도구가 **EntityManager**입니다. 이 객체는 스프링이 미리 빈(Bean)으로 등록해 두므로, 직접 만들 필요 없이 생성자로 주입받아 사용하면 됩니다.

board/BoardRepository.java 를 열어 아래와 같이 작성합니다.

실습 2 board/BoardRepository.java. 리포지토리 골격

```
@RequiredArgsConstructor
@Repository // 스프링이 빈으로 등록한다
public class BoardRepository {

    private final EntityManager em; // 의존성 주입

    // 아래 절에서 메서드를 하나씩 채운다
}
```

2.7.1 게시물 한 건 조회

EntityManager의 `find()` 메서드는 기본 키(PK)로 엔티티 한 건을 조회할 때 사용합니다. 첫 번째 인자에는 조회할 엔티티의 클래스 타입을, 두 번째 인자에는 찾으려는 기본 키 값을 넣어줍니다. 그러면 조건에 맞는 **Board** 엔티티 한 건을 반환합니다.

`board/BoardRepository.java`의 주석 자리에 아래 메서드를 작성합니다.

실습 3 board/BoardRepository.java. 기본 키로 한 건 조회

```
public Board findById(int boardId) {
    return em.find(Board.class, boardId);
}
```

`find()` 메서드는 데이터베이스에 다음과 같은 select 문을 전달합니다.

```
select id, content, created_at, title from board_tb where id = 1;
```

2.7.2 게시물 전체 조회

EntityManager에는 전체 목록을 한 번에 조회하는 전용 메서드가 없습니다. 따라서 전체 데이터를 가져오려면 **JPQL(Java Persistence Query Language)**을 사용해 직접 쿼리를 작성해서 실행해야 합니다.

실습 4 board/BoardRepository.java. JPQL로 전체 조회

```
public List<Board> findAll() {
```

```
return em.createQuery("select b from Board b", Board.class).getResultList();
}
```

`createQuery()`에 JPQL 문자열과 반환 타입을 넘겨 쿼리를 생성한 뒤, `getResultList()`를 호출해 실행합니다. 그러면 하이버네이트가 이 JPQL을 아래와 같은 실제 SQL로 번역해 줍니다.

```
select id, content, created_at, title from board_tb;
```

JPQL의 문법은 다음 절에서 따로 다루겠습니다.

2.7.3 게시물 저장

데이터를 조회할 때 `find()`를 사용했다면, 새로운 데이터를 추가할 때는 `persist()` 메서드를 사용합니다. 새로 만든 엔티티 객체를 `persist()`에 넘겨주기만 하면, 하이버네이트가 이를 분석해 INSERT 쿼리를 생성하고 데이터베이스에 저장합니다.

실습 5 board/BoardRepository.java. 새 게시물 저장

```
public void save(Board board) {
    em.persist(board);
}
```

`persist()`가 데이터베이스에 전달하는 SQL은 다음과 같습니다. `created_at`은 `@CreationTimestamp`가 채운 값이 그대로 들어갑니다.

```
insert into board_tb (content, created_at, title) values ('content3', now(), 'title
3');
```

2.7.4 게시물 수정

JPA에는 데이터를 수정하는 메서드가 없습니다. 조회해 온 엔티티 객체의 값만 변경하면 데이터베이스에 수정이 자동으로 반영됩니다. 이를 더티 체크(변경 감지)이라 부르며, 자세한 원리는 뒤에서 다루겠습니다.

2.7.5 게시물 삭제

게시글을 삭제할 때는 `remove()` 메서드를 사용합니다. 이 메서드에 지우고자 하는 엔티티를 넘겨주면 JPA가 이를 삭제 대상으로 표시하고, 데이터베이스에 삭제 쿼리를 전달합니다.

실습 6 board/BoardRepository.java. 게시물 삭제

```
public void delete(Board board) {  
    em.remove(board);  
}
```

`remove()` 가 데이터베이스에 전달하는 SQL은 다음과 같습니다.

```
delete from board_tb where id = 2;
```

2.8 JPQL

JPQL은 데이터베이스 테이블이 아닌 자바 엔티티와 필드 이름을 기준으로 작성하는 JPA 전용 SQL입니다. 작성된 JPQL은 실행 시점에 JPA가 SQL로 번역하여 데이터베이스에 전달합니다. SELECT, UPDATE, DELETE는 지원하지만 INSERT는 지원하지 않으므로, 새로운 데이터를 추가할 때는 앞서 살펴본 `persist()` 메서드를 사용합니다.

JPQL의 기본 형태는 테이블 이름 대신 엔티티 이름을 적고, 별칭을 사용해 대상을 가리키는 방식입니다. 앞서 전체 조회에 사용한 쿼리가 바로 이 형태입니다.

```
select b from Board b
```

일부 필드만 조회할 때는 별칭 뒤에 점(.)을 찍고 필드 이름을 적습니다. 이때 데이터베이스의 컬럼명 (`created_at`)이 아니라 엔티티의 필드명(`createdAt`)을 적어야 합니다.

```
select b.title, b.content from Board b
```

조건을 추가할 때는 `where` 절에 콜론(:)을 붙여 파라미터를 조건에 넣습니다.

```
select b from Board b where b.id = :id
```

수정과 삭제도 같은 방식으로 작성합니다.

```
update Board b set b.title = '제목 수정' where b.id = :id
```

```
delete from Board b where b.id = :id
```

2.9 영속성 컨텍스트

스프링의 **EntityManager**는 데이터베이스에 직접 SQL을 보내기 전에, 엔티티를 먼저 올려두고 관리하는 전용 공간을 운영하는데, 이 공간을 **영속성 컨텍스트(Persistence Context)** 라고 부릅니다.

`persist()` 나 `find()` 처럼 **EntityManager**를 통해 엔티티가 영속성 컨텍스트에 등록되면, **엔티티는 영속 상태가 됩니다**. 영속성 컨텍스트는 이 객체의 상태 변화를 지켜보고 있다가, 변경된 내용을 데이터베이스에 반영합니다.

영속성 컨텍스트의 특징은 크게 세 가지입니다.

2.9.1 캐싱

캐싱은 한 번 조회한 엔티티를 영속성 컨텍스트에 보관해 두었다가, **똑같은 엔티티를 다시 요청할 때 데이터베이스를 거치지 않고 바로 반환하는 기능입니다**.

처음 조회하는 게시글은 아직 영속성 컨텍스트에 없습니다. 따라서 데이터베이스에 SELECT 쿼리를 보내 데이터를 직접 조회해 온 뒤, 이를 영속성 컨텍스트에 보관하고 반환합니다.



반면 같은 게시글을 다시 조회할 때는 데이터베이스를 거치지 않습니다. 영속성 컨텍스트에 보관된 엔티티를 바로 반환하기 때문에, **한 트랜잭션 안에서** 여러 번 조회하더라도 SELECT 쿼리는 최초 한 번만 실행됩니다.

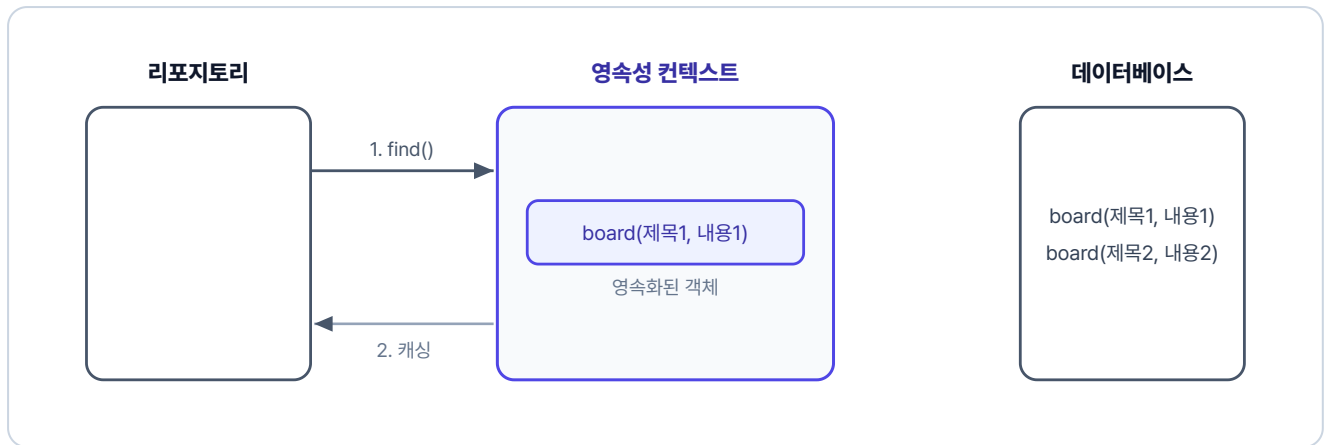


그림 2-21. 캐시 조회

같은 게시글을 두 번 조회해도 데이터베이스에 전달되는 SQL은 한 번뿐입니다.

```
select * from board_tb where id = 1;
```

트랜잭션이란?

트랜잭션(Transaction)은 여러 작업을 하나로 묶어 전부 반영하거나 전부 취소하는 작업 단위입니다.

스프링에서는 보통 서비스 계층의 메서드 하나가 한 트랜잭션 단위가 됩니다.

영속성 컨텍스트도 이 트랜잭션이 실행되는 동안에만 유지됩니다. 메서드가 정상적으로 끝나 트랜잭션이 종료되면 컨텍스트도 닫히고, 안에 있던 엔티티는 더 이상 관리되지 않습니다.

2.9.2 쓰기 지연

우리는 마트에서 물건을 고를 때마다 계산하지 않고, 장바구니에 담은 후 한 번에 결제합니다. 이와 같이 영속성 컨텍스트는 데이터를 변경하는 SQL을 곧바로 실행하지 않고 내부의 임시 공간(버퍼)에 모은 뒤 한 번에 데이터베이스로 전송합니다. 이런 방식을 쓰기 지연(Write Behind)이라고 합니다.

`persist()` 나 `remove()` 같은 메서드가 실행되면 JPA는 쿼리문을 버퍼에 저장한 후 트랜잭션이 성공하는 시점에 데이터베이스로 전달합니다.

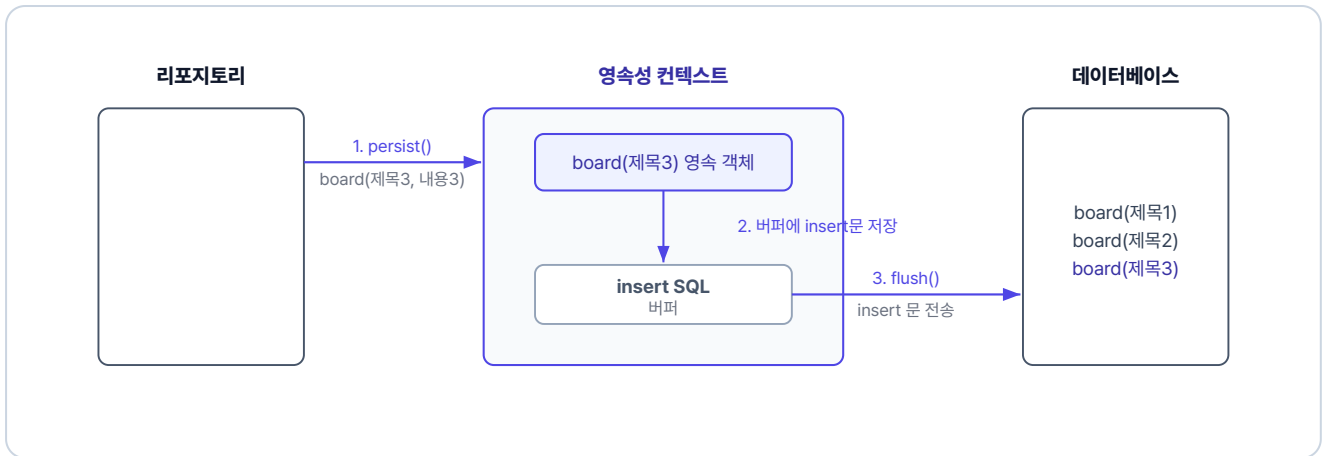


그림 2-22. 쓰기 지연

`persist()`로 저장한 게시글은 `flush()` 시점에 다음 SQL로 데이터베이스에 전달됩니다.

```
insert into board_tb (title, content, created_at) values ('제목3', '내용3', now());
```

flush()란?

flush()는 영속성 컨텍스트의 변경 사항을 데이터베이스에 동기화하기 위해, 버퍼에 쌓여 있던 SQL을 전송하는 메서드입니다. 개발자가 직접 호출하지 않아도 트랜잭션이 성공할 때 JPA가 실행합니다.

단, 기본 키 생성을 DB에 맡기는 IDENTITY 전략에서는 `persist()` 호출 즉시 INSERT 쿼리가 날아갑니다. DB가 만들어준 기본 키를 먼저 알아야 영속성 컨텍스트에 엔티티를 등록할 수 있기 때문입니다.

2.9.3 더티체킹

렌터카를 빌릴 때 직원이 차 상태를 사진으로 남겨 두었다가, 반납할 때 그 사진과 지금 차를 비교해서 달라진 곳을 찾아냅니다. 이와 같이 **영속성 컨텍스트는 엔티티가 영속 상태가 되는 순간의 값을 스냅샷으로 찍어 둡니다**. 이후 엔티티의 값이 변경되면 JPA가 스냅샷과 비교해 변경된 내용을 UPDATE 문으로 버퍼에 담아 둡니다. 이를 **더티체킹(Dirty Checking)**이라고 합니다.

수정할 게시글을 `find()`로 조회하면 영속화가 되며 이 상태를 스냅샷으로 남깁니다.



그림 2-23. 수정할 게시물 조회와 영속화

영속화된 엔티티의 값을 바꾸면 스냅샷과 달라집니다. 영속성 컨텍스트는 이 차이를 감지해 UPDATE 문을 버퍼에 만들어 두고, `flush()` 시점에 데이터베이스로 내보냅니다.

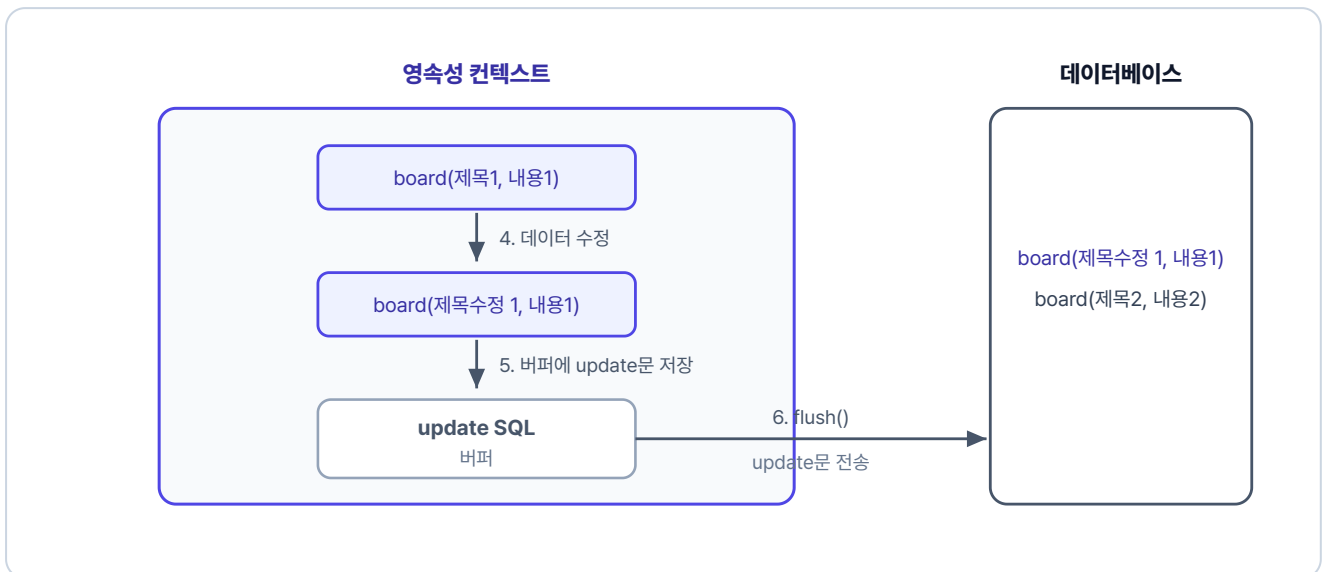


그림 2-24. 더티체킹

2.10 단위 테스트

리포지토리에 조회, 저장, 삭제 메서드 작성을 마쳤으니 기능이 의도대로 동작하는지 검증할 차례입니다. 이렇게 작성한 코드를 검증할 때는 **단위 테스트(Unit Test)** 를 사용합니다.

커피 머신을 예로 들어 보겠습니다. 커피 머신은 원두를 가는 분쇄기와 커피를 내리는 추출기로 구성됩니다. 두 기능이 하나로 결합되어 있다면, 커피가 정상적으로 나오지 않을 때 어느 쪽 문제인지 파악하기 어렵습니다.

반면 두 기능을 분리해 독립적으로 작동시키면 원인 파악이 쉬워집니다. 소프트웨어 역시 문제의 원인을 쉽게 찾기 위해 외부 의존성을 배제하고 가장 작은 기능 단위만 분리해 검증합니다. 이 방식이 단위 테스트입니다.

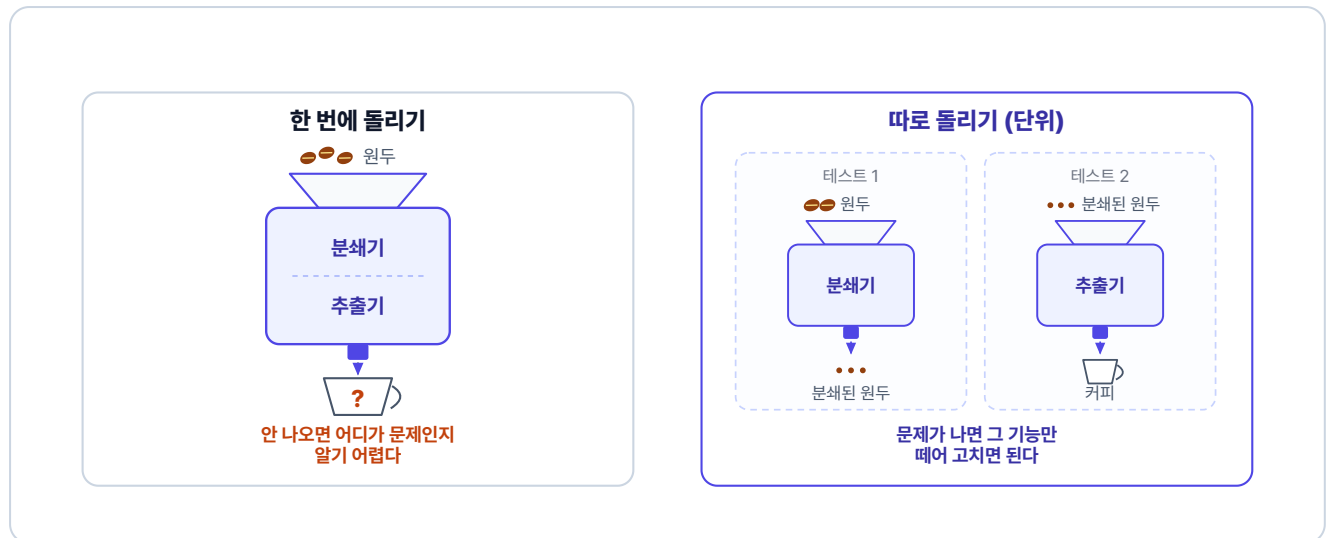


그림 2-25. 단위 테스트가 필요한 이유

자바에서는 이러한 단위 테스트 실행을 **JUnit**이 담당합니다. 검증할 메서드 위에 **@Test** 어노테이션만 추가하면 해당 메서드를 개별적으로 실행할 수 있습니다.

2.10.1 given-when-then 패턴

단위 테스트 코드는 일반적으로 **given-when-then**이라는 세 단계로 나누어 작성합니다. 테스트에 필요한 데이터를 준비하는 **given**, 검증할 대상 메서드를 호출하는 **when**, 그리고 실행 결과가 기대한 대로 나왔는지 확인하는 **then** 순서입니다.

본래 마지막 then 단계에서는 테스트 도구를 사용해 예상값과 실제 결과값을 코드로 비교하고 검증합니다. 하지만 이 책에서는 편의상 복잡한 검증 코드를 작성하는 대신, 실행 결과를 콘솔에 출력하고 눈으로 직접 확인하는 **eye** 단계로 대체하여 진행하겠습니다.



행위 주도 개발(Behavior Driven Development)

Given-When-Then 패턴은 사용자 관점에서 시스템이 **어떤 행위**를 하는지 시나리오 형태로 작성하는 행위 주도 개발 기법에서 유래했습니다. 테스트 코드가 마치 한 편의 설명서처럼 읽히기 때문에, 개발자뿐만 아니라 다른 협업자들도 요구사항과 테스트의 목적을 직관적으로 파악할 수 있습니다.

2.10.2 리포지토리 테스트 작성

이제 given-when-eye 형식에 맞춰 실제 테스트를 만들어 보겠습니다.

테스트 코드는 `src/test/java` 아래에 `src/main/java` 와 같은 패키지 구조로 둡니다. `board` 패키지에 `BoardRepositoryTest.java` 를 생성합니다.

`BoardRepositoryTest.java` 파일에는 아래와 같이 코드를 작성합니다.

실습 7 BoardRepositoryTest.java. 테스트 클래스 골격

```
@Import(BoardRepository.class) // 검증할 BoardRepository를 빈으로 등록한다
@DataJpaTest // 데이터베이스 연결과 EntityManager를 등록한다
public class BoardRepositoryTest {

    @Autowired // 등록된 빈을 주입받는다
    private BoardRepository boardRepository;

    @Autowired
    private EntityManager em;

    // 테스트 메서드를 하나씩 채운다
}
```

먼저 게시글 한 건 조회입니다. 테스트 메서드를 아래와 같이 작성합니다.

실습 8 BoardRepositoryTest.java. 한 건 조회

```
@Test
public void findById_test() {
    // given
    int id = 2;
    // when
    Board board = boardRepository.findById(id);
}
```

```

// eye
System.out.println("=====");
System.out.println("Board Title : " + board.getTitle());
System.out.println("Board Content : " + board.getContent());
}

```

```

Hibernate:
select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
from
    board_tb b1_0
where
    b1_0.id=?
=====
Board Title : title2
Board Content : content2

```

그림 2-27. 한 건 조회 실행 결과

전체 게시물 조회는 결과가 여러 개의 엔티티로 반환되므로, 이를 담기 위해 List 타입을 사용합니다.

실습 9 BoardRepositoryTest.java. 전체 조회

```

@Test
public void findAll_test() {
    // given

    // when
    List<Board> boards = boardRepository.findAll();
    // eye
    System.out.println("=====");
    System.out.println("Board Count : " + boards.size());
    System.out.println("Board 1 title : " + boards.get(0).getTitle());
    System.out.println("Board 2 content : " + boards.get(1).getContent());
}

```

```

Hibernate:
  select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
  from
    board_tb b1_0
=====
Board Count : 2
Board 1 title :title1
Board 2 content :content2
=====

```

그림 2-28. 전체 조회 실행 결과

게시글 저장은 `save()` 메서드를 실행한 후 저장된 결과를 확인하기 위해 `findAll()` 을 한 번 더 사용합니다.

실습 10 BoardRepositoryTest.java. 저장

```

@Test
public void save_test() {
    // given
    Board board = new Board();
    board.setTitle("title3");
    board.setContent("content3");
    // when
    boardRepository.save(board);
    // eye
    List<Board> boards = boardRepository.findAll();
    System.out.println("=====");
    System.out.println("Board Count : " + boards.size());
    System.out.println("Board ID: " + boards.get(2).getId());
    System.out.println("Board Title: " + boards.get(2).getTitle());
    System.out.println("Board Content: " + boards.get(2).getContent());
}

```

```

Hibernate:
    insert
    into
        board_tb
        (content, created_at, title, id)
    values
        (?, ?, ?, default)
Hibernate:
    select
        b1_0.id,
        b1_0.content,
        b1_0.created_at,
        b1_0.title
    from
        board_tb b1_0
=====
Board Count : 3
Board ID: 3
Board Title: title3
Board Content: content3

```

그림 2-29. 저장 실행 결과

게시글 수정은 update 메서드를 만들지 않고 영속 상태 엔티티의 더티체킹을 사용합니다.

실제 서비스에서는 트랜잭션이 끝날 때 JPA가 `flush()` 를 실행하지만, 테스트에서는 검증하는 시점에 트랜잭션이 아직 끝나지 않았으므로 `flush()` 를 직접 호출합니다. 그리고 `clear()` 를 활용해 영속성 컨텍스트를 비운 후, 데이터베이스의 값을 조회합니다.

실습 11 BoardRepositoryTest.java. 수정과 더티체킹

```

@Test
public void update_test() {
    // given
    int id = 2;
    // when
    Board board = boardRepository.findById(id);
    board.setTitle("title-update");
    board.setContent("Update-test");
    em.flush();
    em.clear(); // 영속성 컨텍스트를 강제로 비운다
    // eye
    Board result = boardRepository.findById(id);
    System.out.println("=====");
    System.out.println("Board title : " + result.getTitle());
    System.out.println("Board content : " + result.getContent());
}

```

```
}
```

```
Hibernate:
  select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
  from
    board_tb b1_0
  where
    b1_0.id=?
Hibernate:
  update
    board_tb
  set
    content=?,
    created_at=?,
    title=?
  where
    id=?
Hibernate:
  select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
  from
    board_tb b1_0
  where
    b1_0.id=?
=====
Board title : title-update
Board content : Update-test
```

그림 2-30. 수정 실행 결과

게시글 삭제 테스트를 아래와 같이 작성합니다.

실습 12 BoardRepositoryTest.java. 삭제

```
@Test
public void delete_test() {
    // given
    int id = 2;
    Board board = boardRepository.findById(id);
    // when
    boardRepository.delete(board);
    em.flush();
    // eye
    List<Board> boards = boardRepository.findAll();
    System.out.println("=====");
    System.out.println("Board count : " + boards.size());
}
```

```

Hibernate:
    delete
    from
        board_tb
    where
        id=?
Hibernate:
    select
        b1_0.id,
        b1_0.content,
        b1_0.created_at,
        b1_0.title
    from
        board_tb b1_0
=====
Board count : 1

```

그림 2-31. 삭제 실행 결과

이렇게 단위 테스트로 리포지토리만 따로 검증하면 서버를 실행하지 않아도 기능이 의도대로 동작하는지 확인할 수 있습니다.

2.11 게시물 목록

2.11.1 서비스

먼저 서비스를 작성합니다. 서비스는 리포지토리를 주입받고, `게시글목록()` 에서 리포지토리의 `findAll()` 을 호출해 결과를 반환합니다.

`board/BoardService.java` 를 열어 아래와 같이 작성합니다.

실습 13 board/BoardService.java. 게시물 목록

```

@RequiredArgsConstructor
@Service // 스프링이 빈으로 등록한다
public class BoardService {

    private final BoardRepository boardRepository; // 의존성 주입

    public List<Board> 게시물목록() {
        return boardRepository.findAll();
    }
}

```

```
}
```

2.11.2 컨트롤러

이어서 요청을 받을 컨트롤러를 작성합니다. 컨트롤러는 서비스를 주입받고, 서비스의 `게시글목록()` 을 호출해 결과를 반환합니다.

`board/BoardController.java` 를 열어 아래와 같이 작성합니다.

실습 14 board/BoardController.java. 게시글 목록

```
@RequiredArgsConstructor
@RestController
@RequestMapping("/api/boards") // 공통 주소
public class BoardController {

    private final BoardService boardService; // 의존성 주입

    @GetMapping
    public ResponseEntity<?> findAll() {
        List<Board> responseBoardList = boardService.게시글목록();
        return Resp.ok(responseBoardList);
    }
}
```

2.11.3 API 요청

애플리케이션을 실행합니다.

터미널 애플리케이션 실행

```
./gradlew bootRun
```

서버를 실행했으니, 이제 API를 호출해 잘 동작하는지 확인해 보겠습니다. 이 책에서는 브라우저에서 API를 호출하는 도구인 **Hoppscotch**(<https://hoppscotch.io/>)를 사용합니다.

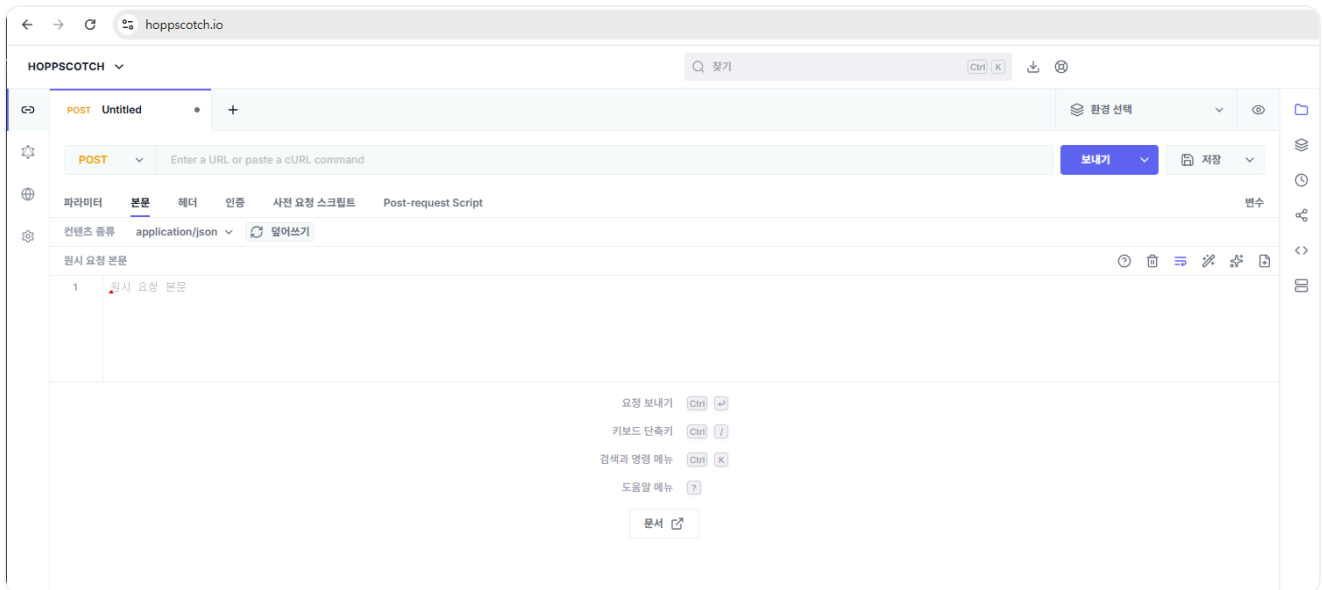


그림 2-32. Hoppscotch 화면

웹 브라우저는 보안 때문에 내 컴퓨터의 localhost로 바로 요청을 보내지 못합니다. 그래서 요청을 대신 전달해 주는 **Hoppscotch Browser Extension**을 Chrome 웹 스토어에서 설치하고, 설정 > Interceptor에서 익스텐션을 선택합니다.

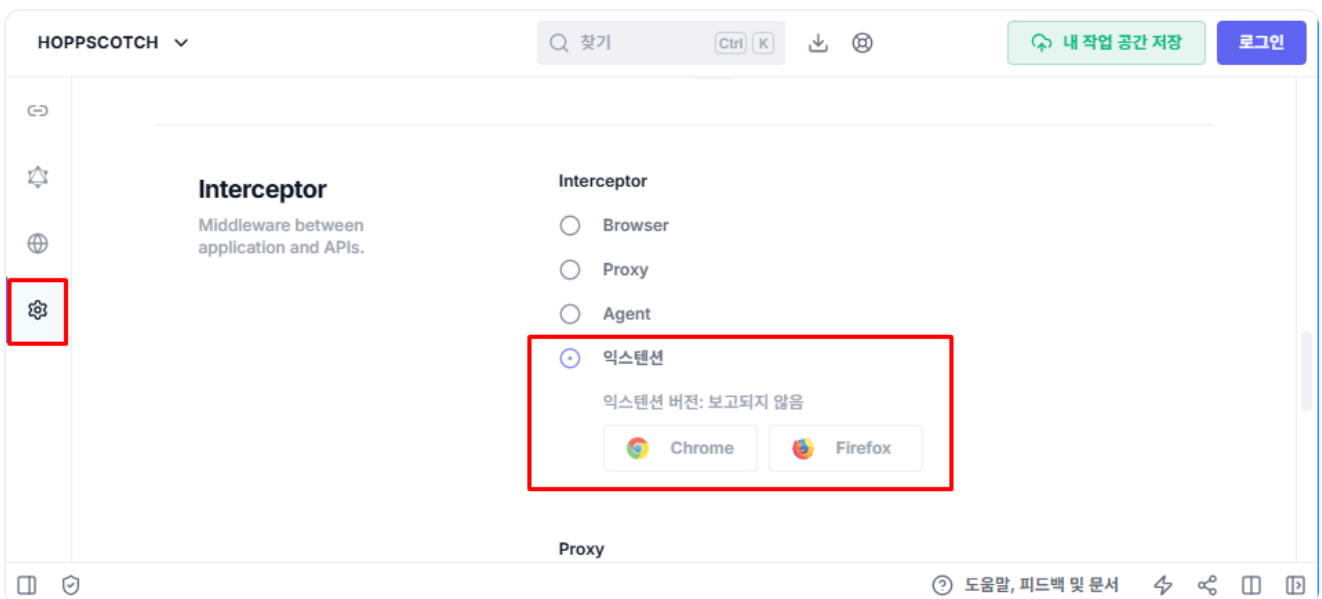


그림 2-33. Browser Extension 인터셉터 설정

확장 프로그램을 설치했다면 게시글 목록 API를 호출합니다.

Hoppscotch 게시글 목록 조회

GET <http://localhost:8080/api/boards>

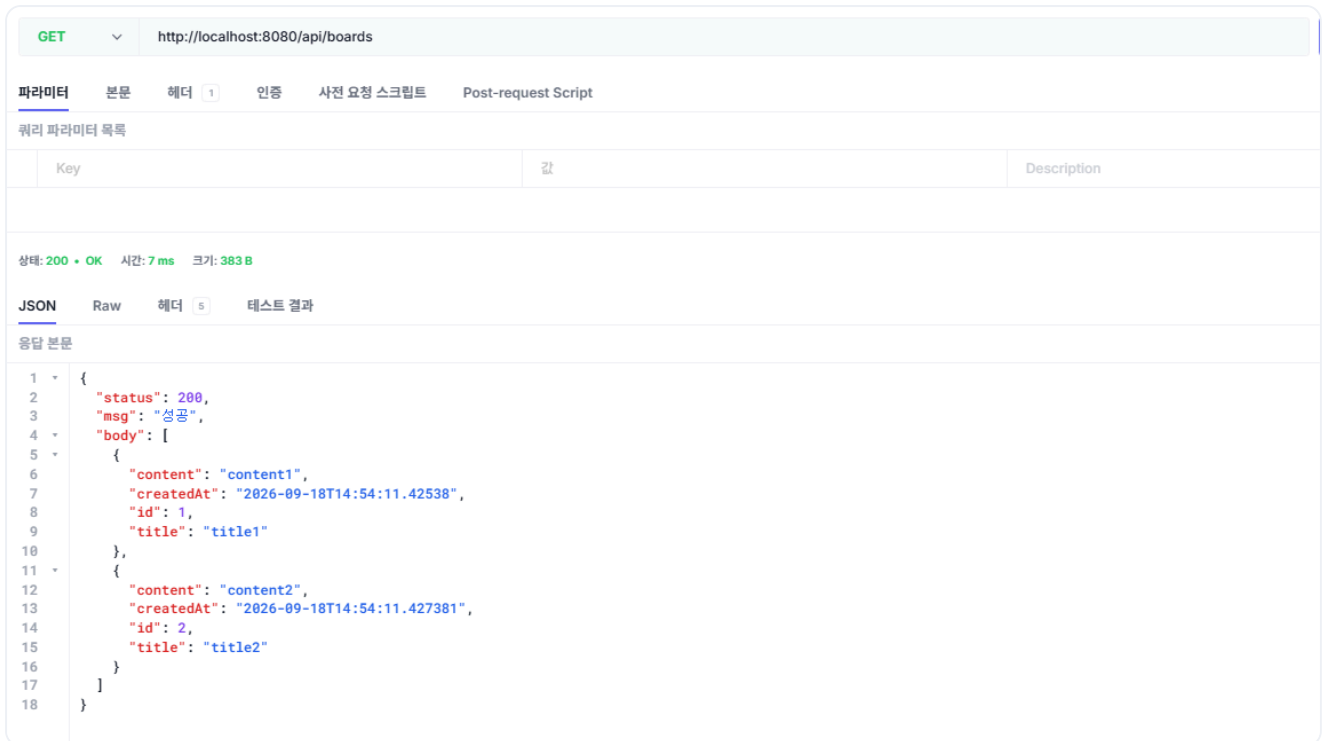


그림 2-34. 게시물 목록 응답

2.12 게시물 상세

2.12.1 서비스

게시글 상세 조회는 기본 키를 사용해 단일 데이터만 가져옵니다. 서비스에서 리포지토리의 `findById()` 를 호출하도록 `board/BoardService.java` 에 다음 메서드를 추가합니다.

실습 15 board/BoardService.java. 게시물 상세

```

public Board 게시물상세(Integer boardId) {
    return boardRepository.findById(boardId);
}

```

2.12.2 컨트롤러

컨트롤러는 URL 경로에 포함된 게시물 번호를 추출하여 서비스로 전달합니다. 경로 변수인 `{boardId}` 값은 `@PathVariable`을 사용해 가져옵니다.

`board/BoardController.java` 에 다음 메서드를 추가해 이를 구현합니다.

실습 16 board/BoardController.java. 게시물 상세

```
@GetMapping("/{boardId}")
public ResponseEntity<?> findById(@PathVariable("boardId") Integer boardId) {
    Board responseBoard = boardService.게시글상세(boardId);
    return Resp.ok(responseBoard);
}
```

서버를 실행한 뒤 1번 게시글의 상세 조회 API를 호출합니다.

Hoppscotch 게시물 상세 조회

GET <http://localhost:8080/api/boards/1>

GET <http://localhost:8080/api/boards/1>

파라미터 본문 헤더 1 인증 사전 요청 스크립트 Post-request Script

쿼리 파라미터 목록

Key	값	Description
-----	---	-------------

상태: 200 • OK 시간: 25 ms 크기: 293 B

JSON Raw 헤더 5 테스트 결과

응답 본문

```
1 {
2   "status": 200,
3   "msg": "성공",
4   "body": {
5     "content": "content1",
6     "createdAt": "2026-09-18T14:54:11.42538",
7     "id": 1,
8     "title": "title1"
9   }
10 }
```

그림 2-35. 게시물 상세 응답

2.13 게시물 추가

2.13.1 서비스

게시글 추가는 데이터를 변경하는 작업입니다. 중간에 오류가 발생하더라도 데이터에 잘못 반영되지 않도록, 메서드 전체를 하나의 트랜잭션으로 묶어 처리합니다.

`board/BoardService.java`에 아래와 같이 메서드를 추가합니다.

실습 17 board/BoardService.java. 게시물 추가

```
@Transactional
public Board 게시물추가(Board requestBoard) {
    boardRepository.save(requestBoard);
    return requestBoard;
}
```

@Transactional은 메서드가 실행되는 동안 트랜잭션을 열어 두었다가, 정상으로 끝나면 변경을 반영하고 중간에 예외가 나면 되돌립니다.

2.13.2 컨트롤러

클라이언트의 요청이 들어오면 디스패처 서블릿이 담당 컨트롤러 메서드를 찾아 호출합니다. 파라미터에 **@RequestBody**가 있다면, 요청 바디에 담긴 JSON 데이터를 자바 객체로 변환해 매개변수로 전달합니다.

`board/BoardController.java`에 아래와 같이 메서드를 추가합니다.

실습 18 board/BoardController.java. 게시물 추가

```
@PostMapping
public ResponseEntity<?> save(@RequestBody Board requestBoard) {
    Board responseBoard = boardService.게시물추가(requestBoard);
    return Resp.ok(responseBoard);
}
```

저장한 게시물을 응답합니다. 클라이언트가 다시 조회하지 않아도 저장된 결과를 바로 확인할 수 있습니다.

제목과 내용을 요청 바디에 담아 게시물 추가 API를 호출합니다.

Hoppscotch 게시물 추가

```
POST http://localhost:8080/api/boards

{
  "title": "title3",
  "content": "content3"
}
```

```
}
```

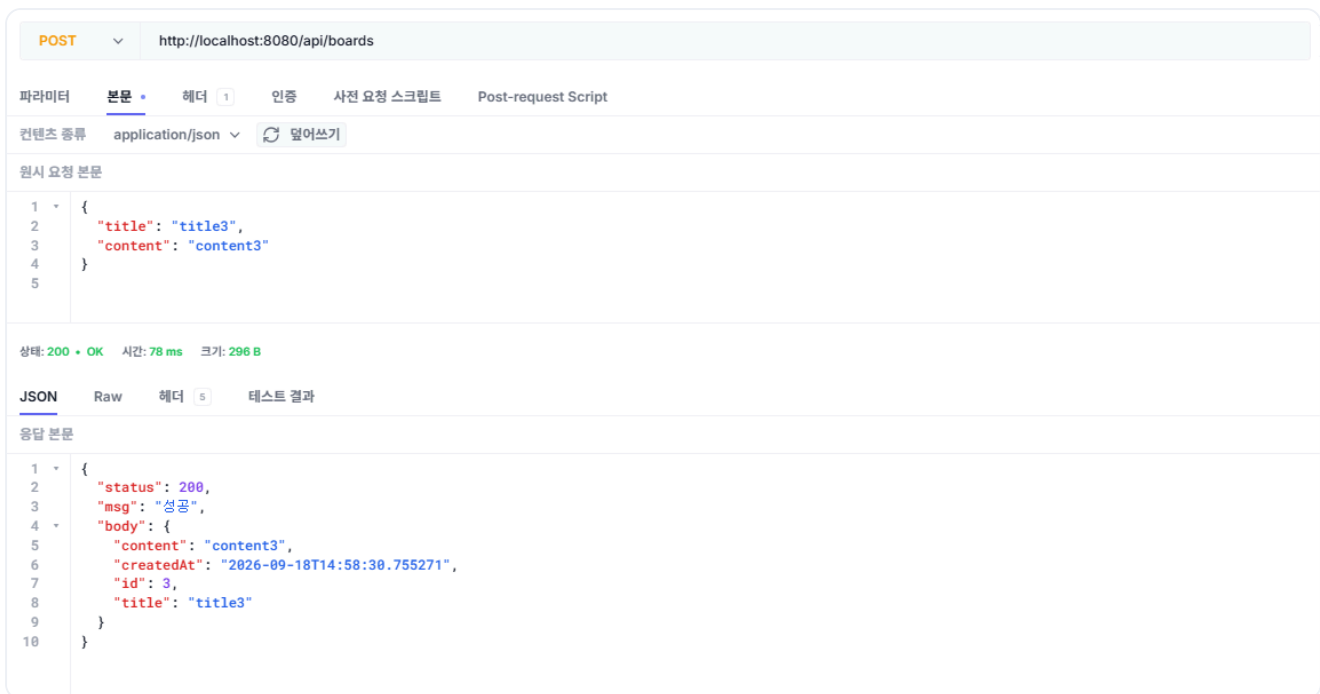


그림 2-36. 게시물 추가 응답

2.14 게시물 수정

2.14.1 서비스

`게시글수정()` 은 URL에 담긴 게시물 번호와 요청 바디로 들어온 값을 함께 전달받습니다.

`board/BoardService.java` 에 아래와 같이 메서드를 추가합니다.

실습 19 board/BoardService.java. 더티체킹으로 수정

```
@Transactional
public Board 게시물수정(Integer boardId, Board requestBoard) {
    // 1. 수정할 게시글을 조회해 영속 상태로 가져온다
    Board board = boardRepository.findById(boardId);
    // 2. 값만 바꾼다. save() 호출이 없다
    board.setTitle(requestBoard.getTitle());
    board.setContent(requestBoard.getContent());
    return board;
} // 트랜잭션이 끝나는 이 지점에서 변경이 반영된다
```

값을 변경하기만 했는데도, **@Transactional**이 끝날 때 `flush()`가 스냅샷과 현재 값을 비교해 달라진 엔티티를 UPDATE로 내보냅니다.

2.14.2 컨트롤러

`board/BoardController.java`에 아래와 같이 메서드를 추가합니다.

실습 20 board/BoardController.java. 게시물 수정

```
@PutMapping("/{boardId}")
public ResponseEntity<?> update(@PathVariable("boardId") Integer boardId,
                                @RequestBody Board requestBoard) {
    Board responseBoard = boardService.게시글수정(boardId, requestBoard);
    return Resp.ok(responseBoard);
}
```

바꿀 제목과 내용을 요청 바디에 담아 1번 게시글의 수정 API를 호출합니다.

Hoppscotch 게시물 수정

```
PUT http://localhost:8080/api/boards/1

{
  "title": "title-update",
  "content": "content-update"
}
```

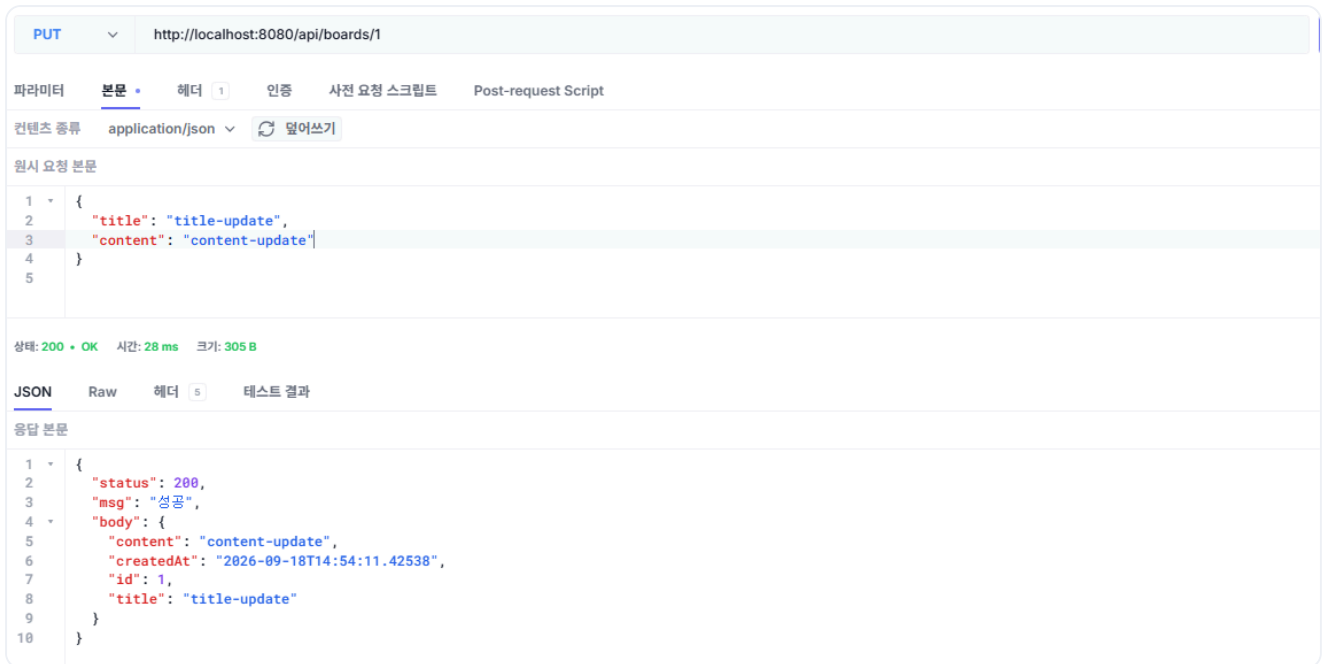


그림 2-37. 게시물 수정 응답

2.15 게시물 삭제

2.15.1 서비스

삭제는 지울 엔티티를 먼저 조회해 리포지토리에 전달합니다.

`board/BoardService.java`에 아래와 같이 메서드를 추가합니다.

실습 21 `board/BoardService.java`. 게시물 삭제

```
@Transactional
public void 게시물삭제(Integer boardId) {
    Board board = boardRepository.findById(boardId);
    boardRepository.delete(board);
}
```

2.15.2 컨트롤러

`board/BoardController.java`에 아래와 같이 메서드를 추가합니다.

실습 22 board/BoardController.java. 게시물 삭제

```
@DeleteMapping("/{boardId}")
public ResponseEntity<?> deleteById(@PathVariable("boardId") Integer boardId) {
    boardService.게시글삭제(boardId);
    return Resp.ok(null);
}
```

삭제는 반환할 데이터가 없으므로 `Resp.ok(null)` 로 성공만 응답합니다.

1번 게시글의 삭제 API를 호출합니다.

Hoppscotch 게시물 삭제

DELETE <http://localhost:8080/api/boards/1>

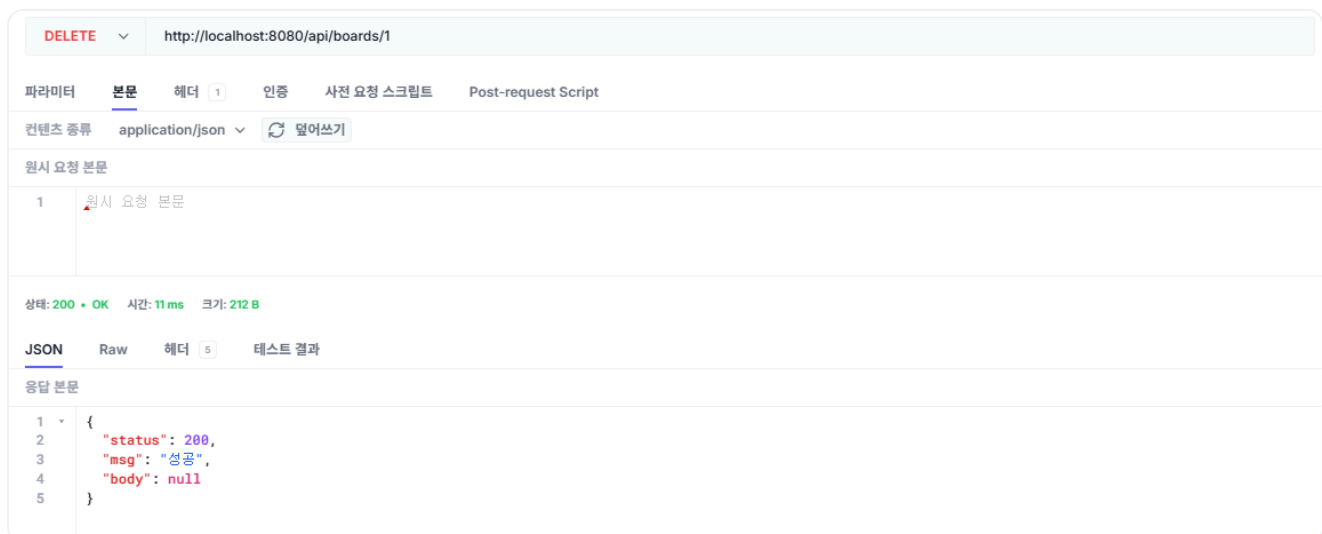


그림 2-38. 게시물 삭제 응답

스프링 프레임워크의 웹 계층 구조를 활용해 게시판의 기본 기능을 완성했습니다. 데이터베이스와 통신하는 리포지토리, 비즈니스 로직을 품은 서비스, 클라이언트의 요청을 받는 컨트롤러까지, 스프링의 구성 요소들이 어떻게 맞물려 하나의 애플리케이션으로 동작하는지 확인해 보았습니다.

다음 챕터에서는 클라이언트와 주고받을 데이터를 DTO로 분리하여 구조를 개선하고, 잘못된 조회 요청에 대비한 예외 처리를 다룹니다.

이것만은 기억하자

- **JPA는 객체와 테이블 사이를 변환합니다.** 테이블이 아니라 엔티티와 필드 이름으로 JPQL을 작성하면 JPA가 이를 SQL로 번역해 데이터베이스에 전달합니다.
- **영속성 컨텍스트는 조회한 엔티티를 트랜잭션 동안 관리합니다.** 같은 엔티티를 다시 조회하면 캐시에 있는 것을 반환하고, 쿼리는 버퍼에 모았다가 `flush()` 시점에 내보냅니다.
- **애플리케이션은 컨트롤러, 서비스, 리포지토리 세 계층으로 나눕니다.** 컨트롤러는 요청을 받고, 서비스는 트랜잭션 내에서 비즈니스 로직을 처리하며, 리포지토리는 데이터베이스를 다룹니다.

CHAPTER

3

예외 처리와 DTO

실패를 응답으로 돌려주다

이번 챕터가 끝나면

- **DTO**로 엔티티를 그대로 내보내지 않고 요청과 응답 전용 클래스를 만듭니다.
- **JpaRepository**로 기본 조회, 저장, 삭제 메서드를 물려받고 결과를 **Optional**로 받습니다.
- **커스텀 예외**와 **전역 예외 처리**로 실패 상황을 상태 코드에 맞는 JSON 응답으로 바꿉니다.

챕터 3. 예외 처리와 DTO

게시판의 기본 기능이 완료되었습니다. 오픈이는 뿌듯한 마음으로 선배에게 화면을 보여줬습니다.

선배는 목록과 상세 조회를 한 번씩 눌러보더니, 주소창 끝에 있는 게시글 번호를 슬쩍 **999**로 바꿨습니다.

선배 : "없는 번호예요. 이런 예외 상황도 테스트해 봤어요?"

엔터를 치자 응답 창에는 여전히 **200 성공**이 떴습니다. 하지만 응답 바디는 비어 있었습니다.

게시글이 없는 번호인데 왜 응답이 성공으로 나오지?

선배 : "데이터베이스에서 게시글을 못 찾으면 빈 값으로 응답이 오는데, 이 빈 값을 그대로 응답에 담아버리니 성공으로 응답이 뜨는 거예요. 만약 없는 게시글을 수정이나 삭제하려고 한다면 오류가 발생하겠죠. 서버에서는 이런 예외(Exception)가 발생했을 경우를 위해 미리 예외 처리를 해야 해요. 그래야 어떤 사유로 요청이 실패했는지 클라이언트에게 명확히 알려줄 수 있어요."

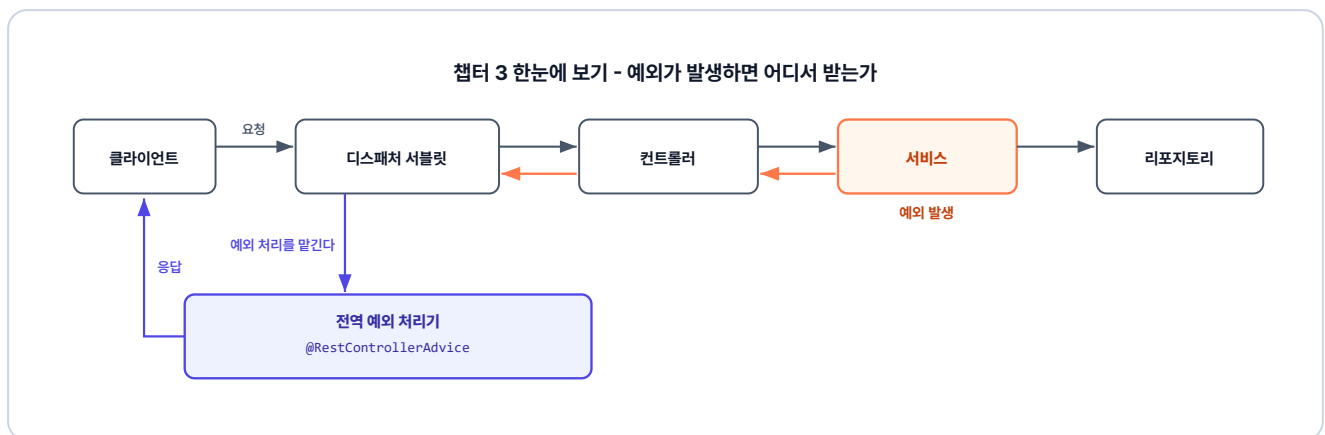


그림 3-1. 챕터 3의 실습 흐름

소스코드 준비

소스코드 준비에서 클론한 예제 저장소에서 이번 챕터 폴더로 이동합니다. 패키지 루트는 챕터 2와 같은 `com.metacoding.spring` 입니다.

터미널 챕터 3 폴더로 이동

```
cd spring-start/ch03
```

이번 챕터에서 새로 만들거나 고치는 파일은 다음과 같습니다. 나머지는 챕터 2 그대로입니다.

ch03 파일 구조

```
spring-start/ch03/src/main/java/com/metacoding/spring/
├─ board/
│   ├── Board.java                # [작성] @Builder + 생성자 추가
│   ├── BoardController.java      # [작성] 요청·응답 타입 DTO로 교체
│   ├── BoardRequest.java         # [작성] 요청 DTO(SaveDTO/UpdateDTO) + toEntity()
│   ├── BoardRepository.java      # [작성] JpaRepository 인터페이스로 전환
│   ├── BoardResponse.java        # [작성] 응답 DTO(DTO/DetailDTO)
│   └── BoardService.java         # [작성] orElseThrow + DTO 반환
├─ core/handler/
│   ├── ex/Exception400,401,403,404,500.java # [작성] 커스텀 예외
│   └── GlobalExceptionHandler.java          # [작성] 전역 예외 처리
```

챕터를 따라 코드를 채우고, 막히면 `spring-end` 의 완성 코드를 참고하세요.

3.1 요청 DTO

챕터 2의 컨트롤러는 요청을 **Board** 엔티티로 직접 받고, 응답할 때도 엔티티를 그대로 반환합니다. 하지만 실무에서는 엔티티를 요청이나 응답에 직접 사용하지 않습니다. 엔티티는 데이터베이스의 테이블 구조와 동일해서, 그대로 사용할 경우 응답에 불필요한 데이터가 노출되거나 테이블 구조가 변경될 때 예상치 못한 오류가 발생할 수 있기 때문입니다. 따라서 클라이언트와 주고받을 때는 필요한 데이터만 담아서 전달할 전용 객체를 사용해야 합니다.

3.1.1 DTO

계층 사이에서 데이터 전달만을 목적으로 하는 객체를 **DTO(Data Transfer Object)** 라고 합니다. 컨트롤러와 서비스처럼 서로 다른 계층 사이에서 필요한 값만 담아 전달합니다. DTO를 사용하면 요청 받을 값과 응답을 보낼 값을 필요한 형태로 정할 수 있습니다.

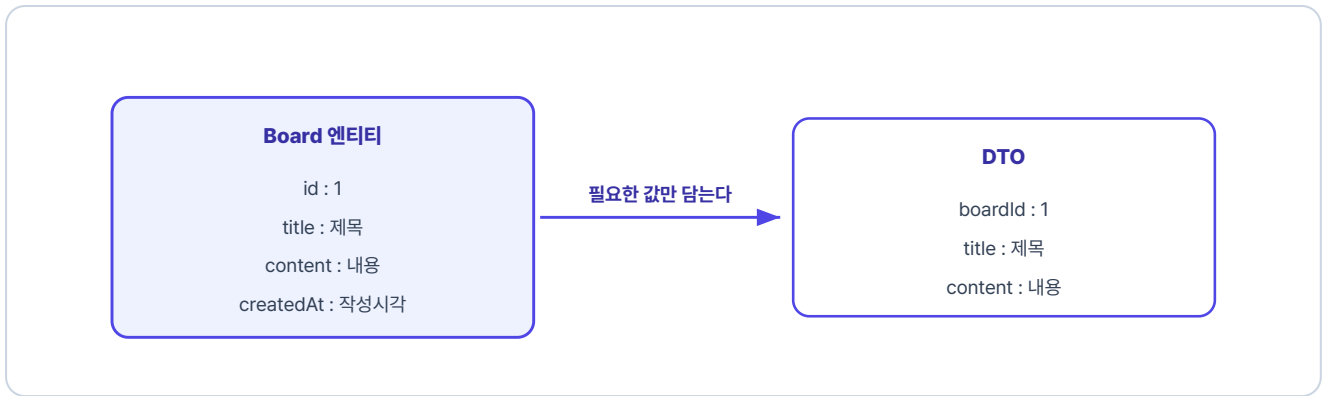


그림 3-2. 엔티티와 DTO가 담는 값

3.1.2 엔티티 생성자

데이터베이스에 저장되는 대상은 영속성 컨텍스트가 관리하는 엔티티입니다. 따라서 요청으로 받은 DTO를 데이터베이스에 저장하려면 엔티티로 변환하는 과정이 필요합니다. 이를 처리할 수 있도록 먼저 **Board** 클래스에 생성자를 추가합니다.

실습 1 board/Board.java. 빌더로 생성할 수 있게

```
@NoArgsConstructor // 기본 생성자 추가
@Data
@Entity
@Table(name = "board_tb")
public class Board {
    // id, title, content, createdAt 필드 (챕터 2와 같음)

    @Builder // 객체 생성 용도
    public Board(Integer id, String title, String content, LocalDateTime createdAt) {
        this.id = id;
        this.title = title;
        this.content = content;
        this.createdAt = createdAt;
    }
}
```

새로운 생성자를 직접 추가하면 자바가 기본으로 제공하던 기본 생성자가 사라집니다. 하지만 JPA 엔티티는 기본 생성자가 반드시 있어야 하므로, **@NoArgsConstructor** 어노테이션을 함께 붙여줍니다.

빌더 패턴(Builder Pattern)이란?

객체를 생성할 때 순서에 의존하지 않고, 어떤 필드에 어떤 값이 들어가는지 명시적으로 지정하는 패턴입니다.

@Builder를 사용하면 복잡한 매개변수 순서를 외울 필요 없이, 내가 원하는 필드의 값을 채워 넣을 수 있습니다.

3.1.3 요청 DTO 만들기

요청 DTO는 클라이언트가 전송한 데이터를 담는 클래스입니다. 클라이언트가 데이터를 보내면, 스프링이 요청 바디에 담긴 값들을 이름이 일치하는 DTO 필드에 자동으로 채워줍니다. 덕분에 컨트롤러는 복잡한 변환 과정 없이 이 DTO를 매개변수로 받아 바로 사용할 수 있습니다.

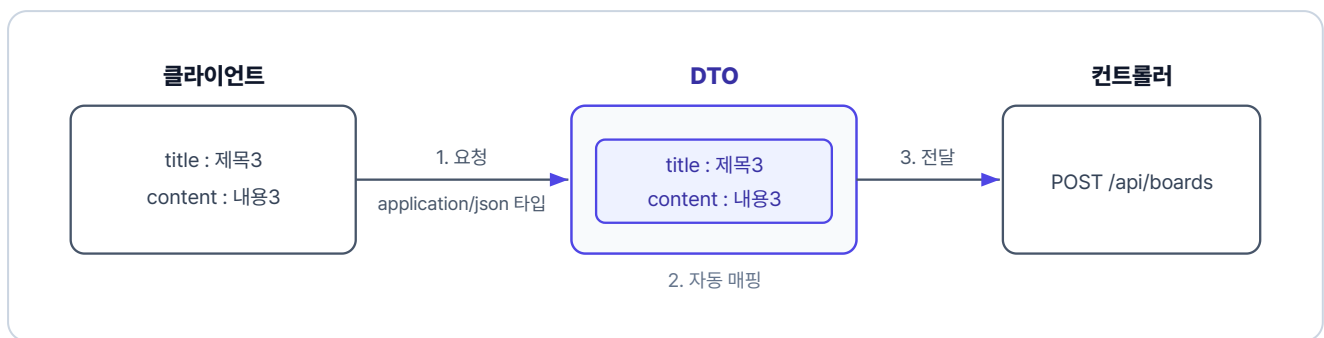


그림 3-3. 요청 데이터와 DTO

SaveDTO는 값만 담은 일반 객체라 영속 상태가 되지 못합니다. 그래서 DTO 안에 엔티티로 바꾸는 `toEntity()` 를 추가합니다.

`board/BoardRequest.java` 를 열어 아래와 같이 작성합니다.

실습 2 board/BoardRequest.java. 요청 DTO와 엔티티 변환

```
public class BoardRequest {
    public record SaveDTO(String title, String content) {

        // DTO를 엔티티로 변환한다
        public Board toEntity() {
            return Board.builder()
                .title(title)
                .content(content)
                .build();
        }
    }

    public record UpdateDTO(String title, String content) {
    }
}
```

```
}
```

`record`는 데이터 전달을 목적으로 하는 특수 클래스입니다. 필드만 선언하면 생성자와 getter 같은 메서드를 자동으로 만들어 줍니다.

3.2 응답 DTO

응답 DTO는 클라이언트에게 내보낼 값만 담는 클래스입니다. 서비스가 조회한 엔티티를 그대로 반환하지 않고, 이 클래스로 옮겨 담아 반환합니다.

`board/BoardResponse.java`를 열어 아래와 같이 작성합니다.

실습 3 board/BoardResponse.java. 응답 DTO

```
public class BoardResponse {

    // 1. 목록용 DTO. 엔티티를 받아 보여줄 값만 담는다
    public record DTO(Integer boardId, String title, String content) {

        public DTO(Board board) {
            this(board.getId(), board.getTitle(), board.getContent());
        }
    }

    // 2. 상세용 DTO. 지금은 목록용과 같지만 앞으로 달라진다
    public record DetailDTO(Integer boardId, String title, String content) {

        public DetailDTO(Board board) {
            this(board.getId(), board.getTitle(), board.getContent());
        }
    }
}
```

응답 DTO는 기능마다 따로 만드는 것이 좋습니다. 다만 이 책에서는 편의를 위해 공통 DTO를 사용하고, 게시글 상세만 별도의 DTO를 만듭니다.

3.3 JpaRepository 적용

챕터 2의 리포지토리는 **EntityManager**를 주입받아 저장과 조회, 삭제 메서드를 직접 작성했습니다. 이 기본 동작들은 **JpaRepository**에 이미 들어 있어, 이 인터페이스를 상속하면 스프링이 대신 구현해 줍니다. 메서드를 적지 않아도 저장과 조회, 삭제를 호출할 수 있습니다.

`board/BoardRepository.java`를 열어 아래와 같이 변경합니다.

실습 4 board/BoardRepository.java. JpaRepository 상속

```
public interface BoardRepository extends JpaRepository<Board, Integer> {  
}
```

리포지토리는 클래스가 아니라 인터페이스로 선언합니다. `JpaRepository`를 상속할 때 다른 엔티티와 기본 키의 타입을 차례로 적습니다.

또한 조회 결과를 **Optional** 타입으로 반환할 수 있습니다. **Optional**은 값이 존재할 수도, 존재하지 않을 수도 있는 상태를 감싸는 래퍼(**Wrapper**) 클래스로, 주로 null로 인한 에러를 방지하기 위해 사용됩니다. 값이 없어도 null이 그대로 넘어가지 않으므로, 게시글을 찾지 못한 경우를 예외로 처리할 수 있습니다.

자주 쓰는 메서드는 다음과 같습니다.

메서드	하는 일
<code>save(엔티티)</code>	저장하고 저장된 엔티티를 반환합니다
<code>findById(기본 키)</code>	기본 키로 한 건을 조회해 Optional 에 담아 반환합니다
<code>findAll()</code>	전체를 List 로 반환합니다
<code>delete(엔티티)</code>	삭제합니다
<code>findBy필드명(값)</code>	직접 선언합니다. findBy 뒤 필드 이름을 보고 select문이 생성됩니다

3.4 예외 처리 추가

스프링에서 예외가 어떻게 처리되는지 흐름부터 알아보겠습니다.

3.4.1 예외 처리 흐름

클라이언트의 요청은 디스패처 서블릿을 통해 서비스 계층까지 전달됩니다. 만약 조회 결과가 없다면 서비스는 실패 사유를 담은 예외를 발생시킵니다. 예외가 발생하면 해당 메서드는 즉시 중단되고 그 아래 코드는 실행되지 않습니다.

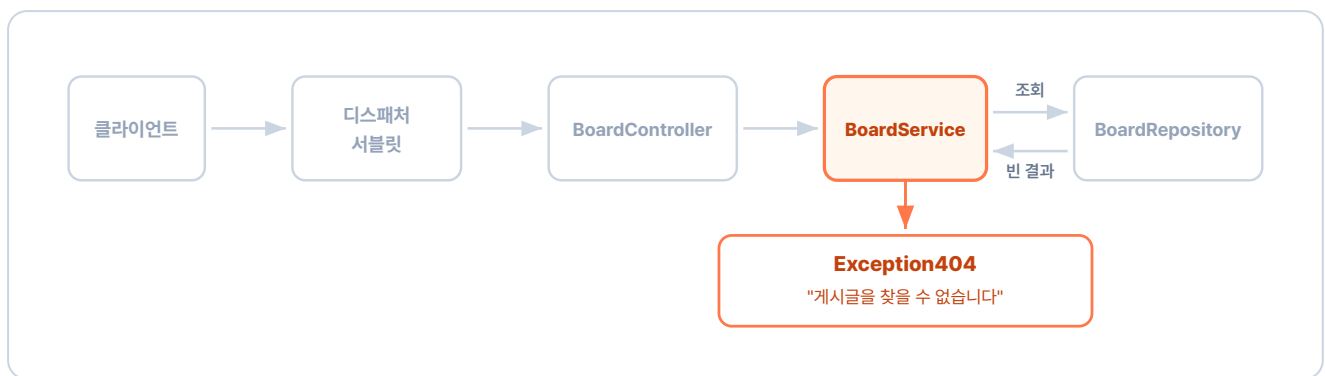


그림 3-4. 예외 발생 지점

중단된 메서드는 자신을 호출한 곳으로 예외를 넘깁니다. 넘겨받은 곳에서 다시 자신을 호출한 곳으로 넘기면서 예외는 디스패처 서블릿까지 전달됩니다.

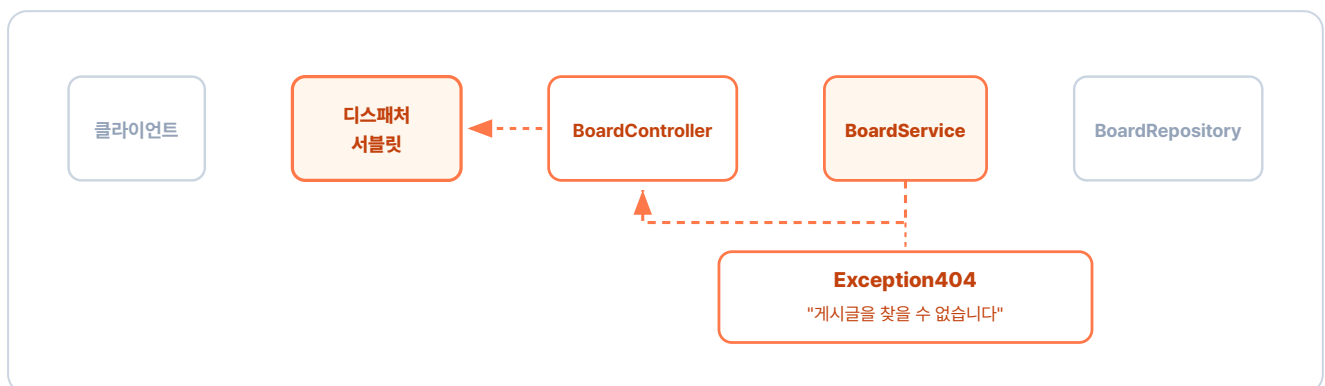


그림 3-5. 예외 전파 경로

디스패처 서블릿은 예외를 직접 처리하지 않습니다. 대신 발생한 예외의 타입에 맞는 메서드를 찾아 실행합니다. 이런 메서드들은 특정 컨트롤러에 속하지 않고 모든 컨트롤러에서 발생한 예외를 받으므로, 이들이 모인 클래스를 전역 예외 처리기라고 부릅니다.

실행된 메서드는 예외에 담긴 사유로 실패 응답을 만들어 반환합니다. 그 값이 디스패처 서블릿을 거쳐 클라이언트에게 응답으로 전달됩니다.

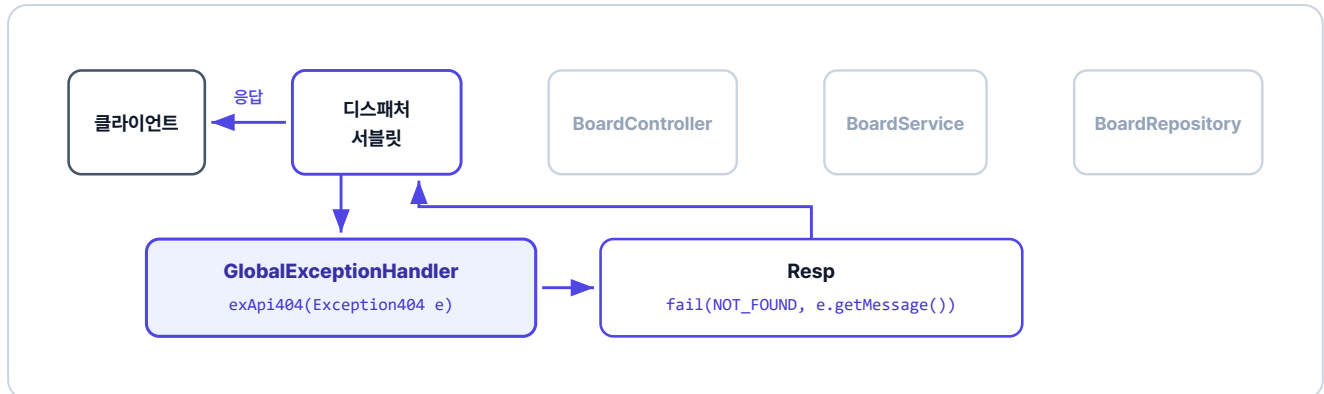


그림 3-6. 전역 예외 처리기의 응답

3.4.2 커스텀 예외 만들기

먼저 게시글을 찾지 못했을 때 발생시킬 예외 **Exception404**를 만듭니다. 서비스에서 이 예외를 발생시키면 전역 예외 처리기가 받아, 상태 코드 404와 실패 사유를 담은 응답을 클라이언트에게 반환합니다.

`core/handler/ex/Exception404.java`를 열어 아래와 같이 작성합니다.

실습 5 core/handler/ex/Exception404.java. 커스텀 예외

```

public class Exception404 extends RuntimeException {
    public Exception404(String message) {
        super(message);
    }
}
  
```

상황에 맞춰 직접 정의한 예외를 커스텀 예외라고 부릅니다. **RuntimeException**을 상속받아 구현하면, 예외가 발생하는 곳이나 메서드를 호출하는 곳 모두 `try-catch` 구문을 생략할 수 있습니다. 동일한 경로에 위치한 **Exception400**, **Exception401**, **Exception403**, **Exception500** 클래스 역시 이름만 다를 뿐 구조는 동일합니다.

이 책에서 사용하는 상태 코드는 다음과 같습니다.

상태 코드	뜻	예
400	처리할 수 없는 요청 값	제목 없이 게시글 작성

상태 코드	뜻	예
401	인증하지 않은 요청	로그인하지 않고 게시글 추가
403	인증했지만 권한이 없는 요청	다른 사람이 쓴 게시글 수정
404	찾을 수 없는 자원	없는 게시글 조회
500	서버 내부 오류	앞의 네 가지에 해당하지 않는 나머지 예외

3.4.3 전역 예외 처리

이제 앞에서 살펴본 전역 예외 처리기를 직접 만들어 보겠습니다. 클래스에 **@RestControllerAdvice**를 적용하면 그 클래스가 전역 예외 처리기가 됩니다. 이후 예외가 발생하면 디스패처 서블릿이 이 클래스 내부에서 예외 타입에 맞는 처리 메서드를 찾아 자동으로 실행합니다.

`core/handler/GlobalExceptionHandler.java`를 열어 아래와 같이 작성합니다.

실습 6 core/handler/GlobalExceptionHandler.java. 전역 예외 처리

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 1. 커스텀 예외는 저마다의 상태 코드로 바꾼다
    @ExceptionHandler(Exception400.class)
    public ResponseEntity<?> exApi400(Exception400 e) {
        return Resp.fail(HttpStatus.BAD_REQUEST, e.getMessage());
    }

    @ExceptionHandler(Exception401.class)
    public ResponseEntity<?> exApi401(Exception401 e) {
        return Resp.fail(HttpStatus.UNAUTHORIZED, e.getMessage());
    }

    @ExceptionHandler(Exception403.class)
    public ResponseEntity<?> exApi403(Exception403 e) {
        return Resp.fail(HttpStatus.FORBIDDEN, e.getMessage());
    }

    @ExceptionHandler(Exception404.class)
    public ResponseEntity<?> exApi404(Exception404 e) {
        return Resp.fail(HttpStatus.NOT_FOUND, e.getMessage());
    }
}
```

```

@ExceptionHandler(Exception500.class)
public ResponseEntity<?> exApi500(Exception500 e) {
    return Resp.fail(HttpStatus.INTERNAL_SERVER_ERROR, e.getMessage());
}

// 2. 나머지 모든 예외는 500으로 처리한다
@ExceptionHandler(Exception.class)
public ResponseEntity<?> exUnknown(Exception e) {
    return Resp.fail(HttpStatus.INTERNAL_SERVER_ERROR, "서버 오류가 발생했습니다");
}
}

```

3.5 서비스와 컨트롤러에 적용

3.5.1 서비스

이제 서비스에서 예외 처리를 해보겠습니다.

BoardService가 DTO로 값을 주고받고, 게시글이 없으면 예외를 발생시키도록 변경합니다.

`board/BoardService.java`를 열어 아래와 같이 작성합니다.

실습 7 board/BoardService.java. 주고받는 타입을 DTO로, 없으면 예외

```

public List<BoardResponse.DTO> 게시글목록() {
    return boardRepository.findAll().stream()
        .map(BoardResponse.DTO::new)
        .toList();
}

public BoardResponse.DetailDTO 게시글상세(Integer boardId) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    return new BoardResponse.DetailDTO(board);
}

@Transactional
public BoardResponse.DTO 게시글추가(BoardRequest.SaveDTO requestDTO) {
    Board savedBoard = boardRepository.save(requestDTO.toEntity()); // DTO -> 엔티
티

    return new BoardResponse.DTO(savedBoard); // 저장된 게시글 반환
}

```

```

@Transactional
public BoardResponse.DTO 게시글수정(Integer boardId, BoardRequest.UpdateDTO request
DTO) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    // 데이터 체크
    board.setTitle(requestDTO.title());
    board.setContent(requestDTO.content());
    return new BoardResponse.DTO(board); // 수정된 게시글 반환
}

@Transactional
public void 게시글삭제(Integer boardId) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    boardRepository.delete(board);
}

```

리포지토리가 반환한 **Optional**에서 값을 꺼낼 때는 `orElseThrow()` 를 사용합니다. 이 메서드는 값이 있으면 게시글을 그대로 반환하고, 비어 있으면 인자로 받은 예외를 발생시킵니다.

3.5.2 컨트롤러

컨트롤러도 엔티티 대신 DTO로 값을 주고받도록 변경합니다.

`board/BoardController.java` 를 열어 아래와 같이 작성합니다.

실습 8 board/BoardController.java. 요청·응답 타입을 DTO로

```

@GetMapping
public ResponseEntity<?> findAll() {
    List<BoardResponse.DTO> respDTOList = boardService.게시글목록();
    return Resp.ok(respDTOList);
}

@GetMapping("/{boardId}")
public ResponseEntity<?> findById(@PathVariable("boardId") Integer boardId) {
    BoardResponse.DetailDTO respDTO = boardService.게시글상세(boardId);
    return Resp.ok(respDTO);
}

@PostMapping
public ResponseEntity<?> save(@RequestBody BoardRequest.SaveDTO requestDTO) {
    BoardResponse.DTO respDTO = boardService.게시글추가(requestDTO);
    return Resp.ok(respDTO);
}

```

```

@PutMapping("/{boardId}")
public ResponseEntity<?> update(@PathVariable("boardId") Integer boardId,
    @RequestBody BoardRequest.UpdateDTO requestDTO) {
    BoardResponse.DTO respDTO = boardService.게시글수정(boardId, requestDTO);
    return Resp.ok(respDTO);
}

@DeleteMapping("/{boardId}")
public ResponseEntity<?> deleteById(@PathVariable("boardId") Integer boardId) {
    boardService.게시글삭제(boardId);
    return Resp.ok(null);
}

```

이제 프로젝트를 실행해 결과를 확인해 보겠습니다. 먼저 1번 게시글을 조회하면 정상적인 데이터를 응답받습니다.

Hoppscotch 게시글 상세 조회

GET <http://localhost:8080/api/boards/1>

The screenshot shows the Hoppscotch interface for a GET request to `http://localhost:8080/api/boards/1`. The response status is 200 OK, with a response time of 293 ms and a size of 260 B. The response body is a JSON object:

```

{
  "status": 200,
  "msg": "성공",
  "body": {
    "boardId": 1,
    "title": "title1",
    "content": "content1"
  }
}

```

그림 3-7. 상세 조회 응답

다음으로 예외를 발생시켜 보겠습니다. 더미 데이터에 없는 999번 게시글을 조회해 응답 결과를 확인합니다.

Hoppscotch 없는 게시글 다시 조회

GET <http://localhost:8080/api/boards/999>

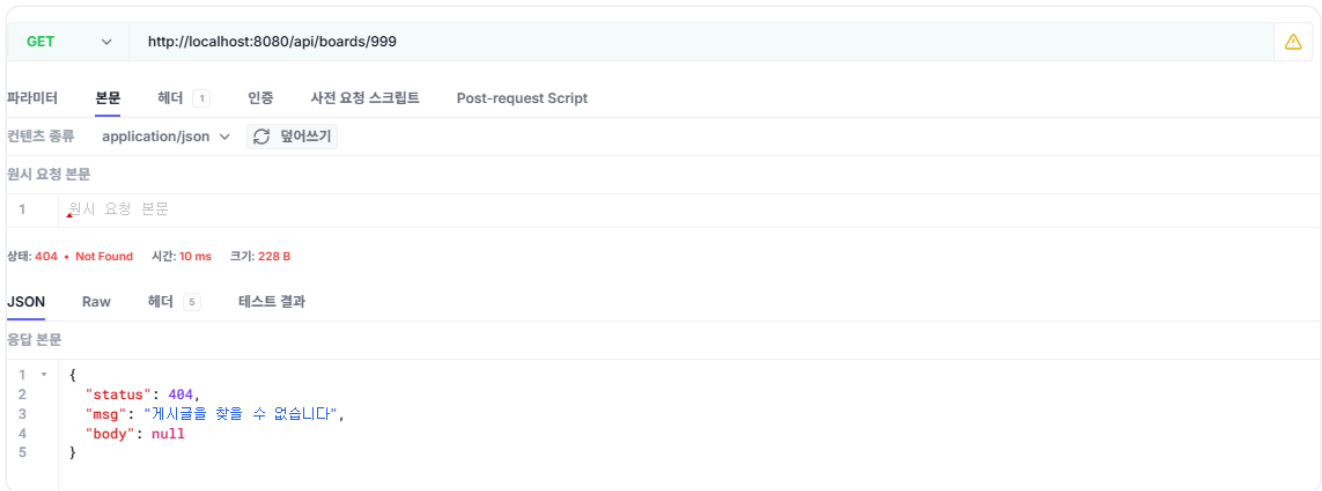


그림 3-8. 404 응답

챕터 2에서는 엔티티를 직접 요청과 응답에 사용했지만, 이번 챕터에서는 DTO를 활용해 데이터를 전달하는 구조로 개선했습니다. 또한 예외 처리를 추가하여, 처리 과정에서 문제가 발생하더라도 클라이언트가 상태 코드와 실패 사유를 응답받을 수 있습니다.

다음 챕터에서는 회원 엔티티를 추가해 게시글과 작성자를 연결하고, 이를 바탕으로 인증과 인가 기능을 구현해 보겠습니다.

이것만은 기억하자

- 엔티티를 요청과 응답에 직접 사용하지 않고 DTO로 주고받습니다. 테이블 구조가 바뀌어도 클라이언트와 주고받는 형식은 영향을 받지 않습니다.
- 요청이 실패하면 상태 코드와 사유를 담아 응답합니다. 서비스가 예외를 발생시키면 전역 예외 처리기가 한 곳에서 받아 상태 코드에 맞는 JSON으로 바꿉니다.

CHAPTER

4

인증과 인가

누구인지 확인하고 권한을 나누다

이번 챕터가 끝나면

- 인증과 인가의 차이를 이해합니다.
- JWT를 발급하고 서명을 검증해, 서버가 로그인 상태를 저장하지 않고도 사용자를 식별합니다.
- 1:N 관계로 회원과 게시글이 연결되는 구조를 이해합니다.

챕터 4. 인증과 인가

게시글 기능이 완성되었으니, 다음은 회원을 추가해 **인증 게시판**을 만듭니다.

오픈이는 회원 테이블을 만들고 회원가입과 로그인 기능을 구상했습니다. 하지만 로그인이 성공한 이후가 문제였습니다. 게시글의 작성자와 요청을 보낸 사용자가 같은지를 어떻게 구분할지 떠오르지 않았습니다.

오픈이는 선배를 찾아갔습니다.

오픈이 : "선배님, 로그인은 아이디와 비밀번호를 확인하면 되는데, 그 이후에 들어오는 요청은 서버가 누가 보냈는지 어떻게 아나요?"

선배 : "서버는 기본적으로 요청만 처리할 뿐, 누가 보냈는지는 기억하지 않아요. 그래서 로그인에 성공한 사용자에게 인증 수단을 줘야 해요. 클라이언트가 이후 요청을 보낼 때마다 이걸 함께 보내면, 서버는 그걸 보고 누군지 식별하는 거예요. 그래야 글을 수정하거나 삭제할 때 작성자 본인이 맞는지 검증할 수 있거든요."

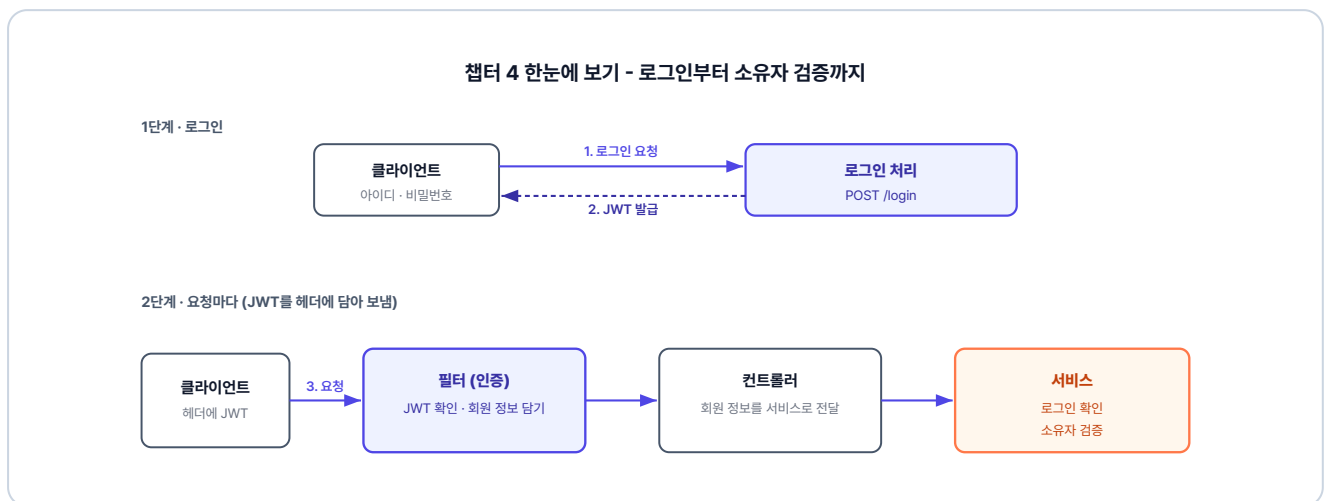


그림 4-1. 챕터 4의 실습 흐름

소스코드 준비

소스코드 준비에서 클론한 예제 저장소에서 이번 챕터 폴더로 이동합니다. 패키지 루트는 앞 챕터와 같은 `com.metacoding.spring` 입니다.

터미널 챕터 4 폴더로 이동

```
cd spring-start/ch04
```

새로 만들거나 고치는 파일은 다음과 같습니다.

ch04 파일 구조

```
spring-start/ch04/src/main/
├─ java/com/metacoding/spring/
│   └─ board/
│       ├── Board.java                # [작성] 작성자 연관관계
│       ├── BoardController.java      # [작성] 로그인 유저 꺼내 전달
│       └── BoardRepository.java      # [작성] 작성자를 함께 가져오는 조
회
│   └─ BoardRequest.java              # [작성] 작성자를 연결하는 toEnti
ty
│   └─ BoardResponse.java             # [작성] 상세 응답에 작성자 · 본인
여부
│   └─ BoardService.java              # [작성] 로그인 확인 · 소유자 검증
│   └─ core/
│       ├── filter/JwtAuthenticationFilter.java # [참고] 인증 필터
│       └─ util/
│           ├── JwtProvider.java       # [참고] 토큰 추출·유저 복원
│           └─ JwtUtil.java           # [참고] JWT 발급·검증
│   └─ user/
│       ├── User.java                 # [작성] 회원 엔티티
│       ├── UserController.java       # [작성] /join, /login
│       ├── UserRepository.java       # [작성] 유저네임으로 조회
│       ├── UserRequest.java          # [작성] 가입·로그인 요청 DTO
│       ├── UserResponse.java         # [작성] 가입 응답 DTO
│       └─ UserService.java           # [작성] 회원가입·로그인(JWT)
└─ resources/db/data.sql              # [참고] 회원·게시글 더미 데이터
```

챕터를 따라 코드를 채우고, 막히면 `spring-end`의 완성 코드를 참고하세요.

4.1 인증과 인가

지금까지 만든 게시판에는 게시글을 작성한 회원 정보가 없습니다. 회원 기능을 추가하면 서버는 요청을 보낸 사용자가 누구인지 검증해야 합니다.

이를 회사의 보안 시스템에 비유해 보겠습니다. 우리가 출근할 때 게이트에 사원증을 찍어 이 회사의 직원임을 증명하듯, 서버에서 요청을 보낸 사람의 신원을 확인하는 과정을 **인증(Authentication)** 이라고 합니다.

하지만 로비를 통과했다고 해서 회사의 모든 곳에 들어갈 수 있는 것은 아닙니다. 자료실처럼 권한을 부여받은 사람만 출입할 수 있는 곳이 따로 있습니다. 서버도 마찬가지입니다. 인증을 마친 사용자라도 해당 동작을 수행할 자격이 있는지 확인해야 하며, 이 과정을 **인가(Authorization)** 라고 합니다.

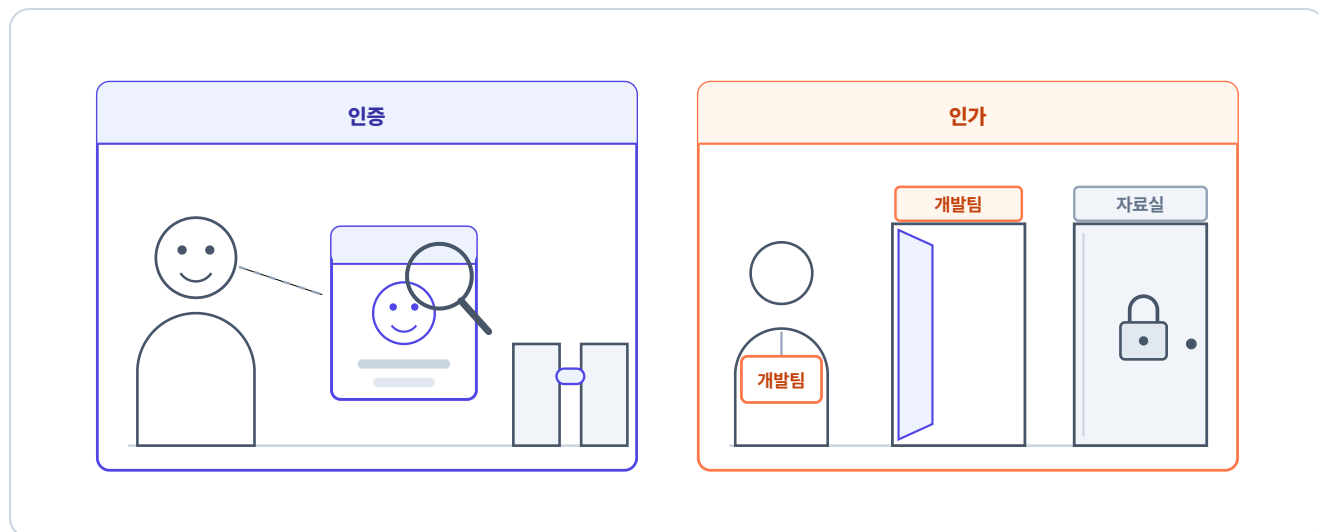


그림 4-2. 인증과 인가의 차이

이처럼 안전한 시스템을 구축하려면 서버가 요청마다 인증과 인가를 검증해야 합니다. 서버가 수많은 사용자의 요청을 매번 검증하기 위해 사용하는 대표적인 기술이 **토큰(Token)** 기반 인증입니다.

4.2 JWT

4.2.1 토큰 인증

토큰을 활용한 인증은 두 단계로 이뤄집니다. 먼저 클라이언트가 서버에 인증을 요청해 성공하면, 서버는 토큰을 발급하여 클라이언트에 전달하고, 클라이언트는 이 토큰을 자신의 기기에 저장합니다.

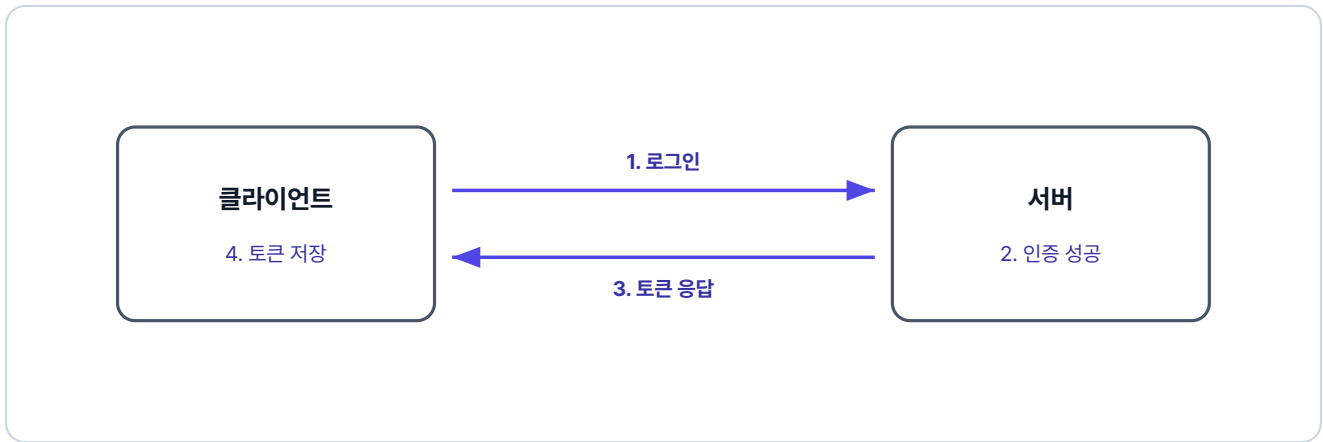


그림 4-3. 토큰 발급

로그인 이후 클라이언트는 요청 시 발급받은 토큰을 함께 전달하며, 서버는 해당 토큰을 검증한 뒤 유효할 경우 요청에 응답합니다.

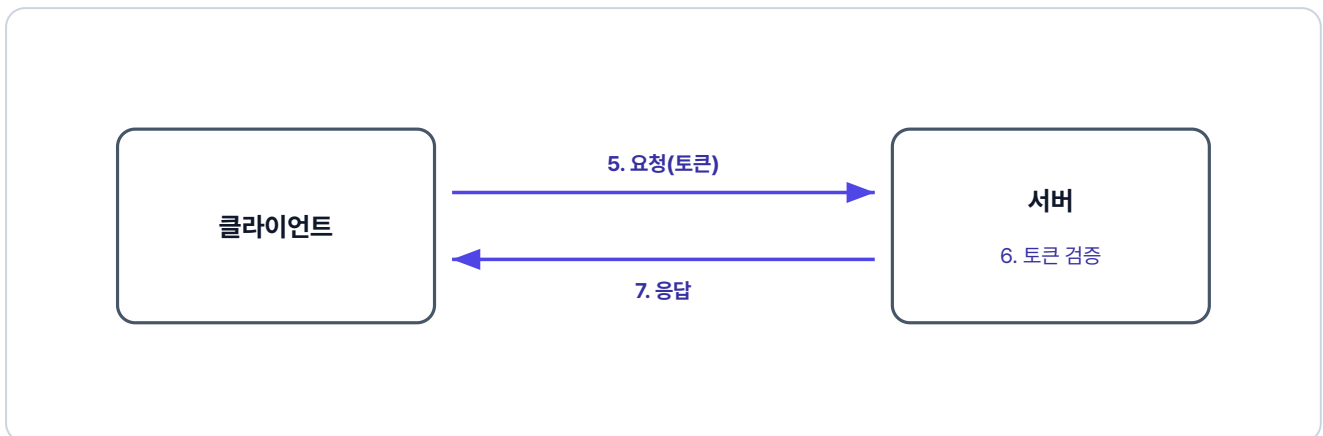


그림 4-4. 토큰 검증

서버가 토큰을 검증하는 방법은 토큰의 종류에 따라 다릅니다. 일반적으로 토큰은 무작위로 만들어진 문자열이라, 서버는 값을 검증하기 위해 발급한 토큰을 저장하고 있어야 합니다. 반면 **JWT(JSON Web Token)**는 내부에 사용자 정보를 보관하고 있습니다. 그래서 서버에 저장하지 않고 JWT 검증만으로도 사용자를 식별할 수 있습니다.

이처럼 서버가 사용자의 상태나 정보를 따로 기억해 두지 않고, 매번 요청이 들어올 때마다 새롭게 검증하는 방식을 **무상태(Stateless)**라고 부릅니다.

4.2.2 상태 유지와 무상태

그렇다면 서버가 사용자 정보를 기억하는 것과 그렇지 않은 것은 실제 시스템 환경에서 어떤 차이를 만들까요?

로드밸런서를 두고 클라이언트의 요청을 여러 대의 서버로 나누어 처리한다고 가정해 보겠습니다. 서버가 로그인 정보를 보관하는 **상태 유지(Stateful)** 에서는 인증 정보가 특정 서버에 종속되기 때문에, 그 서버만 요청을 정상적으로 처리할 수 있습니다. 로드밸런서가 부하 분산을 위해 다른 서버로 요청을 전달하면, 그 서버에는 인증 정보가 없으므로 요청이 실패합니다.

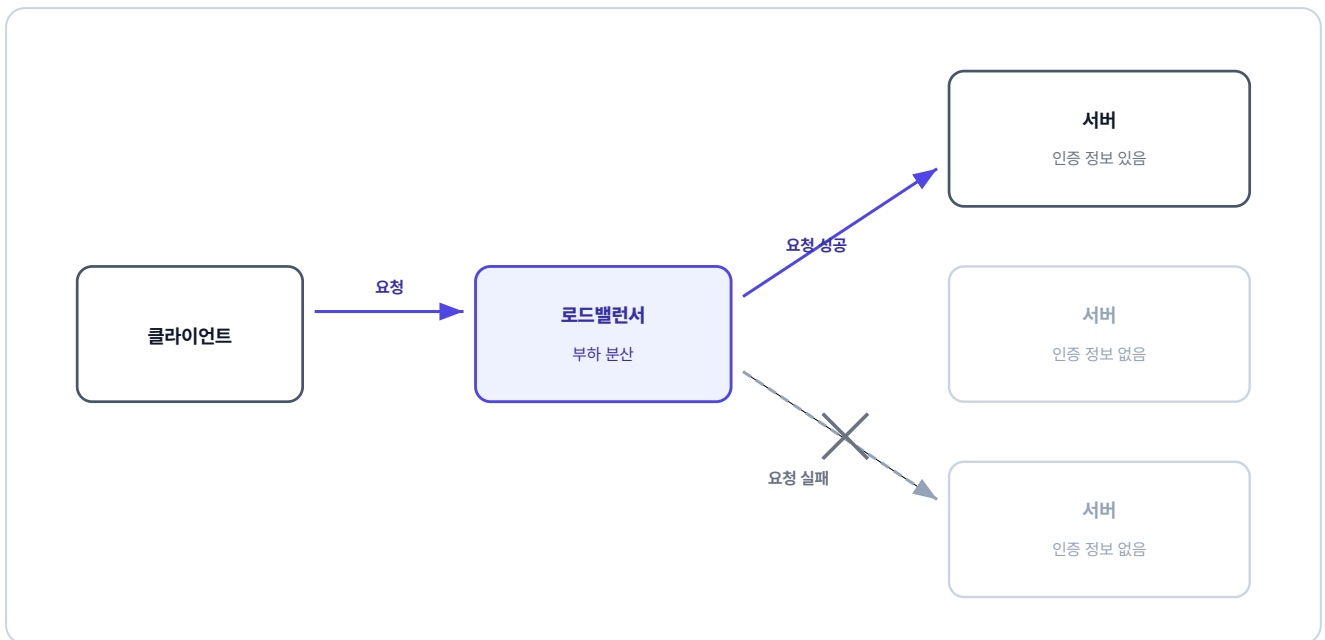


그림 4-5. 상태 유지 방식의 요청 실패

반면 **무상태(Stateless)** 방식에서는 서버가 클라이언트로부터 전달받은 JWT만 검증하면 되므로, 로드밸런서가 어떤 서버로 요청을 전달하더라도 동일하게 인증이 처리되어 정상적인 응답을 받을 수 있습니다.

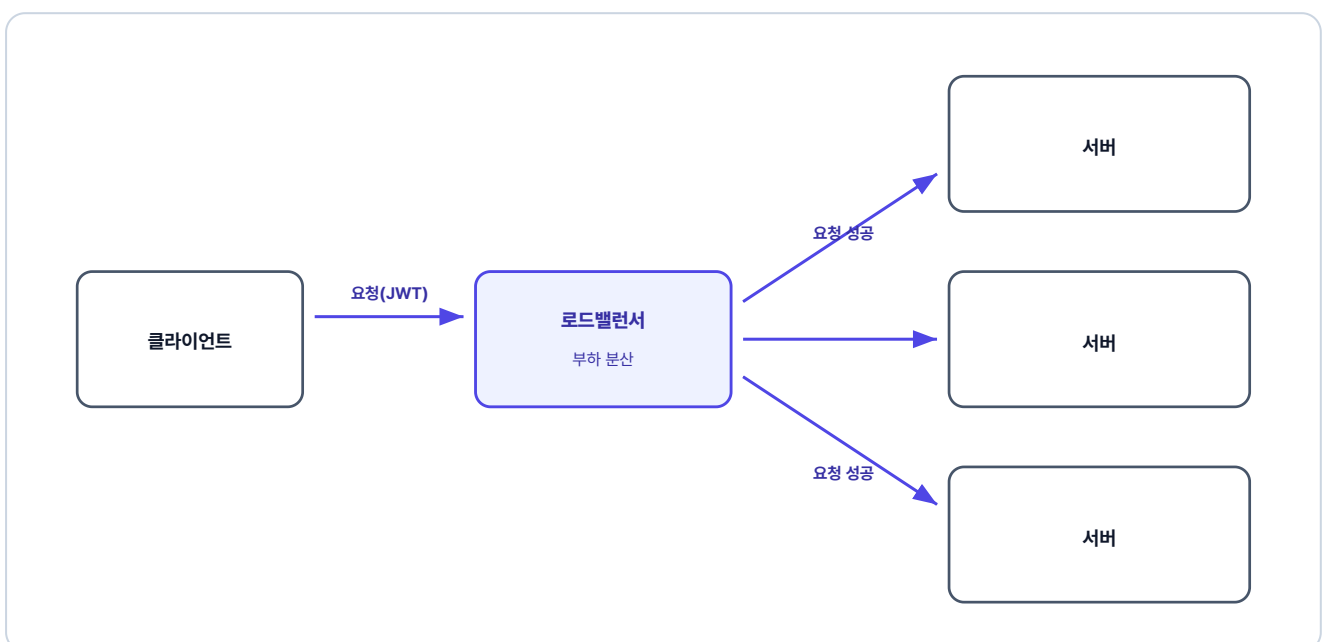


그림 4-6. 무상태 방식의 요청 처리

세션과 쿠키

상태 유지(Stateful) 방식의 대표적인 예는 웹에서 널리 쓰이는 **세션(Session)**입니다.

서버는 로그인한 사용자마다 세션이라는 전용 보관 공간을 만들고, 이 공간을 가리키는 번호를 클라이언트에 전달합니다. 웹 브라우저는 이 번호를 **쿠키(Cookie)**에 저장해 두었다가 서버에 요청을 보낼 때마다 함께 전송하고, 서버는 이 번호를 대조해 사용자를 확인합니다. 단, 이 구조는 동시 접속자가 늘어날수록 서버가 유지해야 할 세션도 함께 늘어나 서버에 큰 부담을 준다는 단점이 있습니다.

4.2.3 JWT의 구조

이번에는 JWT의 구조를 알아보겠습니다. JWT는 점(.)을 기준으로 세 부분으로 나뉘며, 각각 **헤더(Header)**, **페이로드(Payload)**, **서명(Signature)**을 의미합니다.



그림 4-7. JWT의 구성 요소

각 구성 요소의 특징은 다음과 같습니다.

구성 요소	설명
헤더	JSON을 Base64로 인코딩한 값 typ : 이 문자열이 JWT라는 표시 alg : 서명에 사용한 알고리즘 (HS512)
페이로드	JSON을 Base64로 인코딩한 값 sub : JWT 주인의 유저네임 id : 회원의 기본 키 exp : JWT 만료 시각
서명	헤더와 페이로드를 합친 뒤 서버만 아는 비밀키 (SECRET)로 해시한 값

해시

해시(Hash)는 원본 데이터를 고정된 길이의 값으로 변환하는 단방향 함수입니다. 입력값이 같으면 항상 동일한 결과가 나오지만, 단 한 글자만 달라져도 전혀 다른 결과가 출력됩니다. 또한 결과값만으로는 원본 데이터를 복구할 수 없습니다.

이러한 특성 덕분에 해시는 데이터가 중간에 위변조되지 않았음을 증명하는 데 널리 쓰입니다.

서버는 요청에 실려 온 JWT를 점(.)으로 나눠 헤더와 페이로드를 꺼낸 뒤, 비밀키와 함께 같은 방식으로 해시합니다. 그 결과를 JWT의 서명과 대조해 위변조 여부를 검증합니다.

Base64로 인코딩된 페이로드는 디코딩하면 누구나 읽을 수 있으므로 비밀번호 같은 민감한 정보는 넣지 않습니다. 다만 비밀키가 없으면 서명을 다시 만들 수 없어 내용을 위조할 수 없습니다.

4.3 테이블 관계

인증 기능을 구현하려면 회원 정보를 저장할 테이블이 필요합니다. 테이블을 생성하기 전에, 새로운 회원 데이터가 기존 게시글 테이블과 어떤 관계를 맺는지 알아보겠습니다.

H2와 같은 **관계형 데이터베이스(RDBMS)**는 데이터를 행(Row)과 열(Column)로 구성된 테이블에 저장하며, 테이블을 서로 연결하는 **관계**를 통해 전체 데이터를 관리합니다. 테이블과 테이블 사이의 관계는 **1:1, 1:N, N:M(다대다)** 세 가지입니다.

4.3.1 1:N 관계

먼저 1:N은 **한 테이블의 데이터 하나에 다른 테이블의 데이터 여러 개가 연결되는 구조**입니다. 재고가 단 하나뿐인 제품을 판매한다고 가정해 보겠습니다. 고객 한 명은 **여러 제품을 살 수 있지만**, 특정 제품은 **오직 한 명의 고객에게만 판매됩니다**.

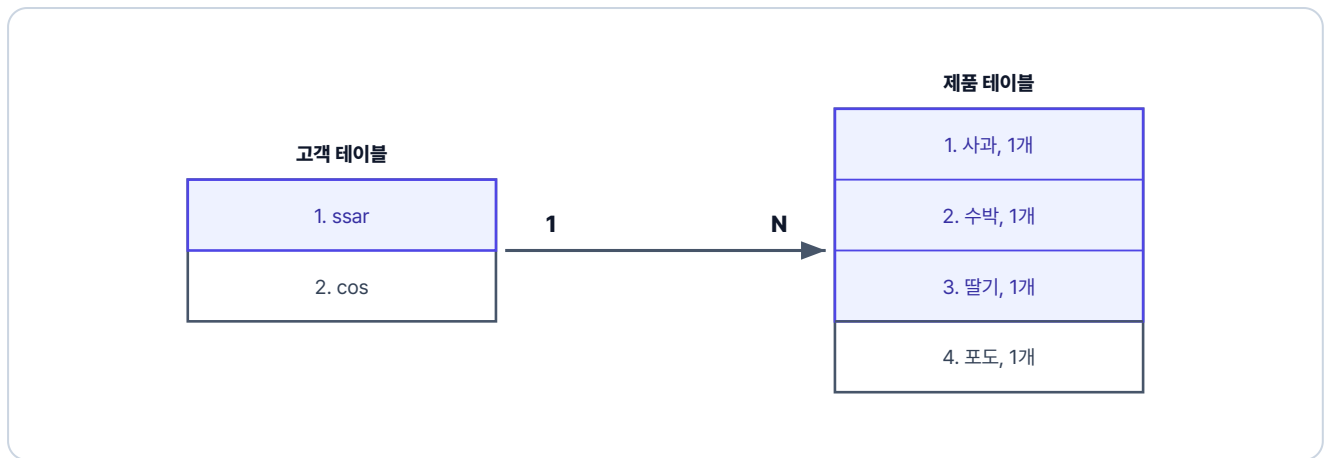


그림 4-8. 1:N 관계

전체 구조로 보면 **소비자(1)** 와 **제품(N)** 은 **1:N 관계**에 해당합니다. 관계형 데이터베이스에서 이 두 테이블의 연관 관계를 맺어주려면 **외래 키(Foreign Key)** 가 필요합니다. 외래 키는 다른 테이블의 기본 키와 연결되는 특수한 컬럼으로, 1:N 관계에서는 **N(다수)** 쪽인 제품 테이블에 외래 키를 정의하여 두 테이블을 연결합니다.

제1정규형

관계형 데이터베이스는 테이블을 체계적으로 구성하기 위해 **정규화(Normalization)** 과정을 거칩니다. 가장 기본이 되는 **제1정규형(First Normal Form)** 은 '하나의 컬럼에 하나의 값만 존재해야 한다'는 규칙입니다. 아래 그림처럼 고객 테이블의 제품 컬럼에 여러 값을 한꺼번에 넣으면 이 원칙에 어긋납니다. 따라서 제품을 별도 테이블로 나누고 외래 키로 연결해 올바른 구조를 만들어야 합니다.

1. ssar	사과, 수박, 딸기
2. cos	포도

그림 4-9. 한 컬럼에 값이 여러 개 들어간 형태

4.3.2 N:M 관계

이번에는 일반적인 상황처럼 제품 테이블에 넉넉한 재고 수량이 있는 경우입니다. 고객 한 명은 여러 제품을 구매할 수 있습니다.

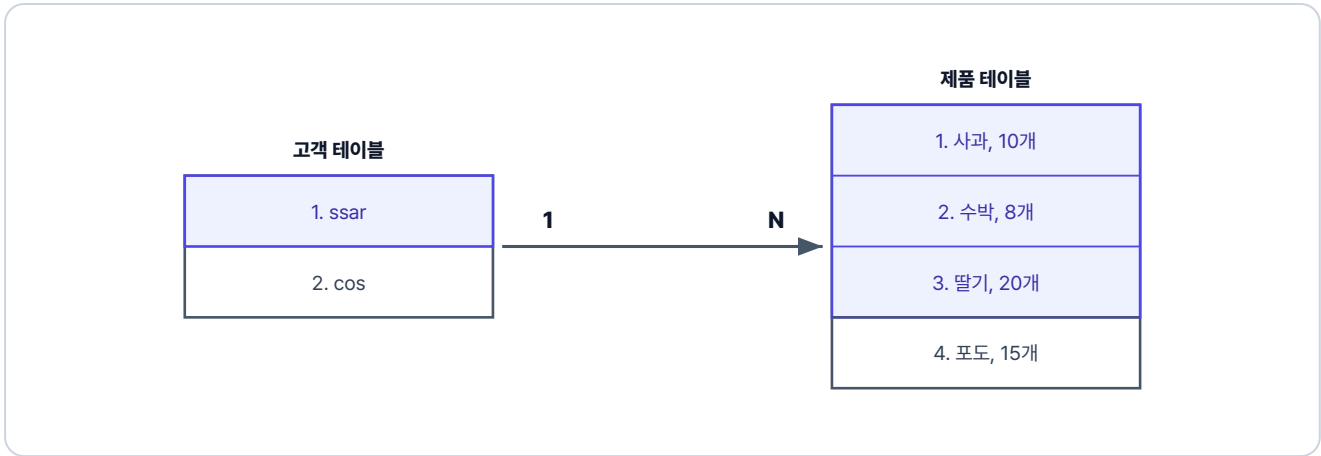


그림 4-10. 고객에서 본 구매 관계

반대로 제품 역시 재고가 있는 만큼 여러 고객이 구매할 수 있습니다.

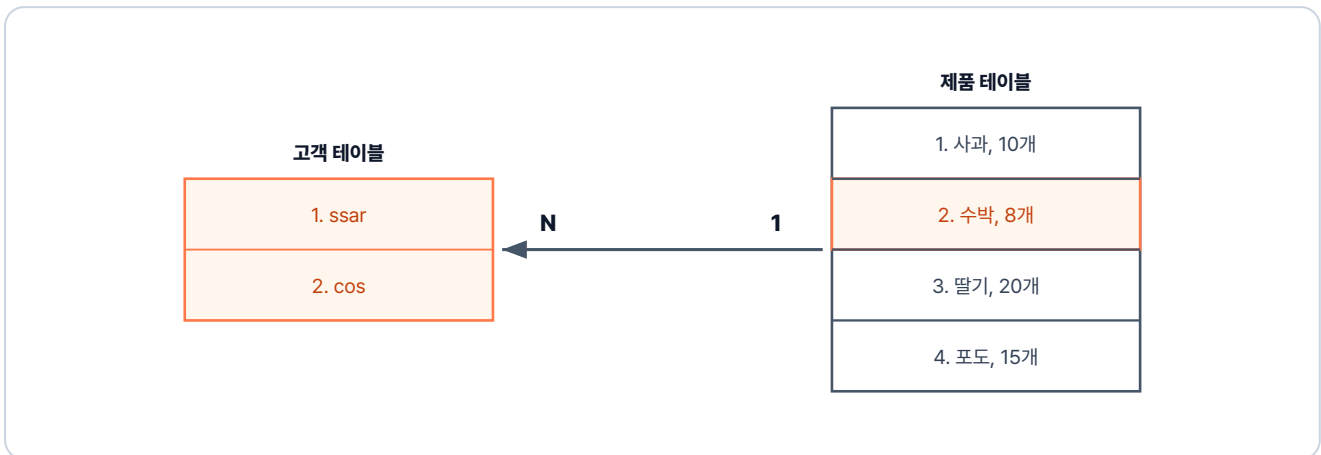
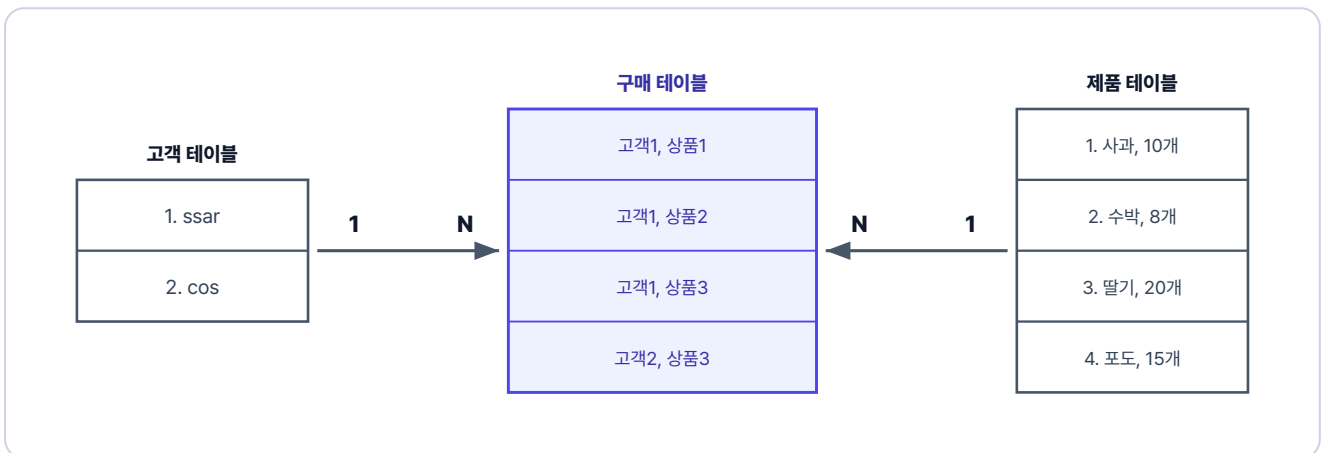


그림 4-11. 제품에서 본 구매 관계

이처럼 양쪽 모두가 여럿과 연결되는 구조를 N:M 관계라고 합니다. 관계형 데이터베이스에서는 N:M 관계를 직접 연결할 수 없으므로, 사이에 **중간 테이블**을 만들어 두 개의 1:N 관계로 나뉘어야 합니다. **구매**라는 중간 테이블을 두면, **고객 한 명이 여러 번 구매하는 구조(1:N)**와 **제품 하나가 여러 고객에게 구매되는 구조(1:N)**로 어떤 고객이 무엇을 구매했는지, 어떤 제품을 누가 구매했는지 조회할 수 있습니다.



회원 테이블과 게시글 테이블의 관계로 살펴보면, 회원 한 명은 여러 개의 게시글을 작성할 수 있고, 하나의 게시글은 작성자가 한 명입니다. 따라서 **회원 테이블과 게시글 테이블은 1:N 관계이고, N에 해당하는 게시글에 외래 키를 추가해야 합니다.**

4.4 테이블 만들기

4.4.1 회원 엔티티

이제 회원 테이블 추가를 위한 엔티티를 만들어 보겠습니다.

`user/User.java`를 열어 아래와 같이 작성합니다.

실습 1 user/User.java. 회원 엔티티

```
@NoArgsConstructor
@Data
@Entity
@Table(name = "user_tb")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(unique = true)
    private String username;
    private String password;
    private String email;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @Builder
    public User(Integer id, String username, String password,
                String email, LocalDateTime createdAt) {
        this.id = id;
        this.username = username;
        this.password = password;
        this.email = email;
        this.createdAt = createdAt;
    }
}
```

```
}
```

`username`에 붙은 `@Column(unique = true)`는 `user_tb`를 생성할 때 `username` 컬럼에 유니크 제약을 추가합니다.

4.4.2 게시글에 작성자 연결

회원 테이블을 만들었으니 이제 게시글 테이블에 회원 테이블을 연결할 외래 키가 필요합니다. 스프링에서는 외래 키 추가를 위해 **@ManyToOne** 어노테이션을 사용합니다.

`board/Board.java`를 열어 작성자 정보를 담을 필드를 아래와 같이 추가합니다.

실습 2 board/Board.java. 작성자 연관관계 추가

```
@ManyToOne
private User user;

@Builder
public Board(Integer id, String title, String content,
              LocalDateTime createdAt, User user) {
    this.id = id;
    this.title = title;
    this.content = content;
    this.createdAt = createdAt;
    this.user = user; // 회원 필드 추가
}
```

데이터베이스에서는 참조할 테이블의 기본 키를 외래 키로 사용합니다. 하지만 JPA에서는 **User** 같은 엔티티를 필드로 선언할 수 있습니다. 이렇게 하면 연관된 회원 데이터를 함께 조회할 수 있다는 장점이 있습니다.

외래 키 컬럼명 생성 규칙

JPA가 만드는 외래 키 컬럼명은 게시글 엔티티에 추가한 필드명(**user**)과 회원 테이블의 기본 키(**id**)를 언더스코어로 합친 **user_id**입니다. 만약 컬럼명을 명시하고 싶다면 `JoinColumn` 어노테이션을 사용해 원하는 컬럼명을 정할 수도 있습니다.

4.4.3 더미 데이터

더미 데이터에 회원을 추가하고, 게시글에는 외래 키 값을 추가합니다.

참고 resources/db/data.sql. 회원과 게시물 더미 데이터

```
insert into user_tb (username, password, email, created_at)
values ('ssar', '1234', 'ssar@metacoding.com', now());
insert into user_tb (username, password, email, created_at)
values ('cos', '1234', 'cos@metacoding.com', now());

insert into board_tb (title, content, created_at, user_id)
values ('title1', 'content1', now(), 1);
insert into board_tb (title, content, created_at, user_id)
values ('title2', 'content2', now(), 2);
```

4.5 core 폴더 살펴보기

로그인한 사용자를 식별하려면 토큰을 발급하고, 이후 요청에 포함된 토큰을 검증하는 로직이 필요합니다.

`core` 폴더에 있는 인증 관련 클래스는 다음과 같습니다.

파일	역할
core/util/JwtUtil.java	회원 정보로 토큰을 생성하고, 토큰을 검증해 회원 정보를 복원합니다
core/util/JwtProvider.java	요청 헤더에서 토큰을 꺼내고, 유효하면 회원 정보를 반환합니다
core/filter/JwtAuthenticationFilter.java	토큰이 유효하면 회원 정보를 요청에 보관합니다

4.5.1 JwtUtil

`JwtUtil`은 로그인한 회원 정보로 JWT를 만들어 서버만 아는 비밀키로 서명합니다. 이후 요청이 오면 같은 비밀키로 서명을 검증해 회원 정보를 복원합니다.

참고 core/util/JwtUtil.java. JWT 발급과 검증

```
public class JwtUtil {

    public static final String HEADER = "Authorization";
    public static final String TOKEN_PREFIX = "Bearer ";
    // 실제 서비스에서는 외부 설정으로 분리한다
    public static final String SECRET = "메타코딩시크릿키";
```

```

public static final Duration EXPIRATION_TIME = Duration.ofDays(7);

// 1. 유저 정보를 담아 토큰을 만든다
public static String create(User user) {
    String jwt = JWT.create()
        .withSubject(user.getUsername())
        .withExpiresAt(Instant.now().plus(EXPIRATION_TIME))
        .withClaim("id", user.getId())
        .sign(Algorithm.HMAC512(SECRET));
    return TOKEN_PREFIX + jwt;
}

// 2. 서명을 확인하고 담긴 정보로 User를 복원한다
public static User verify(String jwt) {
    DecodedJWT decodedJWT = JWT.require(Algorithm.HMAC512(SECRET))
        .build().verify(jwt);
    Integer userId = decodedJWT.getClaim("id").asInt();
    String username = decodedJWT.getSubject();
    return User.builder().id(userId).username(username).build();
}
}

```

`create()` 는 유저 정보를 페이로드에 담고 `SECRET` 으로 서명해 토큰 한 줄을 생성합니다.

`verify()` 는 반대로 서명의 유효 여부를 확인한 뒤 페이로드에서 회원 정보를 꺼냅니다. 서명이 일치하지 않으면 예외가 발생합니다.

JWT를 만들고 검증하는 기능은 **com.auth0:java-jwt** 라이브러리가 제공하며, `build.gradle` 에 추가되어 있습니다.

4.5.2 JwtProvider

JwtProvider는 요청 헤더에서 토큰을 꺼낸 뒤 **JwtUtil**로 검증하고 회원 정보를 반환합니다.

참고 `core/util/JwtProvider.java`. 토큰 추출과 유저 복원

```

@Component
public class JwtProvider {

    // 1. 헤더에서 "Bearer " 접두어를 떼고 토큰만 꺼낸다
    public String resolveToken(HttpServletRequest request) {
        String bearerToken = request.getHeader(JwtUtil.HEADER);
        if (bearerToken != null && bearerToken.startsWith(JwtUtil.TOKEN_PREFIX)) {
            return bearerToken.replace(JwtUtil.TOKEN_PREFIX, "");
        }
        return null;
    }
}

```

```

    }

    // 2. 토큰이 유효하면 User, 아니면 null
    public User getUser(String token) {
        try {
            return JwtUtil.verify(token);
        } catch (Exception e) {
            return null;
        }
    }
}

```

4.5.3 인증 필터

필터(Filter) 는 요청이 컨트롤러에 도달하기 전에 먼저 거쳐 가는 지점입니다. 스프링에서는 **OncePerRequestFilter**를 상속하여 요청 한 건마다 한 번씩 실행되는 필터를 구현합니다.

JwtAuthenticationFilter는 이 지점에서 **JwtProvider**로 회원 정보를 얻어 요청 객체에 저장합니다.

참고 core/filter/JwtAuthenticationFilter.java. 인증 필터

```

@RequiredArgsConstructor
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtProvider jwtProvider;

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain) throws ServletException, IOException {
        // 1. 헤더에서 토큰을 꺼낸다
        String token = jwtProvider.resolveToken(request);
        // 2. 유효하면 회원 정보를 요청에 담아 둔다 (막지는 않는다)
        if (token != null) {
            User loginUser = jwtProvider.getUser(token);
            if (loginUser != null) {
                request.setAttribute("loginUser", loginUser);
            }
        }
        filterChain.doFilter(request, response);
    }
}

```

회원 정보는 `loginUser` 라는 이름으로 요청에 담기며, 컨트롤러는 이 이름으로 꺼냅니다. 토큰이 없거나 서명이 일치하지 않으면 담지 않고 통과시킵니다.

4.6 회원가입

회원가입은 클라이언트가 입력한 회원 정보를 데이터베이스에 저장한 뒤, 저장된 회원 정보를 응답합니다.

4.6.1 리포지토리

먼저 리포지토리를 작성해 보겠습니다. **JpaRepository**를 상속하고, 유저네임으로 회원을 조회하는 메서드를 추가합니다.

`user/UserRepository.java` 를 열어 아래와 같이 작성합니다.

실습 3 `user/UserRepository.java`. 유저네임으로 조회

```
public interface UserRepository extends JpaRepository<User, Integer> {  
  
    Optional<User> findByUsername(String username);  
}
```

메서드 이름과 조회 조건

`findBy` 뒤에 엔티티의 필드 이름을 붙이면 Spring Data JPA가 select 문을 생성합니다. **`findByUsername()`** 은 **`username`**이 일치하는 회원을 조회하며, 매개변수로 받은 값이 그 조건에 들어갑니다.

4.6.2 요청 DTO

다음으로 클라이언트가 입력한 값을 전달받을 DTO를 만들어 보겠습니다.

`user/UserRequest.java` 를 열어 아래와 같이 작성합니다.

실습 4 `user/UserRequest.java`. 회원가입 요청 DTO

```
public class UserRequest {  
  
    public record SaveDTO(String username, String password, String email) {
```

```

    public User toEntity() {
        return User.builder()
            .username(username)
            .password(password)
            .email(email)
            .build();
    }
}

```

SaveDTO는 유저네임과 비밀번호, 이메일을 받습니다. `toEntity()` 는 리포지토리가 저장할 수 있도록 이 값으로 **User** 엔티티를 생성해 반환합니다.

4.6.3 응답 DTO

이어서 회원 정보를 응답할 DTO를 만들어 보겠습니다. 비밀번호를 제외한 컬럼만 필드로 선언합니다.

`user/UserResponse.java` 를 열어 아래와 같이 작성합니다.

실습 5 `user/UserResponse.java`. 회원가입 응답 DTO

```

public class UserResponse {

    public record DTO(
        Integer userId,
        String username,
        String email,
        LocalDateTime createdAt) {

        public DTO(User user) {
            this(
                user.getId(),
                user.getUsername(),
                user.getEmail(),
                user.getCreatedAt());
        }
    }
}

```

4.6.4 서비스

앞에서 만든 리포지토리와 DTO를 사용해 회원가입 메서드를 구현합니다.

`user/UserService.java`를 열어 아래와 같이 작성합니다.

실습 6 user/UserService.java. 회원가입

```
@RequiredArgsConstructor
@Service
public class UserService {

    private final UserRepository userRepository;

    @Transactional
    public UserResponse.DTO 회원가입(UserRequest.SaveDTO requestDTO) {
        if (userRepository.findByUsername(requestDTO.username()).isPresent()) {
            throw new Exception400("이미 존재하는 유저네임입니다");
        }
        User user = userRepository.save(requestDTO.toEntity());
        return new UserResponse.DTO(user);
    }
}
```

실무에서는 비밀번호를 반드시 암호화합니다

비밀번호를 입력받은 그대로 저장하면 데이터베이스에 접근할 수 있는 사람은 누구나 다른 사람의 비밀번호를 알게 됩니다. 그래서 실무에서는 되돌릴 수 없는 해시로 변환해 저장합니다. 이 책은 인증 흐름에 집중하기 위해 비밀번호를 평문으로 다룹니다.

4.6.5 컨트롤러

컨트롤러는 가입 요청을 받아 서비스로 전달합니다.

`user/UserController.java`를 열어 아래와 같이 작성합니다.

실습 7 user/UserController.java. 회원가입 엔드포인트

```
@RequiredArgsConstructor
@RestController
public class UserController {

    private final UserService userService;

    @PostMapping("/join")
    public ResponseEntity<?> join(@RequestBody UserRequest.SaveDTO requestDTO) {
        UserResponse.DTO respDTO = userService.회원가입(requestDTO);
    }
}
```

```

    return Resp.ok(respDT0);
}
}

```

서버를 실행한 뒤 회원가입 API를 호출합니다.

Hoppscotch 회원가입 요청

POST <http://localhost:8080/join>

```

{
  "username": "love",
  "password": "1234",
  "email": "love@metacoding.com"
}

```

The screenshot shows the Hoppscotch interface for a POST request to `http://localhost:8080/join`. The request body is a JSON object: `{ "username": "love", "password": "1234", "email": "love@metacoding.com" }`. The response status is `200 OK` with a time of `954 ms` and size of `308 B`. The response body is a JSON object: `{ "status": 200, "msg": "성공", "body": { "userId": 3, "username": "love", "email": "love@metacoding.com", "createdAt": "2026-09-18T16:12:29.708336" } }`.

그림 4-13. 회원가입 응답

4.7 로그인

로그인은 유저네임과 비밀번호를 받아 회원 일치 여부를 확인하고, 일치하면 JWT를 응답합니다.

4.7.1 요청 DTO

로그인 요청을 담은 DTO를 `user/UserRequest.java`에 아래와 같이 추가합니다.

실습 8 user/UserRequest.java. 로그인 요청 DTO

```
public record LoginDTO(String username, String password) {  
}
```

4.7.2 서비스

전달받은 유저네임과 비밀번호로 회원을 조회해 비밀번호 일치 여부를 확인합니다.

`user/UserService.java`에 아래와 같이 메서드를 추가합니다.

실습 9 user/UserService.java. 로그인

```
public String 로그인(UserRequest.LoginDTO requestDTO) {  
    User user = userRepository.findByUsername(requestDTO.username())  
        .orElseThrow(() -> new Exception401("유저네임 또는 비밀번호가 일치하지 않습  
니다"));  
    if (!user.getPassword().equals(requestDTO.password())) {  
        throw new Exception401("유저네임 또는 비밀번호가 일치하지 않습니다");  
    }  
    return JwtUtil.create(user);  
}
```

유저네임과 비밀번호가 일치하면 `JwtUtil.create()`로 생성한 JWT를 응답합니다.

4.7.3 컨트롤러

컨트롤러는 서비스가 반환한 JWT를 응답합니다.

`user/UserController.java`에 아래와 같이 메서드를 추가합니다.

실습 10 user/UserController.java. 로그인 엔드포인트

```
@PostMapping("/login")  
public ResponseEntity<?> login(@RequestBody UserRequest.LoginDTO requestDTO) {  
    String accessToken = userService.로그인(requestDTO);  
    return Resp.ok(accessToken);  
}
```


4.8.1 JPQL 조인문

이때 사용하는 문법이 JOIN입니다. 조인은 두 테이블의 행을 옆으로 이어 붙여 한 번의 조회로 가져오며, 어떤 행끼리 결합할지는 조건으로 지정합니다. SQL은 그 조건을 `on` 절에 명시하지만, JPQL은 연관관계 필드 이름만 명시합니다.

```
select b from Board b join b.user u
```

Board에 **@ManyToOne**으로 `user` 필드를 추가해 두었으므로, JPQL에서 `b.user` 처럼 필드 이름만으로 회원 엔티티를 조인할 수 있습니다.

조인은 조건이 일치하는 데이터만 가져올 것인지, 아니면 한쪽 테이블을 기준으로 조건이 일치하지 않는 데이터까지 포함할 것인지에 따라 종류가 구분됩니다. JPQL에서 쓰는 것은 두 가지입니다.

조인 종류	기준 테이블	동작 방식 및 조회 결과
(INNER) JOIN	없음	양쪽 테이블 모두에 조건이 일치하는 행만 조회합니다. 조건이 일치하지 않으면 조회 대상에서 제외됩니다
LEFT JOIN	FROM 뒤 테이블	FROM 쪽 테이블의 모든 행을 조회합니다. 상대 테이블에 조건이 일치하는 데이터가 없으면 해당 컬럼은 null로 채웁니다

작성자 정보가 없는 게시글까지 조회하려면 `left join` 을 사용합니다.

```
select b from Board b left join b.user u
```

일반 SQL과 달리 JPQL의 기본 조인은 FROM 절 엔티티만 반환합니다. 게시글의 `user_id` 는 **board_tb**의 컬럼이라 함께 조회되지만, 그 값으로 **User** 엔티티를 채우려면 회원을 조회하는 쿼리가 한 번 더 실행됩니다.

이때 사용하는 것이 **FETCH** 키워드입니다. 이 키워드를 붙이면 JOIN 절 엔티티까지 한 번에 조회해, FROM 절 엔티티의 연관 필드를 실제 데이터로 채워서 반환합니다.

```
select b from Board b join fetch b.user
```

4.8.2 리포지토리

게시글 상세에서 사용할 데이터를 조회해 보겠습니다.

`board/BoardRepository.java`를 열어 **@Query**로 작성자를 함께 조회하는 메서드를 아래와 같이 작성합니다.

실습 11 board/BoardRepository.java. 작성자를 함께 가져오는 조회

```
@Query("select b from Board b join fetch b.user where b.id = :boardId")
Optional<Board> findByIdJoinUser(@Param("boardId") Integer boardId);
```

`findById` 규칙으로는 조인 방식까지 지정할 수 없으므로, JPQL을 직접 적을 때는 **@Query**를 사용합니다.

4.8.3 응답 DTO

`board/BoardResponse.java`를 열어 **DetailDTO**에 작성자의 아이디와 유저네임, 그리고 본인 여부를 알려 주는 값을 아래와 같이 추가합니다.

실습 12 board/BoardResponse.java. 상세 응답에 작성자와 본인 여부 추가

```
public record DetailDTO(
    Integer boardId,
    String title,
    String content,
    Integer userId,
    String username,
    Boolean isOwner) {

    public DetailDTO(Board board, User loginUser) {
        this(
            board.getId(),
            board.getTitle(),
            board.getContent(),
            board.getUser().getId(),
            board.getUser().getUsername(),
            // 비로그인이면 false, 요청자와 작성자가 같으면 true
            loginUser != null
                && loginUser.getId().equals(board.getUser().getId()));
    }
}
```

`isOwner`는 요청자가 이 게시글의 작성자인지를 참과 거짓으로 나타냅니다. 로그인하지 않았으면 `loginUser`가 null이므로 false가 되고, 로그인했다면 요청자의 아이디와 작성자의 아이디를 비교해 일치할 때 true가 됩니다. 클라이언트는 이 값으로 수정·삭제 버튼을 보여줄지 결정하고, 서버는 요청이 들어올 때 401과 403으로 다시 검증합니다.

4.8.4 서비스

작성자를 함께 조회하도록 `board/BoardService.java`의 `게시글상세()`를 아래와 같이 변경합니다.

실습 13 board/BoardService.java. 상세 조회에 fetch 조인과 본인 여부

```
public BoardResponse.DetailDTO 게시글상세(Integer boardId, User loginUser) {
    Board board = boardRepository.findByIdJoinUser(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    return new BoardResponse.DetailDTO(board, loginUser);
}
```

4.8.5 컨트롤러

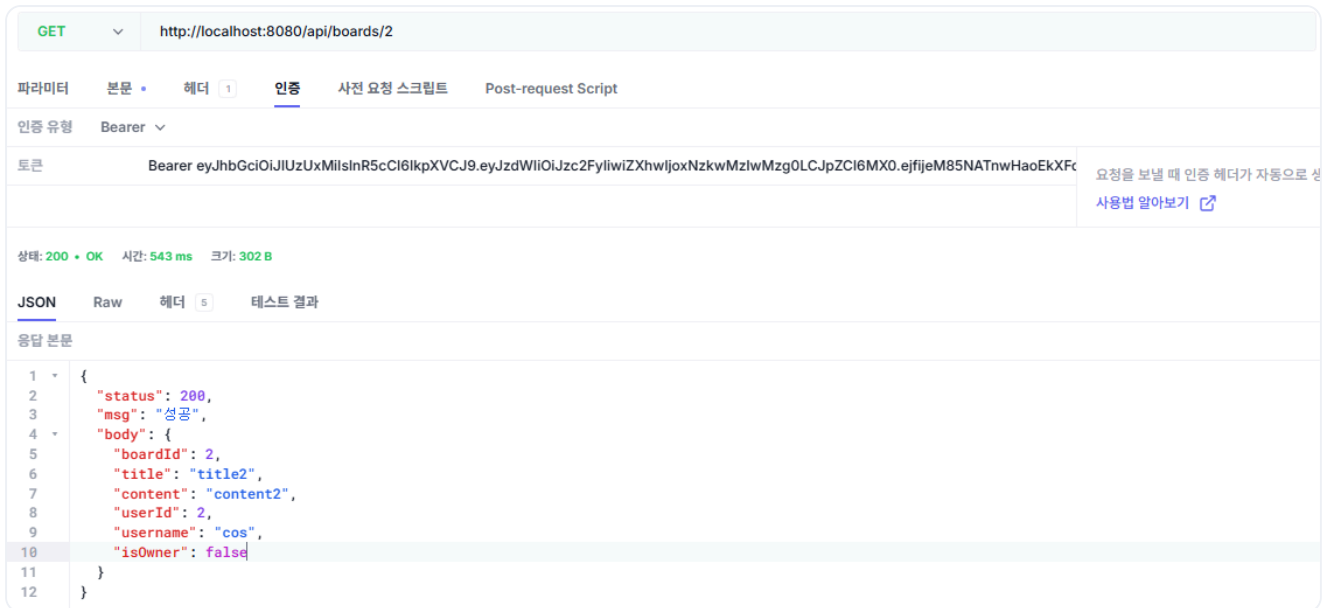
컨트롤러는 필터를 거쳐 요청 객체에 전달된 회원 정보를 꺼내 사용합니다.

`board/BoardController.java`의 상세 메서드를 아래와 같이 변경합니다.

실습 14 board/BoardController.java. 상세 요청에서 로그인 유저 꺼내기

```
@GetMapping("/{boardId}")
public ResponseEntity<?> findById(HttpServletRequest request,
    @PathVariable("boardId") Integer boardId) {
    // 필터가 담아 둔 유저를 꺼낸다 (비로그인이면 null)
    User loginUser = (User) request.getAttribute("loginUser");
    BoardResponse.DetailDTO respDTO = boardService.게시글상세(boardId, loginUser);
    return Resp.ok(respDTO);
}
```

이제 **API 요청 시 JWT를 함께 보내야 합니다.** 인증 탭에서 인증 유형을 Bearer로 선택 후 응답받은 JWT를 추가합니다.



isowner 와 같은 값을 활용해 수정, 삭제와 같은 버튼의 활성화 유무를 제어할 수 있습니다.

게시글을 새로 작성할 때는 로그인한 사용자를 해당 게시글의 작성자로 연결해 저장합니다.

`board/BoardRequest.java`를 열어 `toEntity()`에 `user`를 아래와 같이 추가합니다.

```
public Board toEntity(User user) {  
    return Board.builder()  
        .title(title)  
        .content(content)  
        .user(user)  
        .build();  
}
```

4.9.2 서비스

요청 DTO를 엔티티로 변환해 저장하도록 `board/BoardService.java`의 `게시글추가()`를 아래와 같이 변경합니다.

실습 16 board/BoardService.java. 로그인을 확인하고 작성자를 담아 저장

```
@Transactional
public BoardResponse.DTO 게시글추가(BoardRequest.SaveDTO requestDTO, User loginUser) {
    // 1. 넘어온 유저가 없으면 로그인하지 않은 요청이다
    if (loginUser == null) {
        throw new Exception401("로그인이 필요합니다");
    }
    // 2. 로그인한 사람을 작성자로 연결해 저장한다
    Board savedBoard = boardRepository.save(requestDTO.toEntity(loginUser));
    return new BoardResponse.DTO(savedBoard);
}
```

4.9.3 컨트롤러

컨트롤러는 필터가 담아 둔 유저를 꺼내 서비스로 전달합니다.

`board/BoardController.java`의 추가 메서드를 아래와 같이 변경합니다.

실습 17 board/BoardController.java. 추가 요청에서 로그인 유저 꺼내기

```
@PostMapping
public ResponseEntity<?> save(HttpServletRequest request,
    @RequestBody BoardRequest.SaveDTO requestDTO) {
    // 필터가 담아 둔 유저를 꺼낸다 (비로그인이면 null)
    User loginUser = (User) request.getAttribute("loginUser");
    BoardResponse.DTO respDTO = boardService.게시글추가(requestDTO, loginUser);
    return Resp.ok(respDTO);
}
```

로그인에서 받은 JWT로 게시글 추가 API를 호출합니다.

Hoppscotch 게시글 추가 요청

POST <http://localhost:8080/api/boards>

```
{
  "title": "제목",
  "content": "내용"
}
```

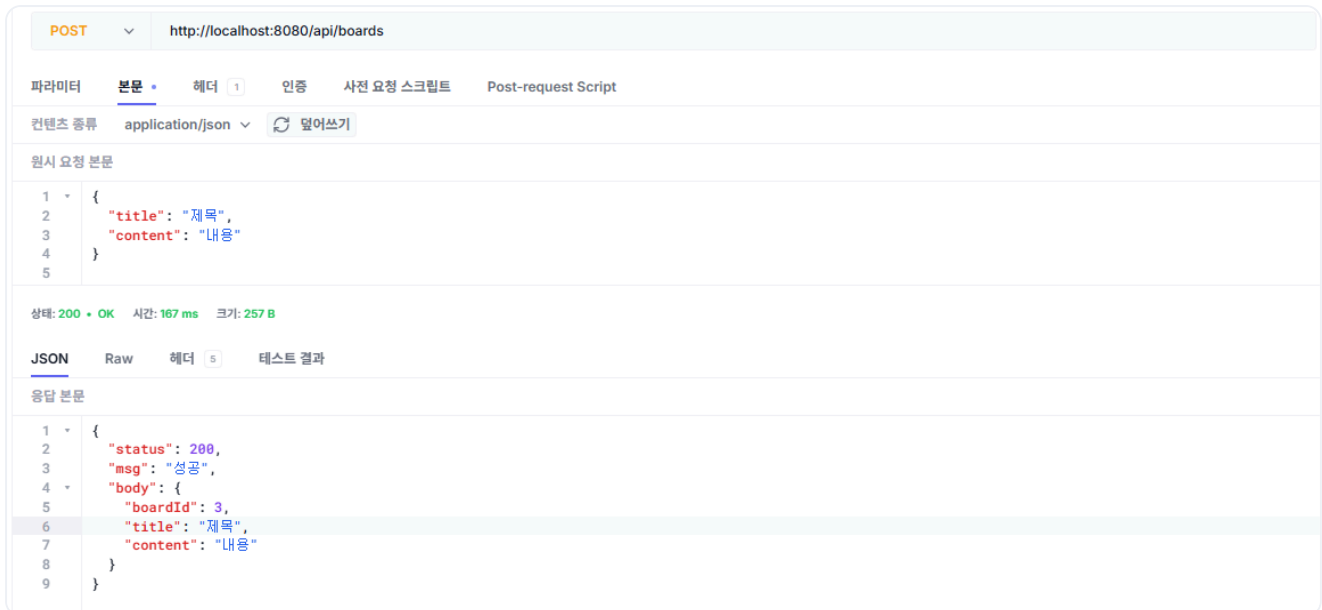


그림 4-18. 게시글 추가 응답

4.10 게시글 수정

4.10.1 서비스

게시글 수정은 현재 사용자와 게시글 작성자가 일치하는지 검증 후, 게시글을 수정해야 합니다. 수정은 동일하게 더티 체크킹으로 진행합니다.

`board/BoardService.java` 의 `게시글수정()` 을 아래와 같이 변경합니다.

실습 18 `board/BoardService.java`. 로그인 확인과 소유자 검증

```
@Transactional
public BoardResponse.DTO 게시글수정(
    Integer boardId, BoardRequest.UpdateDTO requestDTO, User loginUser) {
    // 1. 넘어온 유저가 없으면 로그인하지 않은 요청이다
    if (loginUser == null) {
        throw new Exception401("로그인이 필요합니다");
    }
    Board board = boardRepository.findByIdJoinUser(boardId)
```

```

        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
// 2. 작성자 본인이 아니면 막는다
if (!board.getUser().getId().equals(loginUser.getId())) {
    throw new Exception403("게시글을 수정할 권한이 없습니다");
}
board.setTitle(requestDTO.title());
board.setContent(requestDTO.content());
return new BoardResponse.DTO(board);
}

```

4.10.2 컨트롤러

`board/BoardController.java` 의 수정 메서드를 아래와 같이 변경합니다.

실습 19 board/BoardController.java. 수정 요청에서 로그인 유저 꺼내기

```

@PutMapping("/{boardId}")
public ResponseEntity<?> update(HttpServletRequest request,
    @PathVariable("boardId") Integer boardId,
    @RequestBody BoardRequest.UpdateDTO requestDTO) {
    User loginUser = (User) request.getAttribute("loginUser");
    BoardResponse.DTO respDTO = boardService.게시글수정(boardId, requestDTO, loginU
ser);
    return Resp.ok(respDTO);
}

```

ssar이 작성한 1번 게시글의 수정 API를 호출합니다.

Hoppscotch 게시글 수정 요청

```

PUT http://localhost:8080/api/boards/1

{
  "title": "수정한 제목",
  "content": "수정한 내용"
}

```

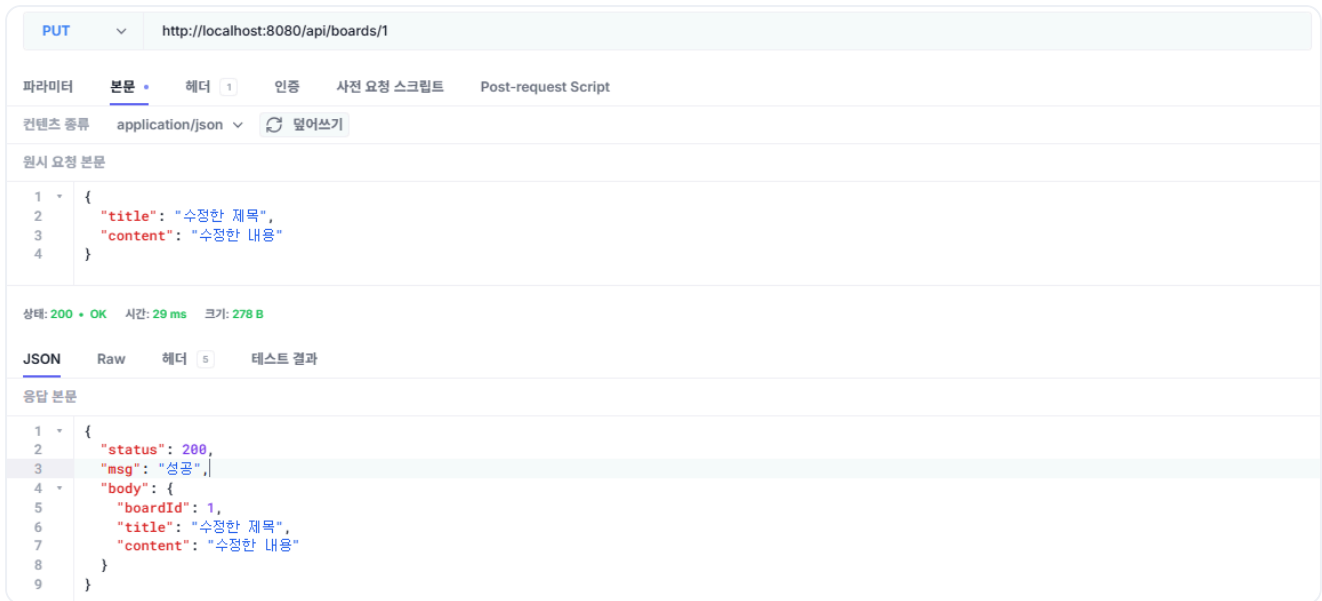


그림 4-19. 게시물 수정 응답

2번 게시물은 작성자가 다르기 때문에 인가에서 실패하여 403 에러가 발생합니다.



그림 4-20. 다른 회원 게시물의 수정 응답

4.11 게시물 삭제

삭제도 대상 게시물의 작성자와 요청자의 일치 여부를 검증합니다. 검증을 통과하면 게시물을 삭제하고, 반환할 데이터가 없으므로 성공만 응답합니다.

4.11.1 서비스

`board/BoardService.java`의 `게시물삭제()`를 아래와 같이 변경합니다.

실습 20 `board/BoardService.java`. 삭제의 로그인 확인과 소유자 검증

```
@Transactional
public void 게시물삭제(Integer boardId, User loginUser) {
    // 1. 넘어온 유저가 없으면 로그인하지 않은 요청이다
    if (loginUser == null) {
        throw new Exception401("로그인이 필요합니다");
    }
    Board board = boardRepository.findByIdJoinUser(boardId)
        .orElseThrow(() -> new Exception404("게시물을 찾을 수 없습니다"));
    // 2. 작성자 본인이 아니면 막는다
    if (!board.getUser().getId().equals(loginUser.getId())) {
        throw new Exception403("게시물을 삭제할 권한이 없습니다");
    }
    boardRepository.delete(board);
}
```

4.11.2 컨트롤러

`board/BoardController.java`의 삭제 메서드를 아래와 같이 변경합니다.

실습 21 `board/BoardController.java`. 삭제 요청에서 로그인 유저 꺼내기

```
@DeleteMapping("/{boardId}")
public ResponseEntity<?> deleteById(HttpServletRequest request,
    @PathVariable("boardId") Integer boardId) {
    User loginUser = (User) request.getAttribute("loginUser");
    boardService.게시물삭제(boardId, loginUser);
    return Resp.ok(null);
}
```

ssar이 작성한 1번 게시물의 삭제 API를 호출합니다.

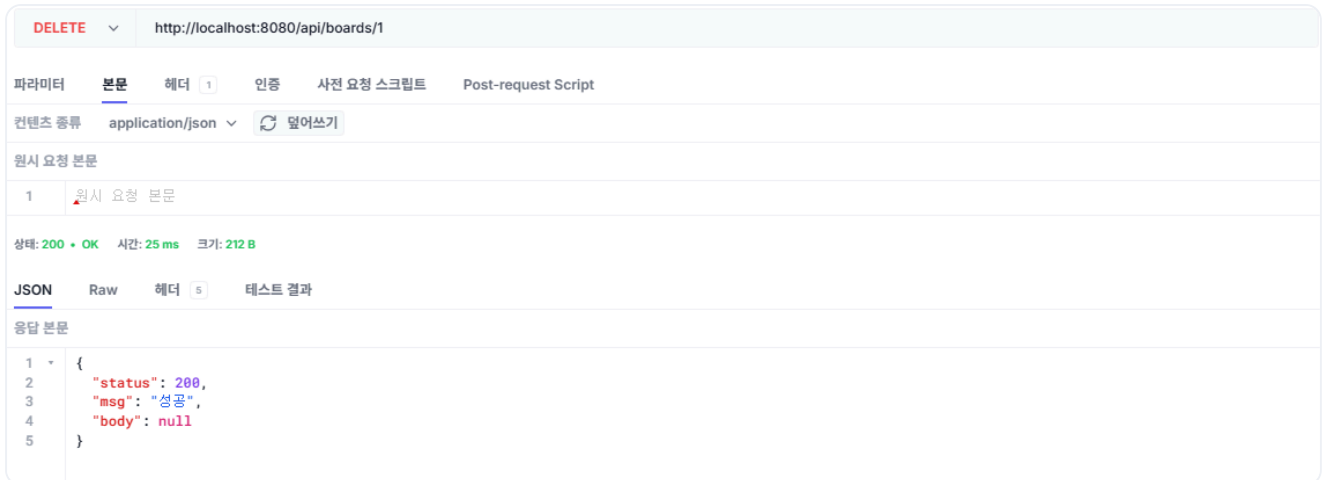
DELETE `http://localhost:8080/api/boards/1`

그림 4-21. 게시글 삭제 응답

이전까지 실습한 게시판은 누가 요청을 보냈는지 알 수 없는 익명 상태였습니다. 하지만 이번 챕터에서는 회원 기능을 추가하여 요청자를 식별하는 구조로 수정하였습니다. 서버가 세션처럼 로그인 상태를 따로 기억할 필요 없이, 요청에 포함된 JWT만으로 회원을 식별할 수 있습니다. 그리고 JWT를 활용해 사용자의 인증과 인가도 함께 구현했습니다.

다음 챕터에서는 게시글에 필요한 댓글과, 이를 위해 회원, 게시글, 댓글 테이블간의 관계를 알아보겠습니다.

이것만은 기억하자

- 인증은 사용자의 신원을 확인하는 과정이고, 인가는 특정 작업을 수행할 권한이 있는지 검증하는 과정입니다.
- 클라이언트는 요청마다 JWT를 함께 전달합니다. JWT에는 회원 정보가 포함되어 있어, 서버는 사용자 정보를 서버에 저장하지 않아도 JWT를 통해 사용자를 식별할 수 있습니다.

CHAPTER

5

댓글과 JPA 심화

연관된 데이터를 함께 조회하다

이번 챕터가 끝나면

- 양방향 매핑으로 게시글과 댓글이 서로를 참조하게 만듭니다.
- 즉시 로딩과 지연 로딩의 차이를 이해합니다.
- `join fetch`로 연관된 데이터를 한 번에 조회합니다.

챕터 5. 댓글과 JPA 심화

오픈이는 인증 게시판에 이어 댓글 기능을 추가하기로 했습니다. 게시글을 조회할 때 여러 개의 댓글을 함께 보여줘야 하고, 게시글과 댓글은 각각 작성자 정보를 가지고 있습니다.

댓글 테이블을 설계하던 오픈이는 머릿속이 복잡해졌습니다.

게시글, 게시글 작성자, 여러 개의 댓글, 그리고 댓글 작성자들까지... 이 많은 데이터를 한 화면에 다 보여주려면 어떻게 가져와야 하지?

오픈이는 선배를 찾아갔습니다.

오픈이 : "선배님, 게시글 상세 화면에 댓글까지 추가되니까 데이터를 어떻게 가져와야 할지 막막하네요."

선배 : "실무에서는 크게 두 가지 방법을 써요. **첫 번째는 쿼리를 두 번으로 나누는 겁니다.** 게시글과 게시글 작성자를 먼저 조회하고, 그다음 이 게시글에 있는 댓글과 댓글 작성자 번호를 추가로 가져오는 거죠. 그리고 두 데이터를 DTO에 담아 합치는 방식이에요. 구조가 단순해서 실무에서도 많이 씁니다."

오픈이 : "그럼 두 번째 방법은요?"

선배 : "**JPA의 양방향 매핑을 쓰는 거예요.** 데이터베이스 테이블은 외래 키 하나로 양방향 조인이 가능하지만, 자바 객체는 참조하는 쪽으로만 갈 수 있는 일방통행이거든요. 그래서 양방향 매핑으로 게시글 객체 쪽에서도 댓글을 참조할 수 있게 열어주는 거예요. 이렇게 해주면 JPA에서 조인을 통해 한 번에 데이터를 가져올 수 있어요."

소스코드 준비

소스코드 준비에서 클론한 예제 저장소에서 이번 챕터 폴더로 이동합니다. 패키지 루트는 챕터 4와 같은 `com.metacoding.spring` 입니다.

터미널 챕터 5 폴더로 이동

```
cd spring-start/ch05
```

이번 챕터에서 새로 만들거나 고치는 파일은 다음과 같습니다.

ch05 파일 구조

```
spring-start/ch05/src/main/java/com/metacoding/spring/
├─ board/
│   ├── Board.java                # [작성] @OneToMany replies, 작성자 지
연 로딩
│   ├── BoardRepository.java      # [작성] findByIdJoinUserAndReplies
│   ├── BoardResponse.java        # [작성] DetailDTO에 replies
│   └── BoardService.java         # [작성] 상세 조회를 join fetch로 교체
├─ reply/
│   ├── Reply.java               # [작성] 댓글 엔티티(@ManyToOne user/bo
ard)
│   ├── ReplyController.java      # [작성] 작성·삭제 엔드포인트
│   ├── ReplyRepository.java      # [작성] JpaRepository 상속으로 저장·조
회·삭제
│   ├── ReplyRequest.java        # [작성] 댓글 요청 DTO
│   ├── ReplyResponse.java       # [작성] 댓글 응답 DTO
│   └── ReplyService.java         # [작성] 댓글 저장·삭제(소유자 검증)
spring-start/ch05/src/main/resources/
├─ db/data.sql                   # [참고] 댓글 더미 데이터
```

챕터를 따라 코드를 채우고, 막히면 `spring-end`의 완성 코드를 참고하세요.

5.1 양방향 매핑

먼저 게시글과 댓글의 관계를 알아보겠습니다. 하나의 게시글에는 여러 개의 댓글이 달릴 수 있습니다. 그리고 하나의 댓글은 하나의 게시글에만 속하므로, **게시글과 댓글은 1:N 관계입니다.**

관계형 데이터베이스에서는 외래 키가 어느 쪽에 있든 두 테이블을 조인하여 연관된 데이터를 가져올 수 있습니다. 예를 들어 외래 키가 댓글 테이블에 있으면, 조인으로 게시글과 댓글 데이터를 한 번에 조회할 수 있습니다.

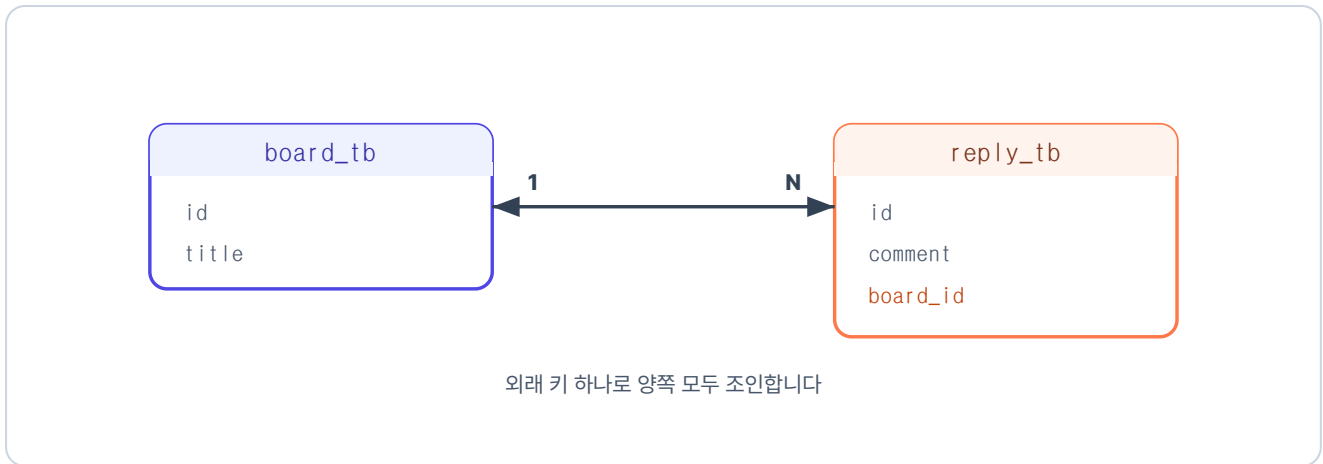


그림 5-1. 테이블의 1:N 관계

반면 자바 객체의 참조는 한 방향으로만 이루어집니다. **Reply** 객체에 **Board** 타입 필드를 두면 댓글에서 게시글을 참조할 수는 있지만, 반대로 **Board** 객체에는 댓글을 가리키는 필드가 없어 게시글에서 댓글에 접근할 수는 없습니다.

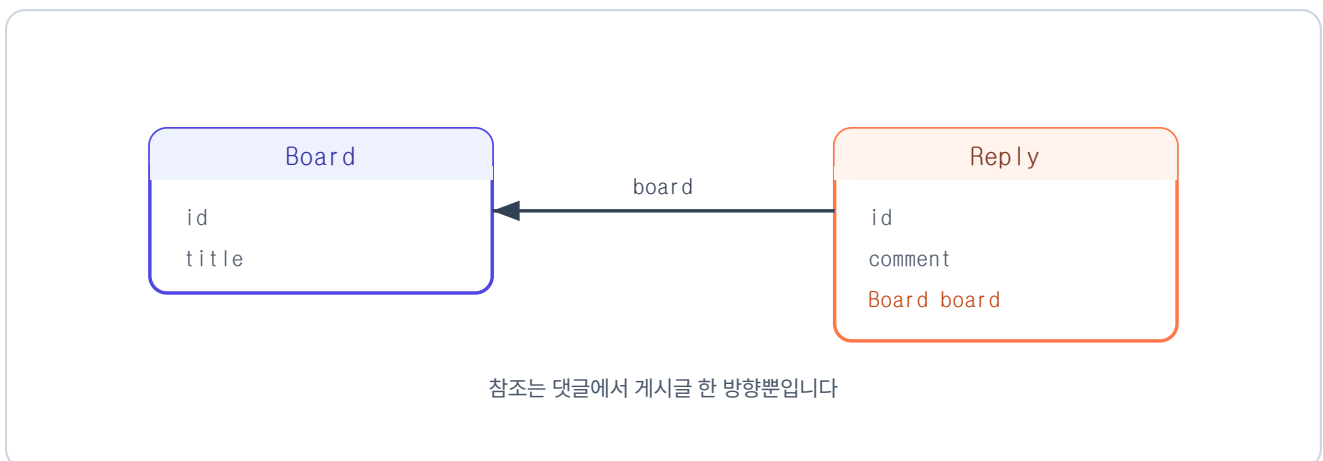


그림 5-2. 자바 객체의 한 방향 참조

따라서 객체에서도 반대 방향으로 접근하려면 반대쪽 객체에도 참조를 위한 필드를 추가해야 합니다. JPA에서는 이런 방식을 **양방향 매핑(Bidirectional Mapping)** 이라고 합니다. 외래 키 참조를 위해 사용된 **@ManyToOne**과 짝을 이루도록, 반대편 객체에 **@OneToMany**를 추가하여 서로를 참조하게 만드는 방식입니다.

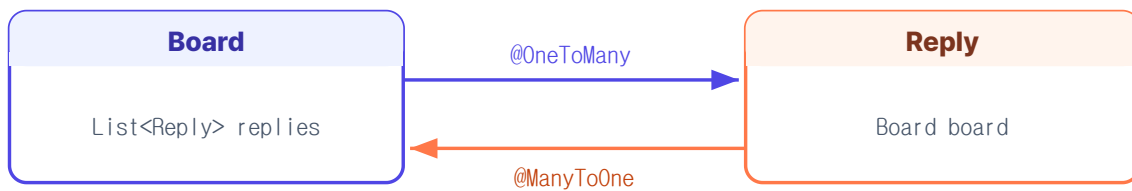


그림 5-3. 양방향 매핑

5.1.1 댓글 엔티티

먼저 댓글 엔티티부터 정의합니다. 댓글 작성자 정보와 댓글이 포함된 게시글 정보를 위해 **User** 엔티티와 **Board** 엔티티를 필드로 추가합니다.

`reply/Reply.java` 를 열어 아래와 같이 작성합니다.

실습 1 reply/Reply.java. 댓글 엔티티

```

@NoArgsConstructor
@Data
@Entity
@Table(name = "reply_tb")
public class Reply {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    private String comment;

    // 외래 키 필드
    @ManyToOne(fetch = FetchType.LAZY)
    private User user;

    // 외래 키 필드
    @ManyToOne(fetch = FetchType.LAZY)
    private Board board;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @Builder
    public Reply(Integer id, String comment, User user, Board board,
        LocalDateTime createdAt) {
        this.id = id;
    }
  
```

```

        this.comment = comment;
        this.user = user;
        this.board = board;
        this.createdAt = createdAt;
    }
}

```

5.1.2 게시물에 댓글 목록 추가

양방향 매핑을 위해 **Board** 엔티티에 댓글 목록을 필드로 추가합니다.

양방향 참조를 설정할 때는 `mappedBy` 속성을 사용합니다. 한쪽 어노테이션에 `mappedBy`를 설정하고 상대방의 필드 이름을 명시해 주지만 하면 두 객체의 매핑이 연결됩니다.

`board/Board.java`를 열어 아래와 같이 작성자 필드를 변경하고 댓글 목록 필드를 추가합니다.

실습 2 board/Board.java. 댓글 목록 추가와 작성자 필드 수정

```

@ManyToOne(fetch = FetchType.LAZY)
private User user;

@OneToMany(mappedBy = "board", fetch = FetchType.LAZY, cascade = CascadeType.REMOVE)
private List<Reply> replies = new ArrayList<>();

```

`cascade = CascadeType.REMOVE`를 설정하면 게시물을 삭제할 때 연관된 댓글도 함께 삭제됩니다. 두 필드에 함께 적은 `fetch`는 연관된 데이터를 언제 가져올지 정하는 속성으로, 다음 절에서 이어서 설명합니다.

5.1.3 더미 데이터

더미 데이터에 댓글을 추가합니다.

참고 resources/db/data.sql. 댓글 더미 데이터

```

insert into reply_tb(comment,board_id,user_id,created_at) values('comment1',1,1,now());
insert into reply_tb(comment,board_id,user_id,created_at) values('comment2',1,1,now());
insert into reply_tb(comment,board_id,user_id,created_at) values('comment3',1,2,now());

```

```
insert into reply_tb(comment,board_id,user_id,created_at) values('comment4',2,2,now
());
```

5.2 즉시 로딩과 지연 로딩

JPA로 개발할 때의 대표적인 성능 문제는 당장 필요 없는 연관 데이터까지 함께 조회할 때 발생합니다. 회원 정보가 필요하지 않은 게시글 목록 화면을 예로 들어 보겠습니다. `findAll()` 로 게시글 전체를 조회할 때 JPA가 연관된 회원 엔티티까지 가져온다면, 게시글 수만큼 회원 조회 쿼리가 추가로 실행됩니다. 결국 쓰지 않는 데이터 때문에 서버와 데이터베이스에 부하를 줍니다.

이러한 성능 저하를 막기 위해 JPA는 연관된 데이터를 **조회하는 시점에 한 번에 가져올지**, 아니면 **실제로 사용하는 시점에 조회할지**를 직접 지정할 수 있습니다.

5.2.1 즉시 로딩

즉시 로딩(Eager Loading) 은 특정 엔티티를 조회할 때 **연관된 엔티티의 데이터까지 한 번에 가져오는 방식**으로, **@ManyToOne** 어노테이션은 즉시 로딩이 기본 전략입니다. 그래서 속성을 별도로 지정하지 않으면, 게시글을 조회하는 순간 JPA가 회원 정보까지 조회해 **Board**의 `user` 필드에 채워 넣습니다.

이러한 설정 때문에 `findAll()` 로 게시글 목록을 조회하면 문제가 발생합니다. JPA가 먼저 **board_tb**에서 게시글 목록을 읽어온 뒤, 각 게시글의 비어 있는 회원 정보를 채우기 위해 **user_tb**를 조회하는 쿼리를 따로 실행하기 때문입니다.

실행 결과

Hibernate:

```
select b1_0.id, b1_0.content, b1_0.created_at, b1_0.title, b1_0.user_id
from board_tb b1_0
```

Hibernate:

```
select u1_0.id, u1_0.created_at, u1_0.email, u1_0.password, u1_0.username
from user_tb u1_0
where u1_0.id=?          ← 1번 게시글의 작성자(ssar)
```

Hibernate:

```
select u1_0.id, u1_0.created_at, u1_0.email, u1_0.password, u1_0.username
from user_tb u1_0
where u1_0.id=?          ← 2번 게시글의 작성자(cos)
```

그림 5-4. 즉시 로딩으로 목록을 조회할 때의 쿼리

5.2.2 지연 로딩

이러한 문제를 해결하기 위해 사용하는 방식이 **지연 로딩(Lazy Loading)** 입니다.

지연 로딩은 **기준이 되는 엔티티만 먼저 가져오고, 연관된 엔티티는 실제로 데이터에 접근하는 순간에 조회하는 방식**입니다. **@ManyToOne**의 `fetch` 속성을 LAZY로 설정하면 지연 로딩이 적용됩니다.

board/Board.java. 작성자 조회를 지연 로딩으로

```
@ManyToOne(fetch = FetchType.LAZY)
private User user;
```

LAZY로 설정하면 JPA는 게시글을 조회할 때 회원 엔티티 대신 외래 키(`user_id`)만 가져옵니다. 그리고 회원 정보가 필요한 시점에 외래 키를 사용해 쿼리를 실행합니다.

실행 결과

Hibernate:

```
select b1_0.id, b1_0.content, b1_0.created_at, b1_0.title, b1_0.user_id
from board_tb b1_0
```

그림 5-5. 지연 로딩으로 목록을 조회할 때의 쿼리

다만 지연 로딩은 연관된 엔티티에 접근할 때마다 조회 쿼리를 실행합니다. 연관된 엔티티를 함께 사용하는 조회에서는 접근하는 횟수만큼 쿼리가 추가로 실행되므로, 즉시 로딩과 마찬가지로 쿼리 수가 늘어납니다.

실무에서는 어떤 로딩 방식을 쓰는가

연관된 데이터를 가져올 때 추가 쿼리가 계속해서 발생하는 문제를 방지하기 위해, 실무에서는 가급적 모든 연관관계를 지연 로딩으로 설정합니다. 두 엔티티를 항상 함께 사용하는 경우에 한해 즉시 로딩을 고려할 수 있지만, 기본적으로는 지연 로딩을 유지하면서 필요한 경우에만 쿼리(join fetch)를 통해 한 번에 가져오는 방식이 더 안전합니다. JPA에서 **@ManyToOne**의 기본 전략은 즉시 로딩(EAGER)이며, **@OneToMany**는 지연 로딩(LAZY)입니다.

5.3 댓글 목록

이제 게시물 상세 화면에 댓글 목록을 추가해 보겠습니다.

게시글 상세 화면에서는 게시물과 게시물 작성자 정보, 그리고 댓글과 댓글 작성자 정보를 함께 가져와야 합니다. 이때 게시글의 작성자는 항상 존재하므로 INNER JOIN을 사용합니다. 반면 댓글도 INNER JOIN을 사용하면 댓글이 없는 게시글은 조회되지 않으므로, LEFT OUTER JOIN을 사용해야 합니다.

5.3.1 리포지토리

댓글 목록을 추가로 가져오기 위해 쿼리를 만들어 보겠습니다.

`board/BoardRepository.java`를 열어 아래와 같이 메서드를 작성합니다.

실습 3 `board/BoardRepository.java`. 작성자와 댓글을 함께 가져오는 조회

```
@Query("select b from Board b join fetch b.user left join fetch b.replies r "
        + "left join fetch r.user where b.id = :boardId")
Optional<Board> findByIdJoinUserAndReplies(@Param("boardId") Integer boardId);
```

게시글 상세 화면의 응답 데이터에는 댓글 목록과 각 댓글 작성자 정보가 모두 포함되어야 합니다. 만약 fetch 없이 일반 조인만 사용하면 댓글과 작성자 정보는 지연 로딩으로 조회되어, 댓글과 작성자 정보를 실제로 사용하는 시점에 추가 쿼리가 실행됩니다. 따라서 댓글과 댓글 작성자 정보도 join fetch를 사용하여 게시물과 함께 한 번에 가져옵니다.

5.3.2 응답 DTO

다음으로 데이터베이스에서 조회된 결과를 담을 수 있도록 DTO를 수정해 보겠습니다. 하나의 게시물에는 여러 개의 댓글이 있을 수 있으므로 DTO 내부에 List 타입의 DTO 필드를 추가하여 하나씩 담습니다.

`board/BoardResponse.java`의 **DetailDTO**를 아래와 같이 변경합니다.

실습 4 `board/BoardResponse.java`. 상세에 댓글 목록 추가

```
public record DetailDTO(
    Integer boardId,
    String title,
    String content,
```

```

        Integer userId,
        String username,
        Boolean isOwner,
        // List 타입의 댓글 DTO 필드
        List<ReplyDTO> replies) {

    public DetailDTO(Board board, User loginUser) {
        this(
            board.getId(),
            board.getTitle(),
            board.getContent(),
            board.getUser().getId(),
            board.getUser().getUsername(),
            // 비로그인이면 false, 요청자와 작성자가 같으면 true
            loginUser != null
                && loginUser.getId().equals(board.getUser().getId()),
            // 댓글 엔티티를 하나씩 ReplyDTO로 변환해 List에 담기
            board.getReplies().stream()
                .map(reply -> new ReplyDTO(reply, loginUser))
                .toList());
    }

    // List 타입의 댓글을 담는 DTO
    public record ReplyDTO(
        Integer replyId,
        String username,
        String comment,
        Boolean isOwner) {

        public ReplyDTO(Reply reply, User loginUser) {
            this(
                reply.getId(),
                reply.getUser().getUsername(),
                reply.getComment(),
                // 비로그인이면 false, 요청자와 작성자가 같으면 true
                loginUser != null
                    && loginUser.getId().equals(reply.getUser().getId()));
        }
    }
}

```

5.3.3 서비스

BoardService의 상세 메서드는 `findByIdJoinUser()` 대신

`findByIdJoinUserAndReplies()` 를 호출합니다.

`board/BoardService.java` 를 아래와 같이 변경합니다.

실습 5**board/BoardService.java. 상세 조회를 join fetch로 교체**

```
public BoardResponse.DetailDTO 게시글상세(Integer boardId, User loginUser) {
    Board board = boardRepository.findByIdJoinUserAndReplies(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    return new BoardResponse.DetailDTO(board, loginUser);
}
```

ssar로 로그인해 게시글 상세 API를 호출하면 결과를 확인할 수 있습니다.

Hoppscotch**게시글 상세 조회**

GET <http://localhost:8080/api/boards/1>

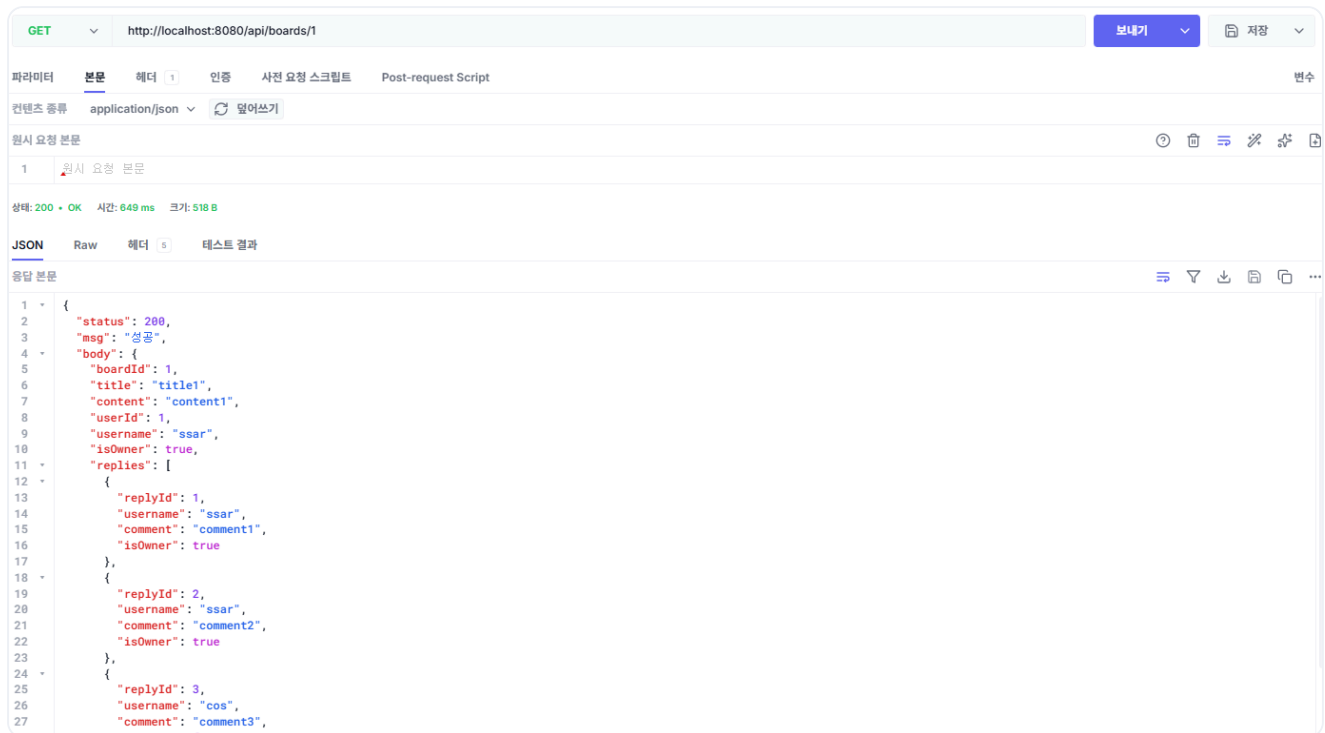


그림 5-6. 댓글이 담긴 상세 응답

5.4 댓글 추가

5.4.1 리포지토리

댓글 추가 기능을 위해 Repository를 구현해야 합니다.

`reply/ReplyRepository.java` 를 열어 아래와 같이 작성합니다.

실습 6 `reply/ReplyRepository.java`. `JpaRepository` 상속

```
public interface ReplyRepository extends JpaRepository<Reply, Integer> {  
}
```

5.4.2 요청 DTO

SaveDTO는 댓글 내용과 대상 게시글의 번호를 받고, 작성자 정보는 필터가 JWT를 확인해 요청 객체에 담아 둔 로그인 회원 정보를 사용합니다.

`reply/ReplyRequest.java` 를 열어 아래와 같이 작성합니다.

실습 7 `reply/ReplyRequest.java`. 댓글 요청 DTO

```
public class ReplyRequest {  
  
    public record SaveDTO(String comment, Integer boardId) {  
  
        public Reply toEntity(User user, Board board) {  
            return Reply.builder()  
                .comment(comment)  
                .user(user)  
                .board(board)  
                .build();  
        }  
    }  
}
```

5.4.3 응답 DTO

댓글을 추가한 결과를 응답으로 반환할 **DTO**도 정의합니다.

`reply/ReplyResponse.java` 를 열어 아래와 같이 작성합니다.

실습 8 `reply/ReplyResponse.java`. 댓글 응답 DTO

```
public class ReplyResponse {  
  
    public record DTO(Integer replyId, String comment, String username) {  
    }  
}
```

```

    public DTO(Reply reply) {
        this(
            reply.getId(),
            reply.getComment(),
            reply.getUser().getUsername());
    }
}

```

5.4.4 서비스

서비스는 요청으로 받은 게시글 번호로 게시글을 조회합니다. 그리고 조회한 게시글과 로그인 회원 정보로 댓글 엔티티를 생성해 저장합니다.

`reply/ReplyService.java` 를 열어 아래와 같이 작성합니다.

실습 9 `reply/ReplyService.java`. 댓글 저장

```

@RequiredArgsConstructor
@Service
public class ReplyService {

    private final ReplyRepository replyRepository;
    private final BoardRepository boardRepository;

    @Transactional
    public ReplyResponse.DTO 댓글추가(ReplyRequest.SavedDTO requestDTO, User loginUser)
    {
        // 1. 넘어온 유저가 없으면 로그인하지 않은 요청이다
        if (loginUser == null) {
            throw new Exception401("로그인이 필요합니다");
        }
        // 2. 댓글을 달 게시글을 찾아 연결한다
        Board board = boardRepository.findById(requestDTO.boardId())
            .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
        Reply savedReply = replyRepository.save(requestDTO.toEntity(loginUser, board));
        return new ReplyResponse.DTO(savedReply);
    }
}

```

5.4.5 컨트롤러

마지막으로 댓글 작성 엔드포인트를 구현합니다.

`reply/ReplyController.java` 를 열어 아래와 같이 작성합니다.

실습 10 `reply/ReplyController.java`. 댓글 작성 엔드포인트

```
@RequiredArgsConstructor
@RestController
@RequestMapping("/api/replies")
public class ReplyController {

    private final ReplyService replyService;

    @PostMapping
    public ResponseEntity<?> save(HttpServletRequest request,
        @RequestBody ReplyRequest.SaveDTO requestDTO) {
        User loginUser = (User) request.getAttribute("loginUser");
        ReplyResponse.DTO respDTO = replyService.댓글추가(requestDTO, loginUser);
        return Resp.ok(respDTO);
    }
}
```

ssar로 로그인해 1번 게시글에 댓글을 답니다.

Hoppscotch 댓글 작성

```
POST http://localhost:8080/api/replies

{
  "comment": "comment5",
  "boardId": 1
}
```

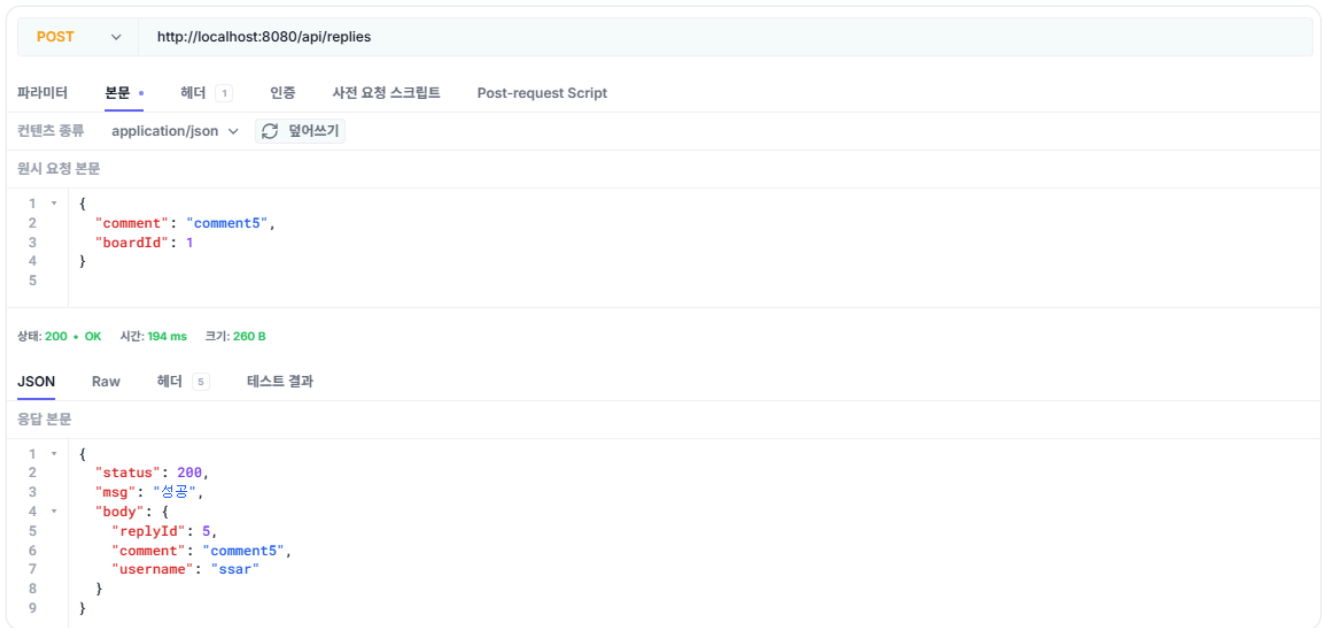


그림 5-7. 댓글 추가 응답

5.5 댓글 삭제

댓글 삭제는 로그인 여부를 확인하고, 댓글 작성자와 요청자가 같은지 검증한 뒤 삭제합니다. 반환할 데이터가 없으므로 성공만 응답합니다.

5.5.1 서비스

`reply/ReplyService.java`에 아래와 같이 메서드를 추가합니다.

실습 11 reply/ReplyService.java. 댓글 삭제

```
@Transactional
public void 댓글삭제(Integer replyId, User loginUser) {
    // 1. 넘어온 유저가 없으면 로그인하지 않은 요청이다
    if (loginUser == null) {
        throw new Exception401("로그인이 필요합니다");
    }
    Reply reply = replyRepository.findById(replyId)
        .orElseThrow(() -> new Exception404("댓글을 찾을 수 없습니다"));
    // 2. 작성자 본인이 아니면 막는다
    if (!reply.getUser().getId().equals(loginUser.getId())) {
        throw new Exception403("댓글을 삭제할 권한이 없습니다");
    }
    replyRepository.delete(reply);
}
```

```
}
```

5.5.2 컨트롤러

`reply/ReplyController.java`에 아래와 같이 메서드를 추가합니다.

실습 12 `reply/ReplyController.java`. 댓글 삭제 엔드포인트

```
@DeleteMapping("/{replyId}")
public ResponseEntity<?> deleteById(
    HttpServletRequest request, @PathVariable("replyId") Integer replyId) {
    User loginUser = (User) request.getAttribute("loginUser");
    replyService.댓글삭제(replyId, loginUser);
    return Resp.ok(null);
}
```

ssar로 로그인해 자신이 쓴 1번 댓글에 삭제를 요청합니다.

Hoppscotch 댓글 삭제

DELETE <http://localhost:8080/api/replies/1>

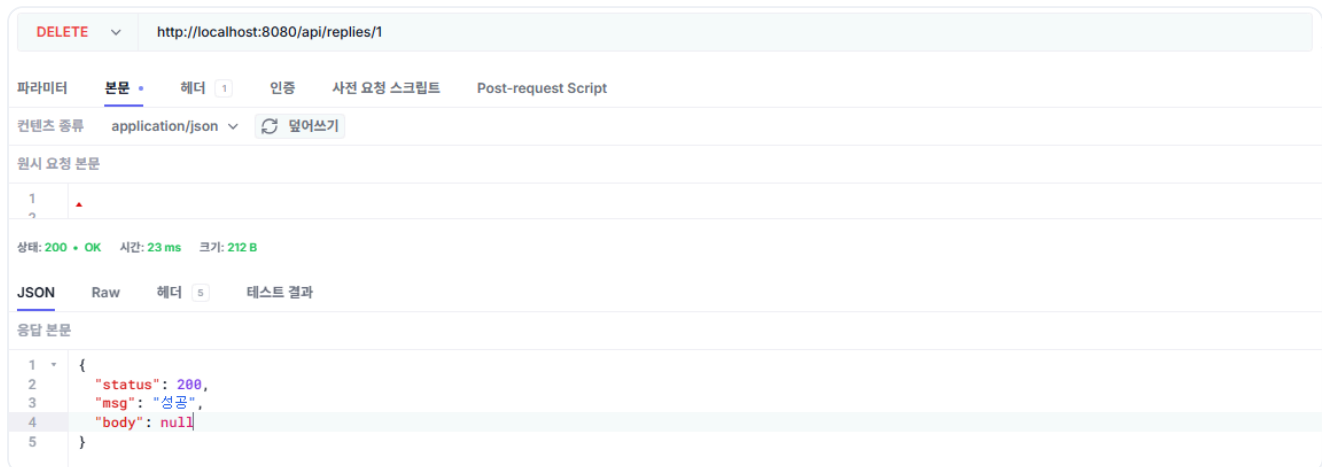


그림 5-8. 댓글 삭제 응답

댓글 기능을 마지막으로 게시판 API에 필요한 기능을 모두 구현했습니다. 컨트롤러와 서비스, 리포지토리가 요청을 계층별로 처리하고, DTO가 요청과 응답의 데이터 형식을 정의하며, 전역 예외 처리가 실패 사유를 상태 코드와 함께 반환합니다. 그리고 필터가 JWT로 요청자를 식별하므로, 게시글과 댓글은 작성자 본인만 변경할 수 있습니다.

이것만은 기억하자

- 객체는 참조 필드가 있어야 접근할 수 있으므로 양방향 매핑을 설정합니다. 테이블은 외래 키 하나로 양쪽을 조인하지만, 객체는 게시글 쪽에 댓글 필드를 추가해야 게시글에서 댓글을 참조할 수 있습니다.
- 연관된 엔티티를 함께 조회할 때는 조인을 활용해 쿼리를 최적화합니다. 불필요한 데이터 조회를 막기 위해 평소에는 지연 로딩을 기본으로 하고, 게시글과 댓글을 동시에 화면에 반환해야 할 때만 조인으로 묶어서 한 번에 가져옵니다.
- 자식 데이터가 존재하지 않을 수 있는 관계는 **LEFT OUTER JOIN**을 사용합니다. INNER JOIN을 쓰면 댓글이 없는 게시글은 조회 결과에서 누락됩니다. 따라서 외부 조인으로 가져온 뒤, 게시글 응답 객체 내부에 댓글 목록을 중첩하여 반환합니다.

에필로그

기나긴 학기가 끝났습니다.

학기 초의 오픈이었다면 제출 마감일이 코앞인 지금까지도 과제의 뼈대만 만들고 있었을 겁니다. 하지만 완성된 프로젝트에는 그 흔적이 없습니다. 뼈대는 스프링에 맡기고, 오픈이는 게시판 기능을 만드는 데만 시간을 썼습니다. 로그인한 사람만 자기가 쓴 글을 고치고 지울 수 있고, 없는 번호를 요청하면 왜 실패했는지가 응답에 담겨 돌아옵니다.

과제를 제출하고 강의실을 나서는 길, 복도에서 선배와 마주쳤습니다.

선배 : "어떻게, 학기 안에 무사히 끝냈나요?"

오픈이 : "네, 스프링 덕분에 쉽게 구현할 수 있었어요."

다음에 만들 프로젝트는 이번 과제보다 훨씬 어려울 겁니다. 하지만 걱정은 없습니다. 상상하던 기능을 실제로 동작하는 코드로 처음부터 끝까지 완성해 본 경험이 남았습니다.

건물을 나서는 오픈이의 발걸음이 가볍습니다. 가방 안에는 방학 동안 만들어 보고 싶은 아이디어를 적어 둔 수첩이 들어 있습니다.